

HelixTerminator — Constitution Compliance Specification

Field	Value
Document	docs/11_constitution_compliance.md
Version	V1
Revision	1
Created	2026-06-28
Last modified	2026-06-28
Status	active
Status summary	Initial authoritative constitution compliance specification for HelixTerminator. Covers all 12 mandatory sections: constitution overview, package naming, AGENTS.MD, CLAUDE.MD, helix-deps.yaml, test types, CI/CD gates, anti-patterns, code review checklist, repository structure, changelog convention, and compliance scoring.
Issues	none
Issues summary	—
Fixed	initial creation
Fixed summary	—
Continuation	Update when HelixConstitution submodule is upgraded or when new services are added.

This document is the authoritative governance reference for HelixTerminator’s compliance with the HelixConstitution. Every engineer, every AI agent, and every automated pipeline operating on the HelixTerminator codebase MUST read and comply with every clause in this document.

Scope note: This document addresses **Module B (Zero-Trust Connection Broker)** of the dual-scope helix_terminator product; see docs/research/mvp/output/CANONICAL_FACTS.md (CD-1) for the full product-scope reconciliation (Module A vs Module B). A dedicated Scope & Module Boundary section is > DEFERRED (next increment).

Table of Contents

1. Constitution Overview and HelixTerminator Scope
 2. Package Naming Conventions
 3. AGENTS MD for HelixTerminator
 4. CLAUDE MD for HelixTerminator
 5. helix-deps yaml Specification
 6. Mandatory Test Types per Constitution
 7. CI/CD Constitution Compliance Gates
 8. Anti-Patterns and Forbidden Patterns
 9. Code Review Checklist (Constitution-Mandated)
 10. Repository Structure Compliance
 11. Changelog Convention
 12. Compliance Scoring and Monitoring
-

Section 1: Constitution Overview and HelixTerminator Scope

1.1 What the HelixConstitution Governs

The HelixConstitution (repository: git@github.com:HelixDevelopment/HelixConstitution.git, included as the constitution/ submodule and pinned at commit e6504c2 — git describe = helixcode-v1.1.0-39-ge6504c2; cited throughout this document as “**HelixConstitution (pinned e6504c2, helixcode-v1.1.0 line)**” per CANONICAL_FACTS.md CD-9 — do NOT assert “v2.0” / “v1.0.0” / “v4.1.0” for this submodule) is the **universal, project-agnostic** engineering governance framework shared by every project inside the HelixDevelopment organisation (org/ domain identity per CANONICAL_FACTS.md CD-2). vasic-digital/* submodule import paths appear throughout this document as the shared-library org prefix; standardizing those paths under

HelixDevelopment is > DEFERRED (next increment) — see CANONICAL_FACTS.md CD-2/CD-11. It defines non-negotiable rules across:

Domain	Governed Behaviour
Anti-bluff covenant (§11.4)	Every PASS must carry positive runtime evidence; no metadata-only, config-only, or grep-based PASSes
Test coverage (§1)	Four-layer floor: pre-build gate, post-build gate, runtime test, meta-test mutation
Credentials handling (§11.4.10)	.env files git-ignored; runtime-load only; no secrets in tracked tree
Data safety (§9)	Hardlinked backup before destructive ops; host RAM ≤ 60%; abort on pre-flight failure
Documentation discipline (§11.4.65)	Every change keeps docs in sync in the same commit
Submodule governance (§11.4.28–§11.4.31)	Owned submodules are equal parts of the codebase; no nested own-org chains
Commit discipline (§11.4.22–§11.4.26)	Official wrapper only; push to all upstreams; no --no-verify
Naming conventions (§11.4.29)	Lowercase snake_case for all directories, files, and submodule names
Repository hygiene (§11.4.30)	Proper .gitignore everywhere; no build artefacts tracked
Dependency manifest (§11.4.31)	Every submodule ships helix-deps.yaml
Spec versioning (§11.4.73)	Two-axis versioning: primary V + secondary Revision N
Catalogue-first discovery (§11.4.74)	Check submodules-catalogue.md before scaffolding any new module

Domain	Governed Behaviour
Container mandate (§11.4.76)	Use vasic-digital/containers for ALL containerised workloads
CodeGraph mandate (§11.4.78)	Use @colbymchenry/codegraph for code intelligence

The Constitution operates in three layers (top-to-bottom, project overrides universal when explicit):

```

Layer 1 (BASE)      constitution/Constitution.md    – universal
rules, all projects
Layer 2 (PROJECT)  HelixTerminator/Constitution.md –
HelixTerminator-specific rules
Layer 3 (SUBDIR)   <subdir>/CLAUDE.md              – module-local
overrides (optional)

```

1.2 How HelixTerminator Adopts the HelixConstitution

1.2.1 Verbatim Adoption Commitment

HelixTerminator **unconditionally** inherits every clause of the HelixConstitution. The following statement appears verbatim at the top of HelixTerminator’s root Constitution.md:

```

This constitution EXTENDS the Helix Universal Constitution at
`constitution/Constitution.md`. All clauses there apply unless
explicitly overridden below with an explicit `Override $X.Y`
section. There are NO overrides in HelixTerminator – all universal
clauses apply at full strength.

```

1.2.2 Submodule Wiring

The constitution is included as a Git submodule pinned to a specific verified commit (CD-9):

```

# In the HelixTerminator repository root:
git submodule add git@github.com:HelixDevelopment/
    HelixConstitution.git constitution
cd constitution
git checkout e6504c2    # pin to verified commit (helixcode-v1.1.0
    line; CANONICAL_FACTS.md CD-9)
cd ..
git add constitution .gitmodules

```

```
git commit -m "chore: add HelixConstitution submodule pinned to
e6504c2 (helixcode-v1.1.0 line)
```

Classification: universal (§11.4.17)

Rationale: HelixTerminator opts into the Helix Universal Constitution."

Every teammate clones with:

```
git clone --recurse-submodules git@github.com:HelixDevelopment/
HelixTerminator.git
# or, if already cloned:
git submodule update --init --recursive
```

1.2.3 Multi-Upstream Push Configuration

After adding the constitution submodule, run:

```
cd constitution
bash install_upstreams.sh
```

This configures remotes for all four providers: GitHub (primary), GitLab, GitFlic, GitVerse. Every commit to the constitution submodule MUST be pushed to ALL four providers.

1.2.4 HelixTerminator-Specific Extensions

HelixTerminator extends the universal constitution with the following project-specific rules:

Extension Rule	Description
HT-001	All 25 microservices MUST be registered in <code>helix-deps.yaml</code>
HT-002	Go version MUST be 1.25 or higher in all <code>go.mod</code> files
HT-003	Flutter client MUST target iOS 17+ and Android 14+
HT-004	All Kafka topics MUST follow <code>helix.terminator.<domain>.<event></code> format
HT-005	All databases MUST use <code>helixterm_<service>_db</code> naming
HT-006	Service-to-service calls MUST use gRPC with protocol buffers

Extension Rule	Description
HT-007	All HTTP APIs MUST serve OpenAPI 3.1 spec at <code>/openapi.json</code>
HT-008	Every service MUST expose <code>/health/live</code> and <code>/health/ready</code> endpoints
HT-009	Circuit breakers MUST be configured with <code>vasic-digital/recovery</code> submodule
HT-010	All structured logs MUST use <code>vasic-digital/observability</code> submodule

1.3 Applicability Matrix

The following matrix shows which constitution rules apply to which HelixTerminator components:

Constitution Rule	Go Services	Shared Libs	Flutter Client	CI/CD	Docs
§11.4 Anti-bluff	✓	✓	✓	✓	✓
§1 Four-layer tests	✓	✓	✓	✓	—
§11.4.10 Credentials	✓	✓	✓	✓	—
§9 Data safety	✓	✓	✓	✓	—
§11.4.28 Submodules	✓	✓	✓	—	—
§11.4.29 <code>snake_case</code>	✓	✓	✓	✓	✓
§11.4.30 <code>.gitignore</code>	✓	✓	✓	✓	—
§11.4.31 <code>helix-deps</code>	✓	✓	✓	—	—
§11.4.73 Spec version	—	—	—	—	✓
§11.4.74 Catalogue-first	✓	✓	✓	—	—
§11.4.76 Containers	✓	—	—	✓	—
§11.4.78 CodeGraph	✓	✓	✓	✓	—
	✓	✓	✓	✓	✓

Constitution Rule	Go Services	Shared Libs	Flutter Client	CI/CD	Docs
§11.4.65 Doc discipline					

1.4 Constitution Versioning and Update Protocol

1.4.1 Version Axes (§11.4.73)

Per §11.4.73 spec-versioning discipline: - **Primary V**: Major rewrites (V1, V2, V3). Increment when fundamental governance model changes. - **Secondary Revision N**: Additive changes within a primary version. Increment for new clauses, clarifications, extensions.

This document is at **V1 Revision 1**.

1.4.2 Update Workflow (§11.4.26)

When updating the constitution submodule:

```
# Step 1: Fetch + pull upstream (BEFORE any edits)
cd constitution
git fetch --all
git pull --ff-only origin main

# Step 2: Validate post-pull
bash scripts/verify-all-constitution-rules.sh

# Step 3: If consuming project files need updating, apply changes
# with §11.4.17 classification + verbatim mandate quote

# Step 4: Commit governance files only – NEVER git add -A
git add Constitution.md CLAUDE.md AGENTS.md QWEN.md
git commit -m "chore(constitution): update to upstream HEAD

Classification: universal (§11.4.17)
Rationale: Routine constitution pull per §11.4.26 workflow."

# Step 5: Push to ALL upstreams
git push origin main # fans out to github, gitlab, gitflic,
                    gitverse

# Step 6: Update consuming project's submodule pointer
cd ..
```

```
git add constitution
git commit -m "chore: bump constitution submodule to new HEAD"
```

```
# Step 7: Run cascade verifier
bash scripts/verify-all-constitution-rules.sh
```

1.4.3 Constitution Inheritance Verification

Every CI run MUST verify the constitution submodule is present and at the expected revision:

```
# scripts/test_constitution_inheritance.sh
#!/usr/bin/env bash
set -euo pipefail

# Pinned per CANONICAL_FACTS.md CD-9: HelixConstitution (pinned
#   e6504c2, helixcode-v1.1.0 line).
# Do NOT compare against a tag – the verified pin is the commit
#   itself, not an exact-match tag.
EXPECTED_COMMIT="e6504c2"
CONSTITUTION_DIR="constitution"

if [ ! -f "${CONSTITUTION_DIR}/Constitution.md" ]; then
    echo "FAIL: constitution submodule not initialised"
    exit 1
fi

ACTUAL_COMMIT=$(cd "${CONSTITUTION_DIR}" && git rev-parse --
    short=7 HEAD 2>/dev/null || echo "UNKNOWN")
if [ "${ACTUAL_COMMIT}" != "${EXPECTED_COMMIT}" ]; then
    echo "FAIL: constitution at ${ACTUAL_COMMIT}, expected $
        {EXPECTED_COMMIT} (helixcode-v1.1.0 line)"
    exit 1
fi

# Verify project CLAUDE.md references the submodule
if ! grep -q "constitution/CLAUDE.md" CLAUDE.md; then
    echo "FAIL: CLAUDE.md does not reference constitution/CLAUDE.md"
    exit 1
fi

# Verify project AGENTS.md references the submodule
if ! grep -q "constitution/AGENTS.md" AGENTS.md; then
    echo "FAIL: AGENTS.md does not reference constitution/AGENTS.md"
    exit 1
fi
```



```
echo "PASS: constitution inheritance verified at $
      {EXPECTED_COMMIT}"
```

Section 2: Package Naming Conventions

2.1 Governing Principles

All naming in HelixTerminator follows §11.4.29 (lowercase snake_case) and HelixDevelopment tradition (e.g., helixtrack.ru/core, digital.vasic.*). The module path is helixterm.io.

2.2 Go Service Module Naming: helixterm.io/services/<name>

Rule set:

Rule	Specification
HT-NAME-001	Module path format: helixterm.io/services/<service_name>
HT-NAME-002	<service_name> MUST be lowercase, hyphen-separated (e.g., auth-gateway, session-manager)
HT-NAME-003	Each service MUST have its own go.mod with this module path
HT-NAME-004	Go package names within a service MUST be lowercase, no hyphens (e.g., package authgateway)
HT-NAME-005	Internal packages unreachable from outside use /internal/ path segment
HT-NAME-006	gRPC generated code goes in <service>/gen/go/<proto_package>/
HT-NAME-007	Service binary entrypoint MUST be in cmd/<service_name>/main.go

Examples:

```
helixterm.io/services/auth-gateway
helixterm.io/services/session-manager
helixterm.io/services/connection-broker
```

```
helixterm.io/services/protocol-handler
helixterm.io/services/event-router
```

go.mod template for a service:

```
module helixterm.io/services/auth-gateway
```

```
go 1.25
```

```
require (
    helixterm.io/pkg/common v0.0.0
    helixterm.io/pkg/observability v0.0.0
    digital.vasic.observability v1.2.0
    digital.vasic.auth v1.1.0
    digital.vasic.recovery v1.0.0
)
```

2.3 Go Shared Library Naming: helixterm.io/pkg/<name>

Rule set:

Rule	Specification
HT-NAME-010	Shared library module path: helixterm.io/pkg/<lib_name>
HT-NAME-011	<lib_name> MUST be lowercase, hyphen-separated
HT-NAME-012	Libraries MUST be project-agnostic; no service-specific logic
HT-NAME-013	Libraries are candidates for promotion to vasic-digital per §11.4.74
HT-NAME-014	Libraries MUST NOT import from helixterm.io/services/*
HT-NAME-015	Each library MUST have a README.md describing its public API

Canonical shared libraries:

helixterm.io/pkg/common	– shared types, errors, constants
helixterm.io/pkg/observability	– logging/tracing wrappers over
digital.vasic.observability	
helixterm.io/pkg/middleware	– HTTP/gRPC middleware chain
helixterm.io/pkg/config	– configuration loading and
validation	

helixterm.io/pkg/testutil – shared test utilities (unit-test only)
helixterm.io/pkg/proto – protobuf definitions and generated stubs

2.4 Flutter Package Naming: `io.helixterm.<module>`

Rule set:

Rule	Specification
HT-NAME-020	Flutter package ID format: <code>io.helixterm.<module_name></code>
HT-NAME-021	Application bundle ID: <code>io.helixterm.client</code>
HT-NAME-022	<code><module_name></code> MUST be lowercase with underscores (Dart convention)
HT-NAME-023	Each Dart package name: in <code>pubspec.yaml</code> MUST match: <code>helixterm_<module></code>
HT-NAME-024	Dart import paths MUST use the package: scheme: <code>package:helixterm_client/</code>
HT-NAME-025	Feature modules follow: <code>io.helixterm.<feature></code> (e.g., <code>io.helixterm.auth</code> , <code>io.helixterm.session</code>)

Dart package examples:

```
# client/pubspec.yaml
name: helixterm_client
description: HelixTerminator Flutter client
publish_to: none
```

```
environment:
  sdk: ">=3.4.0 <4.0.0"
  flutter: ">=3.24.0"
```

`io.helixterm.client` – main application
`io.helixterm.auth` – authentication module
`io.helixterm.session` – session management UI
`io.helixterm.protocol` – protocol selection UI
`io.helixterm.settings` – application settings

2.5 Kafka Topic Naming:

helix.terminator.<domain>.<event>

Rule set:

Rule	Specification
HT-NAME-030	Topic format: helix.terminator.<domain>.<event>
HT-NAME-031	All segments MUST be lowercase with hyphens allowed within segments
HT-NAME-032	<domain> maps to a service domain (e.g., auth, session, connection, billing)
HT-NAME-033	<event> MUST be past-tense verb describing what occurred (e.g., created, terminated, updated)
HT-NAME-034	DLQ topics append .dlq suffix: helix.terminator.<domain>.<event>.dlq
HT-NAME-035	Retry topics append .retry.<n>: helix.terminator.<domain>.<event>.retry.1
HT-NAME-036	Internal command topics: helix.terminator.internal.<service>.<command>
HT-NAME-037	Maximum topic name length: 249 characters (Kafka limit)

Full domain × event matrix:

Domain	Events
auth	user.authenticated, user.session-created, user.session-expired, token.refreshed, token.revoked
session	session.started, session.terminated, session.suspended, session.resumed
connection	connection.established, connection.dropped, connection.migrated, connection.throttled
protocol	protocol.negotiated, protocol.downgraded, protocol.upgraded, protocol.failed
billing	subscription.created, subscription.renewed, subscription.cancelled, usage.recorded
user	user.registered, user.verified, user.deleted, user.profile-updated
node	node.registered, node.deregistered, node.health-changed, node.capacity-updated

Domain	Events
metrics	metrics.snapshot-taken, metrics.threshold-breached, metrics.alert-fired

Full examples:

```
helix.terminator.auth.user.authenticated
helix.terminator.auth.user.authenticated.dlq
helix.terminator.auth.user.authenticated.retry.1
helix.terminator.session.session.started
helix.terminator.connection.connection.dropped
helix.terminator.internal.auth-gateway.invalidate-cache
```

2.6 Database Naming: helixterm_<service>_db

Rule set:

Rule	Specification
HT-NAME-040	Database name format: helixterm_<service_snake>_db
HT-NAME-041	<service_snake> is the service name with hyphens replaced by underscores
HT-NAME-042	Schema names within a database: <domain> (lowercase, no prefix)
HT-NAME-043	Table names: lowercase snake_case, singular noun (e.g., session, connection_event)
HT-NAME-044	Index names: idx_<table>_<columns>
HT-NAME-045	Foreign key names: fk_<table>_<referenced_table>
HT-NAME-046	Migration files: <timestamp>_<description>.sql (e.g., 20260628_001_create_sessions_table.sql)

Database name examples:

```
helixterm_auth_gateway_db
helixterm_session_manager_db
helixterm_connection_broker_db
helixterm_billing_service_db
helixterm_user_registry_db
helixterm_node_manager_db
```

helixterm_metrics_collector_db
helixterm_event_router_db

2.7 Kubernetes Resource Naming

Rule set:

Resource Type	Format	Example
Namespace	helixterm-<env>	helixterm-prod, helixterm-staging
Deployment	helixterm-<service>	helixterm-auth- gateway
Service	helixterm-<service>- svc	helixterm-auth- gateway-svc
ConfigMap	helixterm-<service>- config	helixterm-auth- gateway-config
Secret	helixterm-<service>- secrets	helixterm-auth- gateway-secrets
ServiceAccount	helixterm-<service>- sa	helixterm-auth- gateway-sa
HPA	helixterm-<service>- hpa	helixterm-auth- gateway-hpa
PodDisruptionBudget	helixterm-<service>- pdb	helixterm-auth- gateway-pdb

Mandatory label schema (every Kubernetes resource MUST carry):

labels:

```
app.kubernetes.io/name: helixterm-auth-gateway
app.kubernetes.io/instance: helixterm-auth-gateway-prod
app.kubernetes.io/version: "1.0.0"
app.kubernetes.io/component: microservice
app.kubernetes.io/part-of: helixterm
app.kubernetes.io/managed-by: helm
helix.io/service: auth-gateway
helix.io/domain: auth
helix.io/environment: prod
helix.io/constitution-version: "e6504c2" # HelixConstitution
    pinned commit, helixcode-v1.1.0 line (CD-9)
```

Namespace conventions:

helixterm-prod	– production workloads
helixterm-staging	– staging / pre-prod
helixterm-dev	– development environment
helixterm-test	– automated test runs
helixterm-monitoring	– Prometheus, Grafana, Alertmanager
helixterm-infra	– Kafka, databases, Redis, Vault

2.8 Docker Image Naming: ghcr.io/helixdevelopment/ helixterm- <service> : <version>

Rule set:

Rule	Specification
HT-NAME-050	Image format: ghcr.io/helixdevelopment/ helixterm- <service> : <version>
HT-NAME-051	<version> MUST follow semver with project prefix per §11.4.151: helixterm- <semver>
HT-NAME-052	Tag latest is FORBIDDEN in production deployments
HT-NAME-053	Every image MUST also be tagged with its full SHA-256 digest
HT-NAME-054	Build args MUST NOT embed secrets; use multi- stage builds
HT-NAME-055	Images MUST be built with rootless Podman per §11.4.161
HT-NAME-056	vasic-digital/containers submodule MUST be used for all container orchestration per §11.4.76

Image name examples:

```
ghcr.io/helixdevelopment/helixterm-auth-gateway:helixterm-1.0.0
ghcr.io/helixdevelopment/helixterm-session-manager:helixterm-1.0.0
ghcr.io/helixdevelopment/helixterm-connection-
broker:helixterm-1.2.3
ghcr.io/helixdevelopment/helixterm-client:helixterm-2.0.0-beta.1
```

2.9 Git Branch Naming Convention

Branch Type	Format	Example
Feature	feat/ <ticket> - <short-desc>	feat/HT-042-add- session-resumption

Branch Type	Format	Example
Bug fix	fix/<ticket>-<short-desc>	fix/HT-099-auth-token-expiry-race
Chore	chore/<ticket>-<short-desc>	chore/HT-011-update-constitution
Release	release/<version>	release/1.0.0
Hotfix	hotfix/<ticket>-<short-desc>	hotfix/HT-201-prod-connection-leak
Experiment	experiment/<desc>	experiment/quic-transport-prototype
Constitution	constitution/<clause>	constitution/114-31-helix-deps-yaml

Rules: - Branch names MUST be lowercase with hyphens only (no underscores, no uppercase) - Ticket prefix MUST match the HelixTerminator issue tracker prefix: HT-<NNN> - main and develop are protected; direct push FORBIDDEN - Release branches are cut from develop; hotfix branches from main

2.10 Commit Message Convention

HelixTerminator uses **Conventional Commits** extended with Helix-specific fields.

Format:

<type>(<scope>): <subject>

<body>

<helix-footers>

Classification: <universal|project-specific> (§11.4.17)

Ticket: HT-<NNN>

Constitution: §<X.Y.Z> [if this commit enforces a specific rule]

Co-authored-by: <name> <email> [if applicable]

Allowed types: | Type | When to Use | |——|————| | feat | New feature or capability | | fix | Bug fix | | chore | Maintenance, dependency updates | | refactor | Code restructuring without behaviour change | | test | Adding or updating tests | | docs | Documentation changes only | | perf | Performance improvement | | security | Security fix or hardening | | ci | CI/CD pipeline changes | | constitution | Constitution compliance changes | | revert | Revert a previous commit | | build | Build system changes |

Scope = service or component name: auth-gateway, session-manager, client, pkg/common, k8s, ci, constitution, docs

Example commit messages:

feat(auth-gateway): implement JWT refresh with rotation

Implements §HT-005 token rotation strategy using vasic-digital/auth submodule. Adds Redis-backed token family tracking to prevent replay attacks.

Test coverage:

- Unit: 94% line coverage
- Integration: auth-gateway × redis × vault full stack
- Contract: pact contract added for client ↔ auth-gateway
- Security: SAST clean, no new CVEs in dependency scan

Classification: project-specific (§11.4.17)

Ticket: HT-042

Constitution: §11.4.25 (full automation coverage)

chore(constitution): bump constitution submodule to v1.1.0

Pulls §11.4.170 UI visual proof mandate. Adds paired mutation gate CM-COVENANT-114-170-PROPAGATION to CI pipeline.

Classification: universal (§11.4.17)

Ticket: HT-011

Constitution: §11.4.26 (submodule update workflow)

2.11 All 25 Services — Naming Convention (roster owned by doc-01)

CD-3 (CANONICAL_FACTS.md): the 25-microservice roster is NOT enumerated in this document. 01_core_architecture.md owns the single canonical service registry (name, port, DB, owning team, submodule deps); every other document — including this one — references it rather than re-enumerating divergently. (This section previously listed a 25-service set that also contradicted Appendix A.2’s separate 25-service list below; both have been replaced with this pointer so the document no longer self-contradicts. A transcluded, cross-doc-consistent registry table is > DEFERRED (next increment).)

This document’s role is only to define the per-dimension naming **convention** that every service in doc-01’s registry MUST follow, shown here for one representative example:

Service Name (example, per doc-01 registry)	Go Module Path	Kafka Domain	Database
<service-name>	helixterm.io/ services/ <service-name>	helix.terminator.<domain>.*	helixterm_<service_snake>

Docker image name pattern (ghcr.io registry prefix, applied to every service in doc-01’s registry):

ghcr.io/helixdevelopment/helixterm-<service-name>:<version>

Section 3: AGENTS_MD for HelixTerminator

DEPLOYMENT NOTE: The following content MUST be placed verbatim in the file AGENTS.md at the HelixTerminator repository root. This is the complete, deployable file.

HelixTerminator – AGENTS.md

Field	Value
Revision	1
Created	2026-06-28
Last modified	2026-06-28
Status	active

```
> **Base agent rules: `constitution/AGENTS.md` – READ IT FIRST.**
> The base file is authoritative for any topic not covered here.
> This file EXTENDS, never weakens, the universal AGENTS.md.
```

```
## INHERITED FROM constitution/AGENTS.md
```

All rules in ``constitution/AGENTS.md`` (and the ``constitution/Constitution.md`` it references) apply unconditionally. The critical rules restated below are for agents that do not follow import links.

```
**Critical base rules (always in force):**
```

- Never commit secrets (§11.4.10).
- All commits go through ``scripts/commit_all.sh``.
- Anti-bluff covenant binds every test (§11.4).
- No guessing language: ``likely``, ``probably``, ``maybe``, ``might``, ``appears``, ``seems`` are FORBIDDEN when reporting causes (§11.4.6).
- Test coverage four-layer floor: pre-build, post-build, runtime, meta-test mutation (§1).
- Never force-push without explicit per-session human authorisation (§9.2).
- CONTINUATION.md kept in sync every non-trivial state change (§12.10).
- Credentials NEVER tracked; ``.env`` patterns git-ignored (§11.4.10).
- Submodule-catalogue-first: check ``constitution/submodules-catalogue.md`` BEFORE scaffolding any new module (§11.4.74).

```
@constitution/AGENTS.md
```

```
## Project Overview for Agents
```

HelixTerminator is an enterprise-grade network termination and session management platform consisting of:

- **25 Go microservices** at module path ``helixterm.io/services/<name>``
- **6 shared Go libraries** at module path ``helixterm.io/pkg/<name>``
- **1 Flutter client** with bundle ID ``io.helixterm.client``
- **Go version**: 1.25 (MUST be exactly 1.25 or higher; never downgrade)
- **Flutter version**: 3.24 or higher
- **Dart version**: 3.4 or higher
- **Protobuf**: proto3 exclusively
- **Message broker**: Apache Kafka 3.9+
- **Service mesh**: Istio 1.22+ with mTLS enforced
- **Container runtime**: Podman 5+ rootless mode ONLY (\$11.4.161)

The repository is a **monorepo** managed with ``go.work``. Every service has an independent ``go.mod``. The workspace ``go.work`` file lists all modules.

Tech Stack Agents Must Use

Go Services

```
``go
// Mandatory dependencies for every service
import (
    // Observability – MUST use vasic-digital submodule
    "digital.vasic.observability"    // structured logging,
    tracing
    // Auth – MUST use vasic-digital submodule
    "digital.vasic.auth"            // JWT, session tokens
    // Circuit breaker + fault tolerance
    "digital.vasic.recovery"        // circuit breaker, retry
    // Rate limiting
    "digital.vasic.ratelimiter"     // rate limiting
    // Messaging
    "digital.vasic.messaging"       // Kafka producer/consumer
    // Database
    "digital.vasic.database"        // database abstractions
    // Config
    "digital.vasic.config"          // configuration loading
)
```

Forbidden direct imports (use vasic-digital submodules instead): -
github.com/sirupsen/logrus → use digital.vasic.observability -
github.com/uber-go/zap → use digital.vasic.observability -
github.com/sony/gobreaker → use digital.vasic.recovery - github.com/
Shopify/sarama (directly) → use digital.vasic.messaging - Raw
database/sql in handlers → MUST go through repository layer

Flutter Client

```
# Mandatory dependencies in pubspec.yaml
dependencies:
  # UI design system – per §11.4.162
  opendesign: ^1.0.0
  # State management
  flutter_bloc: ^8.1.0
  # Network
  dio: ^5.4.0
  # Local storage
  flutter_secure_storage: ^9.0.0
  # Observability
  firebase_crashlytics: ^3.4.0
  firebase_performance: ^0.9.3
```

Infrastructure

- Kubernetes manifests: Helm charts in charts/<service>/
- Container images: built with vasic-digital/containers submodule
- Secrets: HashiCorp Vault via helixterm.io/services/key-vault
- CI/CD: GitHub Actions only

Forbidden Patterns and Anti-Patterns

HARD FORBIDDEN (immediate PR rejection)

1. Duplicating vasic-digital submodule functionality

```
// FORBIDDEN: reimplementing circuit breaker
type circuitBreaker struct { ... }
// REQUIRED: use submodule
import "digital.vasic.recovery"
cb := recovery.NewCircuitBreaker(recovery.Config{...})
```

2. Hardcoded credentials anywhere in tracked files

```
// FORBIDDEN
const dbPassword = "s3cr3t"
// REQUIRED: load from environment
dbPassword := os.Getenv("DB_PASSWORD") // loaded from Vault
```

3. Global mutable state in microservices

```
// FORBIDDEN
var globalCache = map[string]interface{}{}
// REQUIRED: inject dependencies
type Service struct { cache Cache }
```

4. Missing context propagation

```
// FORBIDDEN: context not propagated
func (s *Service) Process() error { ... }
// REQUIRED: context first parameter always
func (s *Service) Process(ctx context.Context) error { ... }
```

5. Skipping error handling

```
// FORBIDDEN
result, _ := doSomething()
// REQUIRED
result, err := doSomething()
if err != nil {
    return fmt.Errorf("doing something: %w", err)
}
```

6. Missing timeout on network calls

```
// FORBIDDEN: no timeout
resp, err := http.Get(url)
// REQUIRED: always set deadline
ctx, cancel := context.WithTimeout(ctx, 5*time.Second)
defer cancel()
resp, err := http.Get(url) // use ctx
```

7. Direct database access from HTTP/gRPC handlers

```
// FORBIDDEN: handler directly queries DB
func (h *Handler) GetUser(ctx context.Context, req
    *pb.GetUserRequest) (*pb.User, error) {
    row := h.db.QueryRowContext(ctx, "SELECT ...")
    ...
}
// REQUIRED: handler calls service, service calls repository
```

```
func (h *Handler) GetUser(ctx context.Context, req
    *pb.GetUserRequest) (*pb.User, error) {
    user, err := h.userService.GetByID(ctx, req.Id)
    ...
}
```

8. Missing circuit breaker on external calls

```
// FORBIDDEN: no circuit breaker on external service call
resp, err := externalClient.Call(ctx, req)
// REQUIRED: wrap with circuit breaker
resp, err := s.cb.Execute(ctx, func() (interface{}, error) {
    return externalClient.Call(ctx, req)
})
```

9. Missing structured logging

```
// FORBIDDEN: unstructured logging
fmt.Printf("user %s authenticated\n", userID)
log.Printf("error: %v", err)
// REQUIRED: structured logging via digital.vasic.observability
s.logger.Info("user authenticated",
    "user_id", userID,
    "method", "jwt",
)
s.logger.Error("authentication failed",
    "user_id", userID,
    "error", err,
)
```

10. **Mocks/stubs in non-unit tests (§11.4.27)** Integration, E2E, and all other non-unit tests MUST use real, fully implemented system components. No `httptest.NewServer` with fake responses in integration tests — spin up real services.

11. TODO/FIXME in production code

```
// FORBIDDEN in non-test files
// TODO: implement this properly
// FIXME: this is a hack
```

12. init() functions with side effects

```
// FORBIDDEN: init with side effects
func init() { connectToDatabase() }
// REQUIRED: explicit initialisation in main or constructor
```

13. Panic in library/service code

```
// FORBIDDEN except main.go startup validation
panic("connection failed")
// REQUIRED: return error
return nil, fmt.Errorf("connection failed: %w", err)
```

14. **Missing /health/live and /health/ready endpoints** Every service MUST expose both endpoints; absence blocks deployment.

15. **Using latest tag in production container images** Every production deployment MUST pin exact image digest.

Required Test Coverage (§11.4.25)

Coverage Thresholds

Metric	Threshold	Enforcement
Line coverage	≥ 80% per package	CI gate (hard fail)
Branch coverage	≥ 70% per package	CI gate (hard fail)
Function coverage	≥ 85% per package	CI gate (hard fail)
Mutation score	≥ 75% per package	CI gate (soft warn → hard fail at release)

Test Naming Conventions

```
// Unit test file: <file>_test.go in same package
// Integration test file: <file>_integration_test.go with build tag
// E2E test file: tests/e2e/<scenario>_test.go

// Unit test naming: Test<Function>_<Scenario>_<ExpectedOutcome>
func TestAuthGateway_Authenticate_ValidJWT_ReturnsUser(t
    *testing.T) { ... }
func TestAuthGateway_Authenticate_ExpiredJWT_ReturnsUnauthorized(t
    *testing.T) { ... }
func TestAuthGateway_Authenticate_MalformedToken_ReturnsBadRequest(t
    *testing.T) { ... }
```



```
// Benchmark naming: Benchmark<Function>_<Scenario>
func BenchmarkAuthGateway_Authenticate_ValidJWT(b *testing.B)
    { ... }
```

Four-Layer Test Floor (§1)

Every change **MUST** have tests at all four layers:

1. **Pre-build gate:** go vet ./... + golangci-lint run — ZERO violations
2. **Post-build gate:** unit tests pass + coverage thresholds met
3. **Runtime test:** integration tests with real dependencies
4. **Meta-test mutation:** paired mutation that proves the gate catches regressions

```
// Meta-test mutation example:
// File: tests/meta/coverage_gate_mutation_test.go
//
// Anti-bluff (§1.1): this is a REAL asserting body, not a
// comment-only stub.
// Plant a mutated coverage value below the gate's threshold and
// assert that
// the gate's own pass/fail decision function reports FAIL for it
// — proving
// the coverage gate actually catches a regression instead of
// always passing.
func TestCoverageGateMutation(t *testing.T) {
    const threshold = 80.0 // line-coverage floor asserted
                          by the gate
    const mutatedCoverage = 62.0 // planted regression: well
                                below threshold

    passed := evaluateCoverageGate(mutatedCoverage, threshold)

    if passed {
        t.Fatalf("coverage gate mutation test FAILED: gate
            reported PASS for %.1f%% coverage "+
                "against a %.1f%% threshold — the gate is not catching
            regressions", mutatedCoverage, threshold)
    }
}

// evaluateCoverageGate mirrors the CI coverage gate's pass/fail
// decision.
func evaluateCoverageGate(actualCoverage, thresholdCoverage
    float64) bool {
```

```
    return actualCoverage >= thresholdCoverage
}
```

Required Code Review Checklist

Before submitting a PR, agents MUST verify all items in Section 9 of docs/11_constitution_compliance.md. Key items:

- ☐ All four test layers pass
- ☐ Coverage $\geq 80\%$ line, $\geq 70\%$ branch for all changed packages
- ☐ `golangci-lint` run zero violations
- ☐ `scripts/constitution-check.go` passes with score ≥ 95
- ☐ No vasic-digital functionality duplicated
- ☐ No forbidden patterns present (see above)
- ☐ `helix-deps.yaml` updated if new dependencies added
- ☐ All new public functions/types have godoc comments
- ☐ CHANGELOG.md updated with entry under [Unreleased]
- ☐ CONTINUATION.md updated (§12.10)

Constitution Compliance Requirements

Before submitting ANY PR:

```
# Run constitution compliance check
go run scripts/constitution-check.go --strict

# Expected output:
# HelixTerminator Constitution Compliance Check
# =====
# [PASS] Package naming conventions
# [PASS] helix-deps.yaml present and valid
# [PASS] AGENTS.md references constitution
```

```
# [PASS] CLAUDE.md references constitution
# [PASS] Test coverage thresholds
# [PASS] No forbidden patterns detected
# [PASS] Commit message format
# [PASS] .gitignore completeness
# Compliance Score: 100/100
# Status: COMPLIANT
```

The compliance check MUST score ≥ 95 for PR merge and 100 for releases.

Submodule Usage Rules (§11.4.28 + §11.4.74)

1. **Check catalogue FIRST:** Before writing any new functionality, search constitution/submodules-catalogue.md for an existing submodule.
2. **MUST use these vasic-digital submodules** (NEVER reimplement):
 - vasic-digital/observability → all structured logging and tracing
 - vasic-digital/auth → all authentication and session token handling
 - vasic-digital/recovery → circuit breakers, retry, fault tolerance
 - vasic-digital/ratelimiter → all rate limiting
 - vasic-digital/messaging → all Kafka producer/consumer operations
 - vasic-digital/database → all database connection and query primitives
 - vasic-digital/config → all configuration loading
 - vasic-digital/middleware → all HTTP/gRPC middleware
 - vasic-digital/security → all cryptographic operations
 - vasic-digital/containers → ALL containerised workloads (§11.4.76)
3. **MUST NOT** inject project-specific context INTO submodules. Submodules remain project-not-aware and reusable (§11.4.28-B).
4. **MUST NOT** create nested own-org submodule chains (§11.4.28-C). All submodule dependencies live at <root>/<name>/ or <root>/submodules/<name>/.
5. When a submodule lacks needed functionality, **extend upstream** via PR — never duplicate (§11.4.74).

6. Record catalogue decision in every PR: Catalogue-Check: reuse|extend|no-match.

Performance Requirements Agents Must Meet

Metric	Threshold	Measurement
Service startup time	< 3 seconds	Integration test on CI
p99 gRPC latency (intra-service)	< 10ms	k6 benchmark, 1000 RPS
p99 HTTP API latency	< 50ms	k6 benchmark, 500 RPS
Kafka consumer lag	< 1000 messages	Continuous monitoring
Memory per service (steady state)	< 256 MB	pprof snapshot
CPU per service (idle)	< 5%	pprof snapshot
DB query p99	< 5ms	Query plan analysis

Benchmark regressions >10% in any metric BLOCK merge.

```
// Every service package with performance-sensitive code MUST have
// benchmarks:
// file: bench_test.go
func BenchmarkSessionManager_CreateSession(b *testing.B) {
    svc := newTestService(b)
    b.ResetTimer()
    b.RunParallel(func(pb *testing.PB) {
        for pb.Next() {
            _, err := svc.CreateSession(context.Background(),
                testSessionRequest)
            if err != nil {
                b.Fatal(err)
            }
        }
    })
}
```

Security Requirements Agents Must Never Violate

1. **No secrets in tracked files** — `.env` is git-ignored; secrets loaded from environment variables injected by Vault (§11.4.10).
2. **No SQL injection** — parameterised queries ONLY via `digital.vasic.database`.
3. **No command injection** — never construct shell commands from user input.
4. **No SSRF** — all outbound URLs validated against allowlist.
5. **No insecure random** — use `crypto/rand`, never `math/rand` for security.
6. **No JWT algorithm confusion** — pin algorithm in validation (EdDSA/Ed25519 only for access tokens, per `CANONICAL_FACTS.md` CD-7; RS256/ES256/HS256 are forbidden as the signing choice and MUST only appear in attack-test / negative-example code).
7. **No plaintext secrets in logs** — log only token prefixes (first 8 chars).
8. **No unvalidated redirects** — every redirect URL validated against allowlist.
9. **mTLS enforced** for all service-to-service Kafka and gRPC traffic.
10. **Pre-store leak audit (§11.4.10.A)** — before storing any credential, grep the entire tracked tree AND git history for the literal value.

File Structure Requirements

Every Go service MUST follow this exact layout:

```
services/<service-name>/
├─ cmd/
│   └─ <service-name>/
│       └─ main.go                # binary entrypoint only; no business
logic
├─ internal/
│   └─ handler/                  # gRPC/HTTP handlers; NO DB access
│   └─ service/                  # business logic; NO DB access
│   └─ repository/              # DB access ONLY; NO business logic
│   └─ domain/                  # domain types, interfaces, errors
│   └─ config/                  # service-specific config structs
│   └─ middleware/              # service-specific middleware
├─ api/
│   └─ proto/                   # .proto files
└─ gen/
```

```

|   └─ go/                                # generated protobuf Go code (git-
ignored)
├─ tests/
|   └─ integration/                        # integration tests (real
dependencies)
|       └─ e2e/                            # E2E test scenarios
├─ charts/                                # Helm chart for this service
├─ Dockerfile                             # multi-stage rootless build
├─ go.mod
├─ go.sum
├─ README.md
└─ CHANGELOG.md

```

How Agents Must Run Tests Before Submitting

```

# Step 1: Run linter (zero violations required)
golangci-lint run ./...

```

```

# Step 2: Run unit tests with coverage
go test -race -coverprofile=coverage.out -covermode=atomic ./...
go tool cover -func=coverage.out | grep total | awk '{print $3}'
# Minimum: 80.0%

```

```

# Step 3: Run integration tests (requires running infrastructure)
go test -tags integration -race ./tests/integration/...

```

```

# Step 4: Run contract tests
go test -tags contract ./tests/contract/...

```

```

# Step 5: Run security scan
gosec -fmt sarif -out gosec.sarif ./...

```

```

# Step 6: Run constitution compliance check
go run scripts/constitution-check.go --strict

```

```

# Step 7: Verify all four layers have tests for changed files
bash scripts/verify-test-layers.sh $(git diff --name-only origin/
main)

```

```

# ALL steps must pass before PR submission. No exceptions.

```

How Agents Must Check Constitution Compliance

```
# Full compliance check
go run scripts/constitution-check.go --strict --report

# Check specific rule
go run scripts/constitution-check.go --rule HT-NAME-001

# Verify submodule usage
go run scripts/constitution-check.go --check submodule-catalogue

# Output compliance report
go run scripts/constitution-check.go --output json > compliance-report.json
```

The constitution check covers: - Package naming convention validation (HT-NAME-*) - helix-deps.yaml schema validation - Forbidden pattern detection (AST-level) - Test coverage threshold verification - Submodule usage compliance - Commit message format (last 20 commits) - .gitignore completeness - AGENTS.md/CLAUDE.md presence and inheritance references

Section 4: CLAUDE_MD for HelixTerminator

DEPLOYMENT NOTE: The following content MUST be placed verbatim in the file CLAUDE.md at the HelixTerminator repository root. This is the complete, deployable file.

HelixTerminator – CLAUDE.md

Field	Value
Revision	1
Created	2026-06-28
Last modified	2026-06-28
Status	active

INHERITED FROM constitution/CLAUDE.md

All rules in `constitution/CLAUDE.md` (and the `constitution/Constitution.md` it references) apply unconditionally. Project-specific rules below extend them.

@constitution/CLAUDE.md

Claude's Role in HelixTerminator

Claude Code is a first-class engineering agent on HelixTerminator. Claude is authorised to:

- Read, analyse, and reason about the entire codebase
- Write and modify Go service code, Flutter client code, CI/CD pipelines, Kubernetes manifests, Helm charts, and documentation
- Run tests, linters, and compliance checks
- Generate protobuf definitions and regenerate Go stubs
- Propose and implement architectural changes (with human review)
- Update CHANGELOG.md, CONTINUATION.md, Issues.md, and related docs
- Run `scripts/commit\_all.sh` to commit and push approved changes
- Execute `go run scripts/constitution-check.go` at any time

What Claude Is Allowed to Do

Explicitly Allowed

1. ****Modify any `.go` file**** in `services/`, `pkg/`, `cmd/`, `tests/`, `scripts/`
2. ****Modify Flutter/Dart files**** in `client/`
3. ****Update Helm charts**** in `charts/`
4. ****Update Kubernetes manifests**** in `deploy/`
5. ****Update GitHub Actions workflows**** in `.github/workflows/`
6. ****Update protobuf definitions**** in `api/proto/` and regenerate stubs
7. ****Update configuration files****: `go.work`, `go.mod`, `pubspec.yaml`
8. ****Write all test types****: unit, integration, contract, E2E, performance, security, mutation, chaos, fuzz, golden, smoke
9. ****Update documentation****: `docs/`, `README.md`, `CHANGELOG.md`, `CONTINUATION.md`, `Issues.md`, `Fixed.md`
10. ****Run read-only operations****: `git status`, `git log`, `git diff`, `go build`,

- ``go test`, `go vet`, `golangci-lint run`, `kubectl get`,
`helm lint``
- 11. ****Execute compliance checks****: ``go run scripts/constitution-check.go``
- 12. ****Commit and push**** via ``scripts/commit_all.sh`` ONLY (never raw ``git commit``)
- 13. ****Update `helix-deps.yaml`**** when adding or updating submodule dependencies

Conditionally Allowed (require explicit human confirmation)

- 1. ****Force-push**** (``git push --force``): requires per-session human authorisation per §9.2; NEVER allowed without explicit approval
- 2. ****Deleting branches or tags****: requires human confirmation
- 3. ****Modifying `constitution/` directory contents****: follow §11.4.26 workflow exactly
- 4. ****Modifying `.github/branch-protection` rules****: requires operator approval
- 5. ****Changing Go version in `go.work`/`go.mod`****: requires operator approval
- 6. ****Changing Flutter/Dart SDK constraints in `pubspec.yaml`****: requires approval
- 7. ****Adding new external dependencies**** not in ``vasic-digital`` or ``HelixDevelopment``: requires catalogue check result + operator approval
- 8. ****Running destructive database migrations****: requires human review of migration plan

What Claude Must Never Do

Absolute Prohibitions (no exceptions, ever)

- 1. ****Never commit secrets**** – no API keys, passwords, tokens, certificates, private keys, or ``.env`` content in any tracked file (§11.4.10)
- 2. ****Never skip hooks**** – ``--no-verify`` and ``--no-gpg-sign`` are FORBIDDEN (§11.4)
- 3. ****Never force-push without per-session human authorisation**** (§9.2)
- 4. ****Never use guessing language**** when reporting causes: ``likely``, ``probably``, ``maybe``, ``might``, ``appears``, ``seems``, ``supposedly`` are FORBIDDEN (§11.4.6)
- 5. ****Never report PASS without captured runtime evidence**** (§11.4.2) – a test that passed in a previous run does not count; evidence must be from the current execution

6. ****Never duplicate vasic-digital submodule functionality****
(§11.4.74)
7. ****Never inject project-specific context into submodules****
(§11.4.28-B)
8. ****Never create nested own-org submodule chains**** (§11.4.28-C)
9. ****Never use `git add -A`**** in commit wrappers – stage only the intended files
10. ****Never bypass the four-layer test floor**** (§1) – no change ships without pre-build, post-build, runtime, and meta-test mutation layers passing
11. ****Never use raw `git commit`**** – always use ``scripts/commit_all.sh``
12. ****Never modify `.gitignore` to track build artefacts****
(§11.4.30)
13. ****Never exceed 60% of host RAM**** during build or test operations (§12)
14. ****Never report a feature as complete without end-to-end automation proof****
(§11.4.25) – "tests pass" ≠ "users can use the feature"
15. ****Never access Vault directly**** – only through ``helixterm.io/services/key-vault``
service APIs or the Vault SDK via environment injection

Security-Sensitive Operations Claude Must Not Perform Autonomously

1. ****Key rotation in production**** – must be operator-initiated via runbook
2. ****Database schema migrations on production**** – operator-supervised only
3. ****Changing RBAC / IAM policies**** in Kubernetes or cloud providers
4. ****Modifying firewall rules or security groups****
5. ****Modifying TLS certificate material****
6. ****Changing Kafka ACLs or topic configurations in production****
7. ****Modifying Vault policies or approle credentials****

How Claude Should Handle Ambiguous Requirements

1. ****Never guess****. If a requirement is ambiguous, STOP and ask for clarification.
Document the ambiguity as ``UNCONFIRMED:`` in the code comment.
2. ****State assumptions explicitly**** before implementing:

ASSUMPTION: The session timeout should be 15 minutes based on industry standard for enterprise VPN products. Please confirm or specify. ``

1. **Propose alternatives** when multiple valid interpretations exist:

INTERPRETATION A: Store session tokens in Redis with 15-min TTL
INTERPRETATION B: Store session tokens in PostgreSQL for audit trail
Please specify which approach is preferred.

2. **Never invent API contracts** — if the protobuf definition is unclear, ask for clarification before generating code from it.

3. **For performance thresholds:** use values from Section 3 (AGENTS.MD) as defaults; flag for operator review if the context suggests different needs.

4. **For security decisions:** always choose the more restrictive option by default; ask if relaxation is needed.

Claude's Test-Writing Requirements

Mandatory Test Anatomy

Every test Claude writes MUST contain:

1. **Arrange** — set up inputs, mocks (unit only), stubs (unit only), test data
2. **Act** — call the function/method/endpoint under test
3. **Assert** — verify all relevant output properties AND error paths
4. **Evidence capture** — for runtime tests, capture observable evidence

```
// Example: well-formed unit test
func TestSessionManager_CreateSession_ValidRequest_ReturnsSessionWithID(t
    *testing.T) {
    // Arrange
    t.Parallel()
    repo := &mockSessionRepository{} // mocks allowed in unit
    tests
    svc := session.NewService(repo, testConfig())
    req := &pb.CreateSessionRequest{
        UserID:    "user-123",
        Protocol:   pb.Protocol_WIREGUARD,
```

```

        NodeID:    "node-456",
    }

    // Act
    resp, err := svc.CreateSession(context.Background(), req)

    // Assert
    require.NoError(t, err)
    require.NotNil(t, resp)
    assert.NotEmpty(t, resp.SessionID)
    assert.Equal(t, pb.SessionStatus_ACTIVE, resp.Status)
    assert.WithinDuration(t, time.Now(), resp.CreatedAt.AsTime(),
        5*time.Second)

    // Verify repository was called correctly
    require.Len(t, repo.savedSessions, 1)
    assert.Equal(t, req.UserID, repo.savedSessions[0].UserID)
}

// Example: well-formed integration test (NO mocks – real
// dependencies)
func TestSessionManager_CreateSession_Integration(t *testing.T) {
    // +build integration
    t.Parallel()

    // Arrange: real service with real DB and Redis
    env := testenv.New(t) // spins up real PostgreSQL + Redis via
                           // containers submodule
    svc := session.NewService(
        repository.NewPostgreSQLSessionRepository(env.DB),
        env.Config,
    )
    req := &pb.CreateSessionRequest{
        UserID:    env.TestUserID,
        Protocol:  pb.Protocol_WIREGUARD,
        NodeID:    env.TestNodeID,
    }

    // Act
    resp, err := svc.CreateSession(env.Ctx, req)

    // Assert
    require.NoError(t, err)
    require.NotNil(t, resp)

```

```

    // Evidence: verify session is ACTUALLY in the database
    dbSession, err := env.DB.GetSession(env.Ctx, resp.SessionID)
    require.NoError(t, err, "session must be persisted in real
        database")
    assert.Equal(t, pb.SessionStatus_ACTIVE, dbSession.Status)
    // Capture evidence
    t.Logf("EVIDENCE: session %s created in DB at %v",
        resp.SessionID, dbSession.CreatedAt)
}

```

Coverage Standards per Test File

File Type	Minimum Line Coverage	Minimum Branch Coverage
handler/	85%	75%
service/	85%	75%
repository/	80%	70%
domain/	90%	80%
cmd/	70%	60%

Test Build Tags

```

//go:build integration
// +build integration

```

```

//go:build contract
// +build contract

```

```

//go:build e2e
// +build e2e

```

```

//go:build performance
// +build performance

```

```

//go:build chaos
// +build chaos

```

Claude's Code Review Checklist

Before suggesting a PR is ready for merge, Claude MUST verify:

Architecture - [] Layer boundaries respected: handler → service → repository (no skipping) - [] All external calls have circuit breakers via `digital.vasic.recovery` - [] All network calls have timeouts via context deadlines - [] Dependencies injected (no globals, no `init()` side effects)

Code Quality - [] `golangci-lint run ./...` passes with zero violations - [] All public functions/types have godoc comments - [] All error paths return wrapped errors (`fmt.Errorf("...: %w", err)`) - [] No `panic()` outside `cmd/main.go` startup validation - [] No TODO/FIXME in non-test code

Tests - [] Unit tests: ≥80% line coverage for all changed packages - [] Integration tests: all new repository methods covered - [] Meta-test mutation: new gates have paired mutations - [] Benchmarks: performance-sensitive paths have benchmarks

Security - [] No hardcoded credentials or secrets - [] SQL queries use parameterised form only - [] gRPC calls use mTLS (no `credentials.NewTLS(nil)`) - [] GOSEC scan clean

Constitution - [] `go run scripts/constitution-check.go --strict` score ≥95 - [] No vasic-digital functionality duplicated - [] `helix-deps.yaml` updated for new dependencies - [] Commit message follows conventional commits + helix extensions - [] Classification: line present in commit message

Context Window Management for Large Files

When working with large files (>500 lines), Claude MUST:

1. **Read the file structure first** — `head -100 <file>` and `grep -n "^func\|^type\|^var\|^const" <file>` to build a map before reading the full file.
2. **Work in focused sections** — never load >2000 lines into context simultaneously.
3. **Use grep/AST tools** — prefer `grep -n "<pattern>" <file>` over reading entire files for searches.
4. **For generated files** (protobuf stubs, `gen/go/`): NEVER edit directly; regenerate via `make proto-gen`.

5. **For go.sum:** NEVER edit directly; only go mod tidy or dependency updates should touch this file.
 6. **For large test files:** focus on the failing test only; do not rewrite unrelated tests in the same edit session.
 7. **Working with the 25 services:** when a change is cross-cutting, use subagent delegation per §11.4.20 — delegate one subagent per service, run in parallel, commit per service.
 8. **Monorepo go.work:** read the full file once per session; cache the module list; update when adding new modules.
-

Section 5: helix-deps_yaml Specification

Per Constitution §11.4.31, every owned-by-us submodule MUST ship helix-deps.yaml listing its own-org dependencies. The following is the **complete, deployable helix-deps.yaml** for HelixTerminator.

```
# helix-deps.yaml – HelixTerminator Submodule Dependency Manifest
# Per Constitution §11.4.31 (User mandate 2026-05-15)
#
# SCHEMA VERSION: 1.0
# This file lists ALL owned-by-us submodule dependencies for
#   HelixTerminator.
# It is the single source of truth for dependency graph
#   reconstruction.
#
# Tooling: incorporate-submodule <ssh-url> reads this file and
#   recurses.
# Anti-bluff: each manifest entry paired with a Challenge that
#   bootstraps
# from scratch, asserts layout matches manifest, runs submodule
#   tests,
# captures wire evidence (§11.4.31).

schema_version: "1.0"
project: helixterm
# module_path uses the pervasive `helixterm.io` Go-import
#   convention used throughout this
# document; reconciling it with the canonical primary domain
#   `helixterminator.io`
# (CANONICAL_FACTS.md CD-2) requires a mass import-path rewrite
#   and is
# `> DEFERRED (next increment)`.
module_path: helixterm.io
```

go_version: "1.25"
updated: "2026-06-28"

§1. CONSTITUTION SUBMODULE (mandatory for all HelixDevelopment projects)

constitution:

name: constitution
ssh_url: git@github.com:HelixDevelopment/HelixConstitution.git
https_url: https://github.com/HelixDevelopment/
HelixConstitution.git
ref: e6504c2 # pinned commit, helixcode-v1.1.0 line
(CANONICAL_FACTS.md CD-9)

layout: flat

canonical_path: constitution/

why: >

Universal engineering constitution. All projects must include this submodule per HelixDevelopment governance policy. Provides Constitution.md, AGENTS.md, CLAUDE.md, QWEN.md, submodules-catalogue.md, and enforcement scripts.

validation:

- assert_file_exists: constitution/Constitution.md
- assert_file_exists: constitution/AGENTS.md
- assert_file_exists: constitution/CLAUDE.md
- assert_git_commit: e6504c2 # helixcode-v1.1.0 line

§2. VASIC-DIGITAL SUBMODULE DEPENDENCIES

vasic_digital_dependencies:

- name: containers
ssh_url: git@github.com:vasic-digital/containers.git
https_url: https://github.com/vasic-digital/containers.git
ref: v1.3.0
layout: flat
canonical_path: submodules/containers/
why: >
Container orchestration substrate per §11.4.76 (containers mandate).
MUST be used for ALL containerised workloads. Provides rootless Podman


```

    support, container lifecycle management, health check
    integration.
go_module: digital.vasic.containers
used_by:
  - all 25 services (container builds)
  - CI/CD pipeline (test environment spin-up)
validation:
  - assert_file_exists: submodules/containers/README.md
  - run_tests: submodules/containers/
  - assert_rootless_mode: true

- name: observability
  ssh_url: git@github.com:vasic-digital/observability.git
  https_url: https://github.com/vasic-digital/observability.git
  ref: v2.1.0
  layout: flat
  canonical_path: submodules/observability/
  why: >
    Structured logging, distributed tracing (OpenTelemetry), and
    metrics
    collection. MUST be used by all services; reimplementation
    forbidden
    per §11.4.74. Provides zerolog-based structured logger, OTEL
    tracer,
    Prometheus metrics registry.
  go_module: digital.vasic.observability
  used_by:
    - all 25 services
    - helixterm.io/pkg/observability (thin wrapper)
  validation:
    - assert_file_exists: submodules/observability/README.md
    - run_tests: submodules/observability/
    - assert_no_import: github.com/sirupsen/logrus
    - assert_no_import: go.uber.org/zap

- name: auth
  ssh_url: git@github.com:vasic-digital/auth.git
  https_url: https://github.com/vasic-digital/auth.git
  ref: v3.0.1
  layout: flat
  canonical_path: submodules/auth/
  why: >
    JWT generation/validation, session token management, OAuth2
    flows,
    PKCE support. Used by auth-gateway service and middleware
    layer.

```

```

    EdDSA (Ed25519)-only JWT validation enforced
    (CANONICAL_FACTS.md CD-7).
go_module: digital.vasic.auth
used_by:
  - helixterm.io/services/auth-gateway
  - helixterm.io/pkg/middleware
validation:
  - assert_file_exists: submodules/auth/README.md
  - run_tests: submodules/auth/
  - assert_algorithm: EdDSA

- name: recovery
  ssh_url: git@github.com:vasic-digital/recovery.git
  https_url: https://github.com/vasic-digital/recovery.git
  ref: v1.5.2
  layout: flat
  canonical_path: submodules/recovery/
  why: >
    Circuit breakers, retry with exponential backoff, bulkhead
    patterns,
    timeout enforcement. Required on ALL external service calls.
    Prevents
    cascade failures across the 25-service topology.
  go_module: digital.vasic.recovery
  used_by:
    - all 25 services (circuit breakers on external calls)
  validation:
    - assert_file_exists: submodules/recovery/README.md
    - run_tests: submodules/recovery/

- name: ratelimiter
  ssh_url: git@github.com:vasic-digital/ratelimiter.git
  https_url: https://github.com/vasic-digital/ratelimiter.git
  ref: v1.2.0
  layout: flat
  canonical_path: submodules/ratelimiter/
  why: >
    Token bucket and sliding window rate limiters. Used by api-
    gateway
    and rate-limiter services. Redis-backed for distributed rate
    limiting.
  go_module: digital.vasic.ratelimiter
  used_by:
    - helixterm.io/services/api-gateway
    - helixterm.io/services/rate-limiter
  validation:

```

- `assert_file_exists:` `submodules/ratelimiter/README.md`
- `run_tests:` `submodules/ratelimiter/`

- `name:` `messaging`
`ssh_url:` `git@github.com:vasic-digital/Messaging.git`
`https_url:` `https://github.com/vasic-digital/Messaging.git`
`ref:` `v2.0.0`
`layout:` `flat`
`canonical_path:` `submodules/messaging/`
`why:` >
 Kafka producer/consumer abstractions, topic management,
 consumer group
 management, exactly-once semantics support. Used by event-
 router and
 all event-producing services.
`go_module:` `digital.vasic.messaging`
`used_by:`

- `helixterm.io/services/event-router`
- `helixterm.io/services/audit-logger`
- `helixterm.io/services/metrics-collector`
- `helixterm.io/services/telemetry-exporter`
- `helixterm.io/services/notification-dispatcher`

`validation:`

- `assert_file_exists:` `submodules/messaging/README.md`
- `run_tests:` `submodules/messaging/`
- `assert_kafka_version:` `">=3.9"`

- `name:` `database`
`ssh_url:` `git@github.com:vasic-digital/database.git`
`https_url:` `https://github.com/vasic-digital/database.git`
`ref:` `v1.4.0`
`layout:` `flat`
`canonical_path:` `submodules/database/`
`why:` >
 PostgreSQL connection pool management, query builder,
 migration runner,
 transaction management. All 25 services use this for
 database access.
 Prevents N+1 query patterns and ensures connection pool
 hygiene.
`go_module:` `digital.vasic.database`
`used_by:`

- `all 25 services (database access)`

`validation:`

- `assert_file_exists:` `submodules/database/README.md`
- `run_tests:` `submodules/database/`

- assert_postgres_version: ">=17.2"
- name: config
 - ssh_url: git@github.com:vasic-digital/config.git
 - https_url: https://github.com/vasic-digital/config.git
 - ref: v1.1.0
 - layout: flat
 - canonical_path: submodules/config/
 - why: >
 - Configuration loading from environment variables, files, and remote sources (Consul/etcd). Provides structured config validation. All services load configuration through this module.
 - go_module: digital.vasic.config
 - used_by:
 - all 25 services
 - helixterm.io/pkg/config
 - validation:
 - assert_file_exists: submodules/config/README.md
 - run_tests: submodules/config/
- name: middleware
 - ssh_url: git@github.com:vasic-digital/middleware.git
 - https_url: https://github.com/vasic-digital/middleware.git
 - ref: v1.3.0
 - layout: flat
 - canonical_path: submodules/middleware/
 - why: >
 - HTTP and gRPC middleware chain: request ID injection, authentication, authorisation, rate limiting, logging, tracing. Used by all services that expose HTTP or gRPC endpoints.
 - go_module: digital.vasic.middleware
 - used_by:
 - all services with HTTP or gRPC endpoints (22 of 25)
 - validation:
 - assert_file_exists: submodules/middleware/README.md
 - run_tests: submodules/middleware/
- name: security
 - ssh_url: git@github.com:vasic-digital/security.git
 - https_url: https://github.com/vasic-digital/security.git
 - ref: v2.0.0
 - layout: flat

```
canonical_path: submodules/security/
why: >
    Cryptographic primitives, secret scanning, input validation,
    SSRF
    prevention, command injection prevention. Used by key-vault,
    crypto-engine,
    and auth-gateway services.
go_module: digital.vasic.security
used_by:
    - helixterm.io/services/key-vault
    - helixterm.io/services/crypto-engine
    - helixterm.io/services/auth-gateway
validation:
    - assert_file_exists: submodules/security/README.md
    - run_tests: submodules/security/

- name: concurrency
  ssh_url: git@github.com:vasic-digital/concurrency.git
  https_url: https://github.com/vasic-digital/concurrency.git
  ref: v1.0.0
  layout: flat
  canonical_path: submodules/concurrency/
  why: >
      Generic concurrency utilities: worker pools, semaphores,
      fan-out/fan-in
      patterns, context-aware cancellation. Used by connection-
      broker and
      session-manager for managing concurrent connection
      lifecycle.
  go_module: digital.vasic.concurrency
  used_by:
      - helixterm.io/services/connection-broker
      - helixterm.io/services/session-manager
      - helixterm.io/services/load-balancer
  validation:
      - assert_file_exists: submodules/concurrency/README.md
      - run_tests: submodules/concurrency/

- name: cache
  ssh_url: git@github.com:vasic-digital/cache.git
  https_url: https://github.com/vasic-digital/cache.git
  ref: v1.2.0
  layout: flat
  canonical_path: submodules/cache/
  why: >
```

Redis-backed and in-memory caching with TTL, LRU eviction, and distributed invalidation. Used by auth-gateway (token cache), config-service (config cache), and rate-limiter (counter storage).

go_module: digital.vasic.cache

used_by:

- helixterm.io/services/auth-gateway
- helixterm.io/services/config-service
- helixterm.io/services/rate-limiter
- helixterm.io/services/session-manager

validation:

- assert_file_exists: submodules/cache/README.md
- run_tests: submodules/cache/

- name: discovery

ssh_url: git@github.com:vasic-digital/discovery.git

https_url: https://github.com/vasic-digital/discovery.git

ref: v1.0.0

layout: flat

canonical_path: submodules/discovery/

why: >

Service discovery via Consul and Kubernetes DNS. Used by node-manager and load-balancer for dynamic service endpoint resolution.

go_module: digital.vasic.discovery

used_by:

- helixterm.io/services/node-manager
- helixterm.io/services/load-balancer
- helixterm.io/services/dns-resolver

validation:

- assert_file_exists: submodules/discovery/README.md
- run_tests: submodules/discovery/

#

§3. HELIXDEVELOPMENT SUBMODULE DEPENDENCIES

#

helixdevelopment_dependencies:

- name: helix_qa

ssh_url: git@github.com:HelixDevelopment/helixqa.git

https_url: https://github.com/HelixDevelopment/helixqa.git

ref: v1.0.0

layout: flat

canonical_path: submodules/helix_qa/

```
why: >
  AI-driven QA orchestration for multi-platform testing.
  Required per
  §11.4.27 for full automation coverage. Provides autonomous
  QA sessions
  executing every registered test bank.
used_by:
  - tests/helix_qa/ (QA test execution)
  - CI/CD pipeline (automated QA gate)
validation:
  - assert_file_exists: submodules/helix_qa/README.md
  - run_tests: submodules/helix_qa/
```

```
- name: challenges
  ssh_url: git@github.com:vasic-digital/challenges.git
  https_url: https://github.com/vasic-digital/challenges.git
  ref: v1.0.0
  layout: flat
  canonical_path: submodules/challenges/
```

```
why: >
  Challenge-driven verification framework per §11.4.27. Every
  new feature
  must have a Challenge proving it works end-to-end. Provides
  challenge
  runner, evidence capture, and assertion framework.
used_by:
  - tests/challenges/ (challenge execution)
  - CI/CD pipeline (challenge verification gate)
validation:
  - assert_file_exists: submodules/challenges/README.md
  - run_tests: submodules/challenges/
```

```
# -----
# §4. DEPENDENCY GRAPH VALIDATION RULES
# -----
```

```
validation_rules:
```

```
# Rule 1: No nested own-org submodule chains (§11.4.28-C)
```

```
no_nested_chains:
```

```
  enabled: true
```

```
  description: >
```

```
    All submodule dependencies MUST be at <root>/<name>/ or
    <root>/submodules/<name>/. No submodule may declare own-org
    submodule dependencies that are not already at the root.
```

```
  enforcement: incorporate-submodule tooling validates on add
```

```

# Rule 2: No conflicting refs for the same submodule
no_conflicting_refs:
  enabled: true
  description: >
    If two entries reference the same ssh_url, they MUST have
    the same
    ref. Conflicting refs abort the incorporate-submodule
    operation.
  enforcement: incorporate-submodule tooling validates on add

# Rule 3: All entries must have validation blocks
validation_blocks_required:
  enabled: true
  description: >
    Every dependency entry MUST have at least one validation
    step.
    Anti-bluff: the manifest without validation is a §11.4.31
    violation.
  enforcement: constitution-check.go --rule helix-deps-
    validation

# Rule 4: Layout must be flat or grouped (no other values)
layout_values:
  enabled: true
  allowed: [flat, grouped]
  enforcement: JSON schema validation in CI

# Rule 5: refs must be exact tags or SHAs (no branch names in
# production)
ref_format:
  enabled: true
  description: >
    In production: refs MUST be semver tags (v1.2.3) or full
    SHA-256.
    Branch names (main, develop) are allowed only in development
    helix-deps.yaml variants (helix-deps.dev.yaml).
  enforcement: constitution-check.go --rule helix-deps-ref-
    format

# -----
# §5. UPDATE POLICY
# -----
update_policy:
  frequency: monthly_minimum
  process:
    - step: 1
      action: >

```



```
Run `gh repo view <org>/<repo> --json latestRelease` for
each entry
to check for newer releases.
- step: 2
  action: >
    Review CHANGELOG of each submodule for breaking changes.
- step: 3
  action: >
    Update ref in helix-deps.yaml, run full test suite, verify
    no regressions.
- step: 4
  action: >
    Submit PR with title: `chore: update submodule
    dependencies to <date>`
    with Classification: universal (§11.4.17) in commit
    message.
- step: 5
  action: >
    Ensure all downstream tests pass before merge.
emergency_updates:
  description: >
    CVE fixes or critical bug fixes in dependencies must be
    fast-tracked.
    Emergency update PRs skip the monthly cycle and are merged
    same-day.
  required_review: security-team
  required_tests: full_suite
```

Section 6: Mandatory Test Types per Constitution

Per §11.4.27, the codebase MUST be covered by EVERY supported test type. The following documents all mandatory test types, their constitution rule references, HelixTerminator-specific requirements, pass/fail criteria, and CI gates.

6.1 Unit Tests

Constitution rule: §1, §11.4.27-A

Classification: Mandatory for all packages

Rules: - Test file naming: <source_file>_test.go in same package -
Mocks and stubs PERMITTED in unit tests - MUST NOT spin up external
services, network, or file system - Test function naming:

Test<Type>_<Method>_<Scenario>_<ExpectedOutcome> - MUST run with
-race flag - Tables-driven tests preferred for multiple input/output
scenarios

Coverage thresholds: | Package type | Line | Branch | Function | |
—|—|—|—| | handler/ | ≥85% | ≥75% | ≥90% | | service/ | ≥85% |
≥75% | ≥90% | | repository/ | ≥80% | ≥70% | ≥85% | | domain/ | ≥90%
| ≥80% | ≥95% | | cmd/ | ≥70% | ≥60% | ≥75% | | pkg/ | ≥85% | ≥75%
| ≥90% |

Pass/fail criteria: Zero failures; coverage thresholds met; zero race
conditions detected

CI gate: unit-tests stage (runs on every PR push)

Full example:

```
// File: services/session-manager/internal/service/session_test.go
package service_test

import (
    "context"
    "testing"
    "time"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"
    "google.golang.org/protobuf/types/known/timestamppb"

    pb "helixterm.io/services/session-manager/gen/go/session/v1"
    "helixterm.io/services/session-manager/internal/domain"
    "helixterm.io/services/session-manager/internal/service"
)

// mockSessionRepository implements domain.SessionRepository for
// unit tests.
type mockSessionRepository struct {
    sessions    map[string]*domain.Session
    saveErr     error
    getErr      error
}

func (m *mockSessionRepository) Save(ctx context.Context, s
    *domain.Session) error {
    if m.saveErr != nil {
        return m.saveErr
    }
}
```

```

    m.sessions[s.ID] = s
    return nil
}

func (m *mockSessionRepository) GetByID(ctx context.Context, id
    string) (*domain.Session, error) {
    if m.getErr != nil {
        return nil, m.getErr
    }
    s, ok := m.sessions[id]
    if !ok {
        return nil, domain.ErrSessionNotFound
    }
    return s, nil
}

func TestSessionService_CreateSession_ValidRequest_ReturnsPersisted(t
    *testing.T) {
    t.Parallel()

    // Arrange
    repo := &mockSessionRepository{sessions:
        make(map[string]*domain.Session)}
    svc := service.New(repo, service.Config{
        DefaultTTL: 15 * time.Minute,
    })
    req := &pb.CreateSessionRequest{
        UserID:    "user-123",
        Protocol:  pb.Protocol_WIREGUARD,
        NodeID:    "node-456",
    }

    // Act
    resp, err := svc.CreateSession(context.Background(), req)

    // Assert
    require.NoError(t, err)
    require.NotNil(t, resp)
    assert.NotEmpty(t, resp.SessionID)
    assert.Equal(t, pb.SessionStatus_ACTIVE, resp.Status)
    assert.WithinDuration(t, time.Now().Add(15*time.Minute),
        resp.ExpiresAt.AsTime(), 5*time.Second)

    // Verify persistence
    require.Len(t, repo.sessions, 1)
    saved := repo.sessions[resp.SessionID]

```

```

    assert.Equal(t, "user-123", saved.UserID)
    assert.Equal(t, domain.ProtocolWireGuard, saved.Protocol)
}

func TestSessionService_CreateSession_RepositoryError_ReturnsWrappedError(t
    *testing.T) {
    t.Parallel()

    // Arrange
    repo := &mockSessionRepository{saveErr: domain.ErrStorageFull}
    svc := service.New(repo, service.Config{DefaultTTL: 15 *
        time.Minute})

    // Act
    _, err := svc.CreateSession(context.Background(),
        &pb.CreateSessionRequest{
            UserID: "user-123", Protocol: pb.Protocol_WIREGUARD,
            NodeID: "node-456",
        })

    // Assert
    require.Error(t, err)
    assert.ErrorIs(t, err, domain.ErrStorageFull,
        "error must wrap domain.ErrStorageFull for caller to act
        on it")
}

func TestSessionService_GetSession_NonexistentID_ReturnsNotFound(t
    *testing.T) {
    t.Parallel()

    // Arrange
    repo := &mockSessionRepository{sessions:
        make(map[string]*domain.Session)}
    svc := service.New(repo, service.Config{DefaultTTL: 15 *
        time.Minute})

    // Act
    _, err := svc.GetSession(context.Background(), "nonexistent-
        id")

    // Assert
    require.Error(t, err)
    assert.ErrorIs(t, err, domain.ErrSessionNotFound)
}

```

6.2 Integration Tests

Constitution rule: §11.4.27, §11.4.25

Classification: Mandatory for every service

Rules: - Build tag: `//go:build integration` - MUST use REAL services — no mocks, no stubs, no fake HTTP servers - Infrastructure spun up via `vasic-digital/containers` submodule - Test databases MUST be fresh per test (use transactions or truncation) - Tests MUST be idempotent — repeatable without manual cleanup - MUST test all repository methods with real database - MUST test all service methods with real dependencies

Allowed external dependencies: - PostgreSQL (via `containers` submodule) - Redis (via `containers` submodule) - Kafka (via `containers` submodule) - gRPC stub servers from other services (real binary, not mock)

Pass/fail criteria: - All repository CRUD operations verified against real DB - All Kafka producer/consumer operations verified with real broker - All cache operations verified with real Redis - Zero data leaks between test runs

CI gate: `integration-tests` stage (runs on every PR; parallelised per service)

Full example:

```
//go:build integration
// +build integration

// File: services/session-manager/tests/integration/
//       session_repository_test.go
package integration_test

import (
    "context"
    "testing"
    "time"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"

    "helixterm.io/services/session-manager/internal/domain"
    "helixterm.io/services/session-manager/internal/repository"
```

```

    "helixterm.io/services/session-manager/tests/integration/
      testenv"
)

func TestSessionRepository_Save_GetByID_RoundTrip(t *testing.T) {
    t.Parallel()

    // Arrange: real PostgreSQL via containers submodule
    env := testenv.New(t) // spins up real DB, runs migrations,
                          // provides cleanup
    repo := repository.NewPostgreSQL(env.DB)

    session := &domain.Session{
        ID:          "test-session-" + t.Name(),
        UserID:       "user-integration-test",
        Protocol:     domain.ProtocolWireGuard,
        NodeID:       "node-001",
        Status:       domain.SessionStatusActive,
        CreatedAt:    time.Now().UTC().Truncate(time.Microsecond),
        ExpiresAt:    time.Now().Add(15 *
            time.Minute).UTC().Truncate(time.Microsecond),
    }

    // Act: Save
    err := repo.Save(env.Ctx, session)
    require.NoError(t, err, "save must succeed with real
        database")

    // Act: Get
    retrieved, err := repo.GetByID(env.Ctx, session.ID)
    require.NoError(t, err, "get must succeed for saved session")

    // Assert: round-trip fidelity
    assert.Equal(t, session.ID, retrieved.ID)
    assert.Equal(t, session.UserID, retrieved.UserID)
    assert.Equal(t, session.Protocol, retrieved.Protocol)
    assert.Equal(t, session.Status, retrieved.Status)
    assert.WithinDuration(t, session.CreatedAt,
        retrieved.CreatedAt, time.Millisecond)

    // EVIDENCE: verify directly in database
    var count int
    err = env.DB.QueryRowContext(env.Ctx,
        "SELECT COUNT(*) FROM sessions WHERE id = $1",
        session.ID).Scan(&count)
    require.NoError(t, err)

```

```

    assert.Equal(t, 1, count, "EVIDENCE: session physically
        present in PostgreSQL")
    t.Logf("EVIDENCE: session %s verified in PostgreSQL at %v",
        session.ID, time.Now())
}

```

6.3 End-to-End (E2E) Tests

Constitution rule: §11.4.25, §11.4.27

Classification: Mandatory — one E2E suite per user-facing flow

Rules: - Build tag: `//go:build e2e` - Tests run against a fully deployed staging environment - MUST test complete user journeys, not individual service methods - MUST verify observable outcomes (data in DB, events in Kafka, UI state) - No shortcuts — use actual API Gateway endpoint, not internal endpoints - Tests MUST be executable from CI without human interaction

Required E2E scenarios for HelixTerminator: 1. User registration → email verification → first login 2. User login → session creation → connection establishment → session termination 3. Protocol negotiation → downgrade path → reconnection 4. Subscription creation → billing event → service activation 5. Node registration → health check → load balancing verification 6. Rate limiting → threshold breach → recovery 7. Auth token expiry → refresh → continuation 8. Audit log trail for complete session lifecycle

Pass/fail criteria: - All 8 required scenarios pass - P99 latency within thresholds documented in Section 3 - No error log entries at ERROR level during happy-path scenarios

CI gate: `e2e-tests` stage (runs on develop branch merge and release candidates)

Full example:

```

//go:build e2e
// +build e2e

// File: tests/e2e/session_lifecycle_test.go
package e2e_test

import (
    "context"
    "fmt"
    "testing"

```

```

"time"

"github.com/stretchr/testify/assert"
"github.com/stretchr/testify/require"

"helixterm.io/tests/e2e/client"
"helixterm.io/tests/e2e/env"
)

// TestE2E_SessionLifecycle_FullHappyPath verifies the complete
// session
// lifecycle: create → connect → transfer → terminate → audit
// trail.
func TestE2E_SessionLifecycle_FullHappyPath(t *testing.T) {
    // Not parallel: E2E tests use shared staging environment
    e := env.ForE2E(t) // staging environment config from
        E2E_STAGING_URL env var
    c := client.New(e.APIURL, e.TLSConfig)

    // Step 1: Authenticate
    token, err := c.Auth.Login(e.Ctx, e.TestUser.Email,
        e.TestUser.Password)
    require.NoError(t, err, "authentication must succeed")
    require.NotEmpty(t, token.AccessToken)
    t.Logf("EVIDENCE: authenticated as %s, token prefix: %s",
        e.TestUser.Email, token.AccessToken[:8])

    // Step 2: Create session
    sessionResp, err := c.Session.Create(e.Ctx,
        client.CreateSessionRequest{
            Protocol: "wireguard",
            NodeID:   e.NearestNodeID,
        }, token)
    require.NoError(t, err, "session creation must succeed")
    require.NotEmpty(t, sessionResp.SessionID)
    t.Logf("EVIDENCE: session %s created", sessionResp.SessionID)

    // Step 3: Verify session active in session-manager
    time.Sleep(500 * time.Millisecond) // allow propagation
    status, err := c.Session.GetStatus(e.Ctx,
        sessionResp.SessionID, token)
    require.NoError(t, err)
    assert.Equal(t, "active", status.State,
        "EVIDENCE: session must be ACTIVE after creation")

    // Step 4: Verify connection established in connection-broker

```



```

connStatus, err := c.Connection.GetStatus(e.Ctx,
    sessionResp.SessionID, token)
require.NoError(t, err)
assert.Equal(t, "established", connStatus.State,
    "EVIDENCE: connection must be ESTABLISHED")

// Step 5: Terminate session
err = c.Session.Terminate(e.Ctx, sessionResp.SessionID, token)
require.NoError(t, err, "session termination must succeed")

// Step 6: Verify terminated
time.Sleep(500 * time.Millisecond)
status, err = c.Session.GetStatus(e.Ctx,
    sessionResp.SessionID, token)
require.NoError(t, err)
assert.Equal(t, "terminated", status.State,
    "EVIDENCE: session must be TERMINATED after explicit
    termination")

// Step 7: Verify audit log trail
auditEntries, err := c.Audit.GetBySession(e.Ctx,
    sessionResp.SessionID, token)
require.NoError(t, err)
events := extractEventTypes(auditEntries)
assert.Contains(t, events, "session.created",
    "EVIDENCE: audit log must contain session.created event")
assert.Contains(t, events, "connection.established",
    "EVIDENCE: audit log must contain connection.established
    event")
assert.Contains(t, events, "session.terminated",
    "EVIDENCE: audit log must contain session.terminated
    event")
t.Logf("EVIDENCE: audit trail verified: %v", events)
}

func extractEventTypes(entries []client.AuditEntry) []string {
    types := make([]string, len(entries))
    for i, e := range entries {
        types[i] = e.EventType
    }
    return types
}

```

6.4 Contract Tests (Pact)

Constitution rule: §11.4.25, §11.4.27

Classification: Mandatory for all service-to-service interfaces

Rules: - Build tag: `//go:build contract` - EVERY service that calls another service MUST have a Pact consumer contract - EVERY service that is called MUST verify all consumer contracts (provider side) - Pact broker URL: configured in CI via `PACT_BROKER_URL` environment variable - Consumer contracts committed to repository in `tests/contract/pacts/`

Required contract pairs:

Consumer	Provider	Interface
api-gateway	auth-gateway	gRPC AuthService
api-gateway	session-manager	gRPC SessionService
api-gateway	user-registry	gRPC UserService
session-manager	connection-broker	gRPC ConnectionService
session-manager	node-manager	gRPC NodeService
connection-broker	protocol-handler	gRPC ProtocolService
auth-gateway	key-vault	gRPC VaultService
event-router	all 25 services	Kafka topic schemas
Flutter client	api-gateway	REST + WebSocket

Pass/fail criteria: - All consumer contract tests pass (consumer side) - All provider contract verification tests pass (provider side) - No contract version mismatches between consumer and provider

CI gate: contract-tests stage (runs on every PR)

Full example:

```
//go:build contract
// +build contract

// File: services/api-gateway/tests/contract/
//       auth_gateway_consumer_test.go
package contract_test

import (
    "context"
    "fmt"
```

```

"testing"

"github.com/pact-foundation/pact-go/v2/consumer"
"github.com/pact-foundation/pact-go/v2/matchers"
"github.com/stretchr/testify/require"

authpb "helixterm.io/services/auth-gateway/gen/go/auth/v1"
"helixterm.io/services/api-gateway/internal/clients"
)

func TestContractConsumer_APIGateway_AuthGateway_ValidateToken(t
    *testing.T) {
    pact, err :=
        consumer.NewV4Pact(consumer.MockHTTPProviderConfig{
            Consumer: "helixterm-api-gateway",
            Provider: "helixterm-auth-gateway",
            PactDir: "tests/contract/pacts",
        })
    require.NoError(t, err)
    defer pact.WritePact()

    // Define the interaction
    pact.AddInteraction().
        Given("a valid JWT token exists").
        UponReceiving("a ValidateToken request with a valid JWT").
        WithRequest(consumer.Request{
            Method: "POST",
            Path: matchers.String("/auth.v1.AuthService/
ValidateToken"),
            Headers: matchers.MapMatcher{
                "Content-Type": matchers.String("application/
grpc"),
            },
        }).
        WillRespondWith(consumer.Response{
            Status: 200,
            Body: matchers.MapMatcher{
                "userID": matchers.Like("user-123"),
                "valid": matchers.Like(true),
                "expiresAt":
matchers.Like("2026-12-31T23:59:59Z"),
            },
        })

    // Run the test

```

```

err = pact.ExecuteTest(t, func(config
    consumer.MockServerConfig) error {
    client := clients.NewAuthGatewayClient(
        fmt.Sprintf("localhost:%d", config.Port),
    )
    resp, err := client.ValidateToken(context.Background(),
        "valid-jwt-token")
    require.NoError(t, err)
    require.NotNil(t, resp)
    return nil
})
require.NoError(t, err)
}

```

6.5 Performance Tests

Constitution rule: §11.4.25, §11.4.27

Classification: Mandatory; thresholds defined in AGENTS.MD Section 3

Tool: k6 with Go-based benchmarks for library-level performance

Required k6 scripts (in tests/performance/):

Script	Target	Scenario
k6_auth_gateway.js	auth-gateway	1000 RPS, 5-min sustained load
k6_session_manager.js	session-manager	500 concurrent sessions
k6_connection_broker.js	connection-broker	2000 concurrent connections
k6_api_gateway.js	api-gateway	500 RPS mixed workload
k6_full_stack.js	complete stack	200 concurrent users

Pass/fail thresholds:

```

// tests/performance/thresholds.js (shared across all k6 scripts)
export const thresholds = {
    'http_req_duration{expected_response:true}':
        ['p(99)<50'], // p99 < 50ms for HTTP
    'grpc_req_duration':
        ['p(99)<10'], // p99 < 10ms for gRPC
}

```

```

    'http_req_failed':
        ['rate<0.001'], // < 0.1% error
        rate
    'grpc_req_failed': ['rate<0.001'],
    'checks':
        ['rate>0.999'], // >
        99.9% check pass rate
};

```

Full k6 example:

```

// File: tests/performance/k6_session_manager.js
import http from 'k6/http';
import { check, sleep } from 'k6';
import { Counter, Rate, Trend } from 'k6/metrics';
import { thresholds } from './thresholds.js';

const sessionCreations = new Counter('session_creations');
const sessionCreationDuration = new
    Trend('session_creation_duration_ms');
const sessionCreationErrors = new
    Rate('session_creation_error_rate');

export const options = {
    scenarios: {
        sustained_load: {
            executor: 'constant-arrival-rate',
            rate: 500,
            timeUnit: '1s',
            duration: '5m',
            preAllocatedVUs: 50,
            maxVUs: 200,
        },
        spike: {
            executor: 'ramping-arrival-rate',
            startRate: 50,
            stages: [
                { target: 2000, duration: '30s' },
                { target: 2000, duration: '1m' },
                { target: 50, duration: '30s' },
            ],
        },
    },
    thresholds: {
        ...thresholds,
        session_creation_duration_ms: ['p(99)<20'], // session
        creation p99 < 20ms
    }
};

```

```

        session_creation_error_rate: ['rate<0.001'],
    },
};

const BASE_URL = __ENV.SESSION_MANAGER_URL || 'http://
    localhost:8081';

export default function () {
    const token = getAuthToken(); // pre-generated in setup()

    const startTime = Date.now();
    const response = http.post(
        `${BASE_URL}/api/v1/sessions`,
        JSON.stringify({
            user_id: `user-${__VU}`,
            protocol: 'wireguard',
            node_id: 'node-001',
        }),
        {
            headers: {
                'Content-Type': 'application/json',
                'Authorization': `Bearer ${token}`,
            },
            timeout: '5s',
        }
    );

    const duration = Date.now() - startTime;

    sessionCreations.add(1);
    sessionCreationDuration.add(duration);

    const success = check(response, {
        'status is 201': (r) => r.status === 201,
        'has session_id': (r) => r.json('session_id') !== '',
        'has status active': (r) => r.json('status') === 'active',
        'response time < 50ms': () => duration < 50,
    });

    if (!success) {
        sessionCreationErrors.add(1);
    }

    sleep(0.1);
}

```

```

function getAuthToken() {
    // In real tests, tokens are pre-generated in setup() and
    // stored in shared state
    return __ENV.TEST_AUTH_TOKEN || 'test-token';
}

```

Go benchmark example:

```

// File: services/session-manager/internal/service/bench_test.go
package service_test

import (
    "context"
    "testing"

    "helixterm.io/services/session-manager/internal/service"
    "helixterm.io/services/session-manager/tests/benchutil"
)

func BenchmarkSessionService_CreateSession_Sequential(b
    *testing.B) {
    svc := benchutil.NewServiceWithRealDeps(b) // real DB, real
    Redis
    req := benchutil.ValidCreateSessionRequest()

    b.ResetTimer()
    b.ReportAllocs()
    for i := 0; i < b.N; i++ {
        _, err := svc.CreateSession(context.Background(), req)
        if err != nil {
            b.Fatal(err)
        }
    }
}

func BenchmarkSessionService_CreateSession_Parallel(b *testing.B)
    {
        svc := benchutil.NewServiceWithRealDeps(b)
        req := benchutil.ValidCreateSessionRequest()

        b.ResetTimer()
        b.ReportAllocs()
        b.RunParallel(func(pb *testing.PB) {
            for pb.Next() {
                _, err := svc.CreateSession(context.Background(), req)

```

```

        if err != nil {
            b.Fatal(err)
        }
    }
}
})
}

```

CI gate: performance-tests stage (runs on release candidate branches and develop weekly)

6.6 Security Tests

Constitution rule: §11.4.10, §11.4.25, §11.4.27

Classification: Mandatory

Types:

Type	Tool	Frequency
SAST (static)	gosec	Every PR
Dependency scanning	govulncheck + trivy	Every PR
DAST (dynamic)	OWASP ZAP	Nightly + release
Secret scanning	truffleHog + gitleaks	Every commit (pre-commit hook)
Container scanning	trivy on images	Every image build
License scanning	golicense	Weekly

Pass/fail criteria: - SAST: zero HIGH or CRITICAL findings -
 Dependency scan: zero CVEs with CVSS ≥ 7.0 in direct dependencies -
 Secret scan: zero detected secrets - Container scan: zero CVEs in base images with CVSS ≥ 7.0

CI gate: security-scan stage (blocks merge on HIGH/CRITICAL findings)

SAST configuration:

```

# .gosec.yaml
global:
  audit: false
  nsec: false
  tests: false

```


rules:

```
G101: enabled # Hardcoded credentials
G102: enabled # Bind to all interfaces
G103: enabled # Audit use of unsafe block
G104: enabled # Errors unhandled
G106: enabled # Audit SSH hostkey
G107: enabled # URL provided to HTTP request as taint input
G108: enabled # Profiling endpoint
G109: enabled # Potential integer overflow
G110: enabled # Potential DoS via decompression bomb
G111: enabled # Directory traversal
G112: enabled # Slowloris
G114: enabled # Unsafe http.Serve
G201: enabled # SQL query construction
G202: enabled # SQL query construction
G203: enabled # Use of unescaped data in HTML templates
G204: enabled # Audit use of command execution
G301: enabled # Poor file permissions
G302: enabled # Poor file permissions
G303: enabled # Creating tempfile using a predictable path
G304: enabled # File path provided as taint input
G305: enabled # File traversal when extracting zip archive
G306: enabled # Poor file permissions
G401: enabled # Use of weak cryptographic primitive (MD5/SHA1)
G402: enabled # TLS bad configuration
G403: enabled # TLS minimum version
G404: enabled # Use of weak random number generator
G501: enabled # Blocklisted import - crypto/md5
G502: enabled # Blocklisted import - crypto/des
G503: enabled # Blocklisted import - crypto/rc4
G504: enabled # Blocklisted import - net/http/cgi
G505: enabled # Blocklisted import - crypto/sha1
G601: enabled # Implicit memory aliasing
```

6.7 Mutation Tests

Constitution rule: §1.1, §11.4.25

Classification: Mandatory; every gate **MUST** have a paired mutation

Tool: go-mutesting OR mutagen

Threshold: $\geq 75\%$ mutation score per package (number of killed mutants / total mutants)

Mutation operators required: - Arithmetic operator replacement (+ → -, * → /) - Relational operator replacement (< → <=, == → !=) - Logical operator replacement (&& → ||) - Statement deletion (delete a statement, check tests fail) - Return value mutation (return nil instead of value, vice versa) - Off-by-one mutation (boundary conditions)

Pass/fail criteria: - Mutation score ≥75% per package - Every CI gate has a corresponding mutation test that proves the gate catches regressions (§1.1) - Mutation score < 75% blocks release (soft warn on PR, hard fail on release)

CI gate: mutation-tests stage (runs on release candidates, weekly on develop)

Meta-test mutation example:

```
// File: tests/meta/gates_mutation_test.go
package meta_test

import (
    "os/exec"
    "strings"
    "testing"
)

// TestMetaMutation_CoverageGate verifies the coverage gate
// catches low coverage.
// This is the PAIRED MUTATION for the CM-COVERAGE-THRESHOLD gate.
func TestMetaMutation_CoverageGate(t *testing.T) {
    // Run coverage check with a package that has artificially low
    // coverage
    cmd := exec.Command("go", "run", "scripts/constitution-
        check.go",
        "--check", "coverage",
        "--mock-coverage", "50.0", // simulate 50% coverage
    )
    output, err := cmd.CombinedOutput()

    // The gate MUST fail when coverage is below threshold
    if err == nil {
        t.Fatalf("META-MUTATION FAIL: coverage gate did not fail
            on 50%% coverage.\n"+
            "Output: %s\n"+
            "This means the coverage gate is a bluff gate (§1.1
            violation).", string(output))
    }
}
```

```

    if !strings.Contains(string(output), "FAIL") {
        t.Fatalf("META-MUTATION FAIL: gate exited with error but
            output does not contain FAIL.\n"+
                "Output: %s", string(output))
    }

    t.Logf("META-MUTATION PASS: coverage gate correctly catches
        50%% coverage: %s",
            string(output))
}

```

6.8 Chaos Tests

Constitution rule: §11.4.25

Classification: Mandatory — specified failure scenarios must be tested

Tool: chaos-mesh on staging Kubernetes cluster

Required chaos scenarios:

Scenario	Target	Expected behaviour
Pod kill: auth-gateway	auth-gateway deployment	Other services degrade gracefully; no panic; circuit breakers open
Network partition: session-manager ↔ PostgreSQL	session-manager DB connection	Connection retries with backoff; errors surfaced to client with appropriate code
CPU throttle: connection-broker	connection-broker pods	Graceful degradation; p99 latency increases but below 2× baseline
Memory pressure: node-manager	node-manager pods	OOM handling; service restarts cleanly; no data corruption
Kafka broker kill	Kafka cluster (1 of 3 brokers)	Producers retry; consumers rebalance; no message loss
Redis unavailability: cache	Redis pod	Services fall back to DB; latency increases; circuit breaker opens
Service mesh: drop 10% of gRPC packets	Istio virtualservice	Retry policies engage; error rate <1% end-to-end

Scenario	Target	Expected behaviour
Node drain	Worker node with 5 services	Pods reschedule; PDBs prevent availability loss; zero downtime

Pass/fail criteria: - No service produces panics during chaos injection - Error rates during chaos < 5% (graceful degradation) - Services recover to normal within 60 seconds of chaos removal - No data corruption detected in post-chaos consistency checks

CI gate: chaos-tests stage (runs weekly on staging, always on release candidates)

6.9 Fuzz Tests

Constitution rule: §11.4.25

Classification: Mandatory for parsing, deserialization, and cryptographic functions

Tool: Go native fuzzing (testing.F)

Required fuzz targets:

Function	Service	Why
JWT parsing	auth-gateway	Malformed tokens must not panic or cause unexpected behaviour
Protocol negotiation	protocol-handler	Malformed client hello must not crash
Config deserialization	config-service	Malformed YAML/JSON must not cause unexpected state
gRPC request parsing	all services	Malformed protobuf

Function	Service	Why
Database query builder	pkg/database wrapper	must not panic SQL injection via unusual Unicode sequences
Kafka message deserialization	event-router	Malformed Avro/JSON must not crash consumer

Example fuzz test:

```
// File: services/auth-gateway/internal/service/fuzz_test.go
package service_test

import (
    "context"
    "testing"

    "helixterm.io/services/auth-gateway/internal/service"
    "helixterm.io/services/auth-gateway/tests/testutil"
)

// FuzzValidateToken ensures ValidateToken never panics on
// arbitrary input.
func FuzzValidateToken(f *testing.F) {
    svc := testutil.NewServiceWithRealDeps(f)

    // Seed corpus: valid JWT, empty string, malformed header,
    // truncated token
    f.Add("eyJhbGciOiJSUzI1NiJ9.eyJzdWIiOiJ1c2VyLTEyMyJ9.signature")
    f.Add("")
    f.Add("not.a.jwt")
    f.Add("eyJhbGciOiJub251In0.eyJzdWIiOiJhZG1pbiJ9.") //
    // algorithm=none attack
    f.Add(string(make([]byte, 65536))) // very
    // long token

    f.Fuzz(func(t *testing.T, token string) {
        // Must not panic
        _, _ = svc.ValidateToken(context.Background(), token)
    })
}
```

```

        // Any error is acceptable; panic is not.
    })
}

```

CI gate: fuzz-tests stage (runs for 5 minutes on every PR, extended 30-min run on releases)

6.10 Golden Tests

Constitution rule: §11.4.25

Classification: Mandatory for deterministic outputs

Required golden files (in tests/golden/):

Output	Service	Update trigger
OpenAPI spec JSON	api-gateway	Any API change
Protobuf descriptor	all services	Any .proto change
Default config YAML	config-service	Any config schema change
Kafka topic manifest	event-router	Any topic addition/change
Error message catalogue	pkg/errors	Any error code change
gRPC health check response	all services	Any health endpoint change

Golden test example:

```

// File: services/api-gateway/tests/golden/openapi_test.go
package golden_test

import (
    "flag"
    "os"
    "testing"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"

    "helixterm.io/services/api-gateway/internal/spec"

```

```

)

var update = flag.Bool("update-golden", false, "Update golden
    files")

func TestOpenAPISpec_MatchesGoldenFile(t *testing.T) {
    generated, err := spec.GenerateOpenAPI()
    require.NoError(t, err)

    goldenPath := "tests/golden/openapi.json"

    if *update {
        err = os.WriteFile(goldenPath, generated, 0644)
        require.NoError(t, err, "golden file update must succeed")
        t.Logf("GOLDEN: updated %s", goldenPath)
        return
    }

    expected, err := os.ReadFile(goldenPath)
    require.NoError(t, err, "golden file must exist; run with
        -update-golden to create it")

    assert.JSONEq(t, string(expected), string(generated),
        "OpenAPI spec must match golden file. "+
        "If this is intentional, run: go test ./tests/golden/...
        -update-golden")
}

```

CI gate: golden-tests stage (every PR; failure with diff output)

6.11 Accessibility Tests

Constitution rule: §11.4.162, §11.4.170

Classification: Mandatory for Flutter client; WCAG 2.1 AA required

Tools: - Flutter: flutter_accessibility_tools package + Semantics assertions - Visual regression: Roborazzi / Paparazzi (per §11.4.170 — device-independent host-rendered pixel proof)

Requirements: - All interactive elements must have semantic labels - Contrast ratio $\geq 4.5:1$ for normal text, $\geq 3:1$ for large text (WCAG 2.1 AA) - All screens must have light AND dark mode rendered pixel tests - OCR/ vision oracle verifies rendered text + labels + control bounds - No element overlap or label-over-label scenarios

Accessibility test example:

```
// File: client/test/accessibility/session_screen_a11y_test.dart
import 'package:flutter/material.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:helixterm_client/features/session/
    session_screen.dart';

void main() {
  group('SessionScreen accessibility', () {
    testWidgets('meets WCAG 2.1 AA – no accessibility
        violations', (tester) async {
      await tester.pumpWidget(
        MaterialApp(
          theme: ThemeData.light(),
          home: const SessionScreen(),
        ),
      );

      // Verify all interactive elements have semantic labels
      final connectButton = find.bySemanticsLabel('Connect');
      expect(connectButton, findsOneWidget,
        reason: 'Connect button must have semantic label for
            screen readers');

      final disconnectButton =
        find.bySemanticsLabel('Disconnect');
      expect(disconnectButton, findsOneWidget);

      // Verify no overlapping elements
      final renderObjects = tester.allRenderObjects.toList();
      for (final obj in renderObjects) {
        // No render object should have zero size if visible
        if (obj.paintBounds.isEmpty) {
          // Invisible elements are acceptable
          continue;
        }
      }
    }

    // Verify accessibility tree
    await expectLater(tester,
      meetsGuideline(androidTapTargetGuideline));
    await expectLater(tester,
      meetsGuideline(iOSTapTargetGuideline));
    await expectLater(tester,
      meetsGuideline(labeledTapTargetGuideline));
  });
}
```



```

        await expectLater(tester,
            meetsGuideline(textContrastGuideline));
    });

    testWidgets('dark mode – no accessibility violations',
        (tester) async {
        await tester.pumpWidget(
            MaterialApp(
                theme: ThemeData.dark(),
                home: const SessionScreen(),
            ),
        );

        await expectLater(tester,
            meetsGuideline(textContrastGuideline));
    });
});
}

```

CI gate: accessibility-tests stage (runs on every Flutter client change)

6.12 Smoke Tests

Constitution rule: §11.4.25, §11.4.38

Classification: Mandatory production readiness gate

Purpose: Verify basic system health immediately after deployment.

Execution time: < 5 minutes

Environment: Runs against production-equivalent environment

Required smoke test checks:

Check	Service	Pass Criterion
/health/live	All 25 services	HTTP 200 within 3 seconds
/health/ready	All 25 services	HTTP 200 with {"status": "ready"}
Kafka connectivity	event-router	Can produce and consume a test message
Database connectivity	All DB-dependent services	Can execute SELECT 1 successfully
Redis connectivity	auth-gateway, session-manager	Can set and get a test key
		grpc_cli ls returns service list

Check	Service	Pass Criterion
gRPC reflection	All services with gRPC	
Auth flow	auth-gateway	Can authenticate test user and receive valid token
Session flow	session-manager	Can create and immediately terminate a test session

Pass/fail criteria: - 100% of health endpoints respond within 3 seconds
 - Auth flow completes in < 1 second - Zero connectivity failures to any infrastructure dependency

CI gate: smoke-tests stage (runs on EVERY deployment to staging and production; blocks promotion on failure)

Smoke test script:

```
#!/usr/bin/env bash
# scripts/smoke-test.sh
# Purpose: Post-deployment smoke test for all 25 HelixTerminator
#          services
# Usage: HELIXTERM_BASE_URL=https://api.helixterm.io bash scripts/
#        smoke-test.sh
# Inputs: HELIXTERM_BASE_URL, TEST_AUTH_TOKEN (from Vault)
# Outputs: Exit 0 on pass, exit 1 on fail with detailed error
#          report
# Side-effects: Creates and immediately terminates a test session
# Dependencies: curl, jq, grpc_cli
# Cross-references: docs/11_constitution_compliance.md §6.12
```

```
set -euo pipefail
```

```
BASE_URL="${HELIXTERM_BASE_URL:-http://localhost:8080}"
```

```
FAIL_COUNT=0
```

```
PASS_COUNT=0
```

```
check() {
    local name="$1"
    local result="$2"
    if [ "$result" = "PASS" ]; then
        echo "[PASS] $name"
        PASS_COUNT=$((PASS_COUNT + 1))
    else
        echo "[FAIL] $name: $result"
        FAIL_COUNT=$((FAIL_COUNT + 1))
    fi
}
```

```

    fi
}

# Check all 25 services health endpoints
SERVICES=(
    "auth-gateway:8081" "session-manager:8082" "connection-
        broker:8083"
    "protocol-handler:8084" "event-router:8085" "user-
        registry:8086"
    "billing-service:8087" "node-manager:8088" "metrics-
        collector:8089"
    "notification-dispatcher:8090" "config-service:8091" "audit-
        logger:8092"
    "rate-limiter:8093" "load-balancer:8094" "health-monitor:8095"
    "traffic-analyzer:8096" "crypto-engine:8097" "dns-
        resolver:8098"
    "geo-router:8099" "subscription-manager:8100" "key-vault:8101"
    "api-gateway:8080" "scheduler:8102" "backup-service:8103"
    "telemetry-exporter:8104"
)

for svc_port in "${SERVICES[@]}; do
    svc="${svc_port%%:*}"
    port="${svc_port##*:}"
    http_status=$(curl -s -o /dev/null -w "%{http_code}" \
        --max-time 3 "http://localhost:${port}/health/live" 2>/
        dev/null || echo "000")
    if [ "$http_status" = "200" ]; then
        check "health/live: $svc" "PASS"
    else
        check "health/live: $svc" "HTTP $http_status"
    fi
done

echo ""
echo "Smoke Test Results: ${PASS_COUNT} PASS, ${FAIL_COUNT} FAIL"

if [ "$FAIL_COUNT" -gt 0 ]; then
    echo "SMOKE TESTS FAILED – deployment blocked"
    exit 1
fi

echo "SMOKE TESTS PASSED – deployment approved"
exit 0

```

Section 7: CI/CD Constitution Compliance Gates

7.1 Complete GitHub Actions Workflow

```
# File: .github/workflows/constitution-compliance.yml
# Purpose: HelixTerminator Constitution compliance CI pipeline
# Runs on: every PR, every push to develop/main, every release tag

name: Constitution Compliance

on:
  push:
    branches: [main, develop, 'release/**', 'hotfix/**']
  pull_request:
    branches: [main, develop]
  release:
    types: [published]
  schedule:
    # Weekly full compliance run
    - cron: '0 2 * * 1'

concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: ${{ github.event_name == 'pull_request' }}

env:
  GO_VERSION: '1.25'
  FLUTTER_VERSION: '3.24.0'
  CONSTITUTION_COMMIT: 'e6504c2' # helixcode-v1.1.0 line
    (CANONICAL_FACTS.md CD-9)
  MINIMUM_COMPLIANCE_SCORE: '95'

jobs:
  # -----
  # Job 1: Constitution submodule validation
  # -----
  constitution-submodule:
    name: "CI-01 Constitution Submodule"
    runs-on: ubuntu-24.04
    steps:
      - uses: actions/checkout@v4
        with:
```

```

submodules: recursive
fetch-depth: 0

- name: Verify constitution submodule present
run: |
    if [ ! -f "constitution/Constitution.md" ]; then
        echo "FAIL: constitution submodule not initialised"
        exit 1
    fi
    echo "PASS: constitution/Constitution.md exists"

- name: Verify constitution pinned to expected tag
run: bash scripts/test_constitution_inheritance.sh

- name: Verify AGENTS.md references constitution
run: |
    if ! grep -q "constitution/AGENTS.md" AGENTS.md; then
        echo "FAIL: AGENTS.md must reference constitution/
AGENTS.md"
        exit 1
    fi
    echo "PASS: AGENTS.md references constitution"

- name: Verify CLAUDE.md references constitution
run: |
    if ! grep -q "constitution/CLAUDE.md" CLAUDE.md; then
        echo "FAIL: CLAUDE.md must reference constitution/
CLAUDE.md"
        exit 1
    fi
    echo "PASS: CLAUDE.md references constitution"

- name: Verify helix-deps.yaml present
run: |
    if [ ! -f "helix-deps.yaml" ]; then
        echo "FAIL: helix-deps.yaml missing (§11.4.31)"
        exit 1
    fi
    echo "PASS: helix-deps.yaml present"

- name: Validate helix-deps.yaml schema
run: go run scripts/validate-helix-deps.go

```

```

# Job 2: Package naming convention validation

```

```
# 

---


naming-conventions:
  name: "CI-02 Package Naming Conventions"
  runs-on: ubuntu-24.04
  steps:
    - uses: actions/checkout@v4

    - uses: actions/setup-go@v5
      with:
        go-version: ${ env.GO_VERSION }

    - name: Check Go module paths (HT-NAME-001 to HT-NAME-007)
      run: |
        bash scripts/check-go-module-names.sh

    - name: Check Flutter package names (HT-NAME-020 to HT-NAME-025)
      run: |
        bash scripts/check-flutter-package-names.sh

    - name: Check Kafka topic names (HT-NAME-030 to HT-NAME-037)
      run: |
        bash scripts/check-kafka-topic-names.sh

    - name: Check database names (HT-NAME-040 to HT-NAME-046)
      run: |
        bash scripts/check-database-names.sh

    - name: Check Kubernetes resource names
      run: |
        bash scripts/check-k8s-names.sh

    - name: Check Docker image names (HT-NAME-050 to HT-NAME-056)
      run: |
        bash scripts/check-docker-image-names.sh

    - name: Check directory naming (§11.4.29 snake_case)
      run: |
        find . -type d -name '[A-Z]*' \
          -not -path "./.git/*" \
          -not -path "*/vendor/*" \
          -not -path "*/node_modules/*" \
          | grep -v "^$" && echo "FAIL: uppercase directories found" && exit 1 \
```

```
|| echo "PASS: all directories are lowercase"
```

```
# -----  
# Job 3: Test presence verification  
# -----  
test-presence:  
  name: "CI-03 Required Test Types Present"  
  runs-on: ubuntu-24.04  
  steps:  
    - uses: actions/checkout@v4  
  
    - name: Verify unit tests present for all services  
      run: bash scripts/verify-test-types.sh --type unit  
  
    - name: Verify integration tests present for all services  
      run: bash scripts/verify-test-types.sh --type integration  
  
    - name: Verify contract tests present for required pairs  
      run: bash scripts/verify-test-types.sh --type contract  
  
    - name: Verify performance tests present  
      run: bash scripts/verify-test-types.sh --type performance  
  
    - name: Verify security tests configured  
      run: bash scripts/verify-test-types.sh --type security  
  
    - name: Verify fuzz tests for required functions  
      run: bash scripts/verify-test-types.sh --type fuzz  
  
    - name: Verify golden files exist  
      run: bash scripts/verify-test-types.sh --type golden  
  
    - name: Verify smoke test script present  
      run: |  
        if [ ! -f "scripts/smoke-test.sh" ]; then  
          echo "FAIL: scripts/smoke-test.sh missing"  
          exit 1  
        fi  
        echo "PASS: smoke-test.sh present"  
  
# -----  
# Job 4: Unit tests + coverage  
# -----  
unit-tests:
```

```
name: "CI-04 Unit Tests + Coverage"
runs-on: ubuntu-24.04
strategy:
  matrix:
    service:
      - auth-gateway
      - session-manager
      - connection-broker
      - protocol-handler
      - event-router
      - user-registry
      - billing-service
      - node-manager
      - metrics-collector
      - notification-dispatcher
      - config-service
      - audit-logger
      - rate-limiter
      - load-balancer
      - health-monitor
      - traffic-analyzer
      - crypto-engine
      - dns-resolver
      - geo-router
      - subscription-manager
      - key-vault
      - api-gateway
      - scheduler
      - backup-service
      - telemetry-exporter
  steps:
    - uses: actions/checkout@v4
      with:
        submodules: recursive

    - uses: actions/setup-go@v5
      with:
        go-version: ${ env.GO_VERSION }

    - name: Run unit tests for ${ matrix.service }
      run: |
        cd services/${ matrix.service }
        go test -race \
          -coverprofile=coverage.out \
```



```
-covermode=atomic \
-timeout 5m \
./...
```

- name: Check coverage thresholds

```
run: |
  cd services/${{ matrix.service }}
  TOTAL=$(go tool cover -func=coverage.out | grep total |
awk '{print $3}' | tr -d '%')
  echo "Coverage: ${TOTAL}%"
  if (( $(echo "$TOTAL < 80.0" | bc -l) )); then
    echo "FAIL: coverage ${TOTAL}% is below 80% minimum"
    exit 1
  fi
  echo "PASS: coverage ${TOTAL}% meets 80% minimum"
```

- name: Upload coverage report

uses: codecov/codecov-action@v4

with:

```
file: services/${{ matrix.service }}/coverage.out
flags: ${{ matrix.service }}
```

```
#
# Job 5: Linting (zero violations required)
#
```

lint:

name: "CI-05 Linting (Zero Violations)"

runs-on: ubuntu-24.04

steps:

- uses: actions/checkout@v4

with:

```
submodules: recursive
```

- uses: actions/setup-go@v5

with:

```
go-version: ${{ env.GO_VERSION }}
```

- name: golangci-lint

uses: golangci/golangci-lint-action@v6

with:

```
version: latest
```

```
args: --timeout=10m --config=.golangci.yml
```

- name: go vet

```
run: go vet ./...
```

- name: Check for forbidden patterns (AST-level)
run: go run scripts/constitution-check.go --check forbidden-patterns

```
# -----  
# Job 6: Security scan  
# -----
```

security-scan:

name: "CI-06 Security Scan"

runs-on: ubuntu-24.04

steps:

- uses: actions/checkout@v4
with:
submodules: recursive
- uses: actions/setup-go@v5
with:
go-version: \${ env.GO_VERSION }
- name: Install gosec
run: go install github.com/securego/gosec/v2/cmd/gosec@latest
- name: Run SAST (gosec)
run: |
gosec -conf .gosec.yaml -fmt sarif -out gosec.sarif ./...
if grep -q '"level":"error"' gosec.sarif; then
echo "FAIL: gosec found HIGH or CRITICAL vulnerabilities"
cat gosec.sarif
exit 1
fi
echo "PASS: gosec – no HIGH/CRITICAL findings"
- name: Run govulncheck
run: |
go install golang.org/x/vuln/cmd/govulncheck@latest
govulncheck ./...
- name: Secret scanning (gitleaks)
uses: gitleaks/gitleaks-action@v2
env:
GITHUB_TOKEN: \${ secrets.GITHUB_TOKEN }

```
- name: Upload SARIF
  uses: github/codeql-action/upload-sarif@v3
  with:
    sarif_file: gosec.sarif
```

```
# -----
# Job 7: Forbidden pattern detection
# -----
```

```
forbidden-patterns:
```

```
  name: "CI-07 Forbidden Pattern Detection"
```

```
  runs-on: ubuntu-24.04
```

```
  steps:
```

```
    - uses: actions/checkout@v4
```

```
    - uses: actions/setup-go@v5
```

```
      with:
```

```
        go-version: ${ env.GO_VERSION }
```

```
- name: Detect hardcoded credentials
```

```
  run: |
```

```
    if grep -rn \
      -e 'password\s*=\s*"[^"]\|password\s*:=\s*"[^"]\|
      +" \|apiKey\s*=\s*"[^"]\|+' \
      --include="*.go" \
      --exclude-dir=".git" \
      --exclude-dir="vendor" \
      --exclude="*_test.go" \
      .; then
      echo "FAIL: hardcoded credentials detected"
      exit 1
    fi
    echo "PASS: no hardcoded credentials"
```

```
- name: Detect global mutable state
```

```
  run: |
```

```
    if grep -rn \
      -e '^var [A-Z][a-zA-Z]* = \(map\|sync\.\|make\)\' \
      --include="*.go" \
      --exclude-dir=".git" \
      --exclude="*_test.go" \
      services/ pkg; then
      echo "FAIL: global mutable state detected"
      exit 1
    fi
```

```
fi
echo "PASS: no exported global mutable state"
```

- name: Detect missing context propagation
run: go run scripts/constitution-check.go --check context-propagation
- name: Detect direct DB access in handlers
run: go run scripts/constitution-check.go --check handler-db-access
- name: Detect submodule functionality duplication
run: go run scripts/constitution-check.go --check submodule-duplication
- name: Detect TODO/FIXME in non-test files
run: |
if grep -rn \
-e '//\s*TODO\|//\s*FIXME\|//\s*HACK\|//\s*XXX' \
--include="*.go" \
--exclude="*_test.go" \
--exclude-dir=".git" \
services/ pkg/; then
echo "FAIL: TODO/FIXME found in production code"
exit 1
fi
echo "PASS: no TODO/FIXME in production code"

```
# -----  
# Job 8: Commit message format verification  
# -----  
commit-messages:  
  name: "CI-08 Commit Message Format"  
  runs-on: ubuntu-24.04  
  if: github.event_name == 'pull_request'  
  steps:  
    - uses: actions/checkout@v4  
      with:  
        fetch-depth: 0  
  
    - name: Check commit message format (last 20 commits)  
      run: |  
        BRANCH="${{ github.head_ref }}"  
        BASE="${{ github.base_ref }}"  
        COMMITS=$(git log origin/${BASE}..HEAD --format="%H %s"  
        | head -20)
```

```

PATTERN='^(feat|fix|chore|refactor|test|docs|perf|
security|ci|constitution|revert|build)(\[a-z0-9-\]+\))?: .
{10,72}$'

FAIL=0

while IFS= read -r line; do
    HASH="${line%% *}"
    MSG="${line#* }"
    if ! echo "$MSG" | grep -qE "$PATTERN"; then
        echo "FAIL: invalid commit message format: $MSG"
        echo "      (commit $HASH)"
        FAIL=1
    fi
done <<< "$COMMITTS"

if [ "$FAIL" -eq 1 ]; then
    echo ""
    echo "Commit messages must follow: <type>(<scope>):
<subject>"
    echo "Valid types: feat|fix|chore|refactor|test|docs|
perf|security|ci|constitution|revert|build"
    exit 1
fi
echo "PASS: all commit messages valid"

```

```

# -----
# Job 9: Full constitution compliance score
# -----

compliance-score:
  name: "CI-09 Constitution Compliance Score"
  runs-on: ubuntu-24.04
  needs:
    - constitution-submodule
    - naming-conventions
    - test-presence
    - lint
    - security-scan
    - forbidden-patterns
    - commit-messages
  steps:
    - uses: actions/checkout@v4
      with:
        submodules: recursive

```

- uses: actions/setup-go@v5
 with:
 go-version: \${ env.GO_VERSION }
- name: Run full constitution compliance check
 id: compliance
 run: |
 SCORE=\$(go run scripts/constitution-check.go \
 --strict \
 --output json \
 | jq '.score')
 echo "score=\${SCORE}" >> \$GITHUB_OUTPUT
 echo "Compliance Score: \${SCORE}/100"

 if ((\$(echo "\$SCORE < \$
 {{ env.MINIMUM_COMPLIANCE_SCORE }}" | bc -l))); then
 echo "FAIL: compliance score \${SCORE} is below minimum
 \${MINIMUM_COMPLIANCE_SCORE}"
 exit 1
 fi
 echo "PASS: compliance score \${SCORE} meets minimum \$
 {{ env.MINIMUM_COMPLIANCE_SCORE }}"
- name: Generate compliance report
 if: always()
 run: |
 go run scripts/constitution-check.go \
 --strict \
 --report \
 --output markdown \
 > compliance-report.md
- name: Post compliance report as PR comment
 if: github.event_name == 'pull_request' && always()
 uses: thollander/actions-comment-pull-request@v2
 with:
 filePath: compliance-report.md
 comment_tag: constitution-compliance

```
# Job 10: .gitignore validation (§11.4.30)
```

```
gitignore-audit:
  name: "CI-10 .gitignore Audit (§11.4.30)"
  runs-on: ubuntu-24.04
```

steps:

- uses: actions/checkout@v4
- name: Verify .gitignore files present in all service directories
run: |
MISSING=0
for svc_dir in services/*/; do
if [! -f "\${svc_dir}.gitignore"]; then
echo "FAIL: missing .gitignore in \${svc_dir}"
MISSING=1
fi
done
if ["\$MISSING" -eq 1]; then exit 1; fi
echo "PASS: .gitignore present in all service directories"
- name: Verify no build artefacts tracked
run: |
git ls-files | grep -E '\.(out|exe|so|dll|class|pyc|jar)\$' \
&& echo "FAIL: build artefacts tracked in git" && exit 1 \
|| echo "PASS: no build artefacts tracked"
- name: Verify no .env files tracked
run: |
git ls-files | grep -E '^\.env\$|/\.env\$' \
&& echo "FAIL: .env file tracked in git (\$11.4.10 + \$11.4.30 violation)" && exit 1 \
|| echo "PASS: no .env files tracked"

Job 11: Release gate (score must be 100)

release-gate:

name: "CI-11 Release Gate (Score = 100)"
runs-on: ubuntu-24.04
if: startsWith(github.ref, 'refs/tags/helixterm-')
needs:

- compliance-score
- unit-tests
- security-scan

steps:

- uses: actions/checkout@v4
with:

```

    submodules: recursive

- uses: actions/setup-go@v5
  with:
    go-version: ${ env.GO_VERSION }

- name: Full compliance check – release requires score 100
  run: |
    SCORE=$(go run scripts/constitution-check.go \
      --strict \
      --output json \
      | jq '.score')
    echo "Release compliance score: ${SCORE}/100"
    if [ "$SCORE" -ne 100 ]; then
      echo "FAIL: release requires compliance score 100, got
${SCORE}"
      echo "Fix all compliance issues before releasing."
      exit 1
    fi
    echo "PASS: release compliance score is 100"

- name: Verify release tag format (§11.4.151)
  run: |
    TAG="${ github.ref_name }"
    if ! echo "$TAG" | grep -qE '^helixterm-[0-9]+\.[0-9]+\.[
0-9]+(-[a-z]+\.[0-9]+)?$'; then
      echo "FAIL: release tag '${TAG}' must match
'helixterm-X.Y.Z' format"
      exit 1
    fi
    echo "PASS: release tag format valid: ${TAG}"

```

7.2 Complete Constitution Checker Script

```

// File: scripts/constitution-check.go
// Purpose: HelixTerminator Constitution compliance checker
// Usage: go run scripts/constitution-check.go [--strict] [--
report] [--output json|markdown|text]
// Inputs: HelixTerminator repository root (current directory)
// Outputs: Compliance score (0-100), detailed report
// Side-effects: None (read-only)
// Dependencies: Go 1.25+, standard library + go/ast for AST
analysis
// Cross-references: docs/11_constitution_compliance.md §7, §12

```

```
package main
```



```

import (
    "encoding/json"
    "flag"
    "fmt"
    "go/ast"
    "go/parser"
    "go/token"
    "os"
    "path/filepath"
    "regexp"
    "strings"
)

// CheckResult holds the result of a single compliance check.
type CheckResult struct {
    Rule      string `json:"rule"`
    Category  string `json:"category"`
    Status    string `json:"status"` // "PASS" | "FAIL" | "WARN"
    Message   string `json:"message"`
    Details   string `json:"details,omitempty"`
}

// ComplianceReport is the full output of the compliance checker.
type ComplianceReport struct {
    Score      int      `json:"score"`
    Status     string   `json:"status"` // "COMPLIANT" | "NON-COMPLIANT" | "RELEASE-BLOCKED"
    Checks     []CheckResult `json:"checks"`
    Summary    string     `json:"summary"`
}

var (
    strict      = flag.Bool("strict", false,
        "Exit with error if any check fails")
    report      = flag.Bool("report", false, "Generate detailed report")
    outputFmt   = flag.String("output", "text", "Output format: text|json|markdown")
    check       = flag.String("check", "", "Run specific check only")
    rule        = flag.String("rule", "", "Run specific rule only")
    mockCov     = flag.Float64("mock-coverage", -1, "Override coverage for testing (meta-mutation)")
)

```

```

func main() {
    flag.Parse()

    checker := NewConstitutionChecker()
    // *check is now actually read by RunAll (see below) – it is
    // no longer a
    // declared-but-unused dispatch flag (anti-bluff §1.1 / former
    // BLUFF gate).
    report := checker.RunAll(*check)

    switch *outputFmt {
    case "json":
        enc := json.NewEncoder(os.Stdout)
        enc.SetIndent("", " ")
        enc.Encode(report)
    case "markdown":
        fmt.Print(report.ToMarkdown())
    default:
        fmt.Print(report.ToText())
    }

    if *strict && report.Score < 95 {
        os.Exit(1)
    }
}

// ConstitutionChecker performs all compliance checks.
type ConstitutionChecker struct {
    results []CheckResult
    root    string
}

func NewConstitutionChecker() *ConstitutionChecker {
    root, _ := os.Getwd()
    return &ConstitutionChecker{root: root}
}

// RunAll runs every named check, or exactly one if `only` is non-
// empty.
// `only` is the value of the `--check` flag: previously this
// parameter did
// not exist and the flag was declared but never read, so every CI
// step
// invoking `--check <name>` (§7) silently ran the full suite
// regardless of
// which name was passed – a BLUFF gate. It is now a real,
// exercised filter.

```

```

func (c *ConstitutionChecker) RunAll(only string)
    *ComplianceReport {
    fmt.Println("HelixTerminator Constitution Compliance Check")
    fmt.Println("=====")

    // checks is the ordered, named check registry backing --check
    <name>.
    // Job-comment numbers below (Check 1..Check 10) intentionally
    do NOT use
    // "$N" – that prefix is reserved for HelixConstitution clause
    numbers
    // elsewhere in this document and must not collide with it.
    checks := []struct {
        name string
        fn func()
    }{
        {"submodule-catalogue", c.checkConstitutionSubmodule}, //
        Check 1: Constitution submodule
        {"package-naming",
        c.checkPackageNaming}, // Check 2: Package
        naming
        {"agent-files",
        c.checkAgentFiles}, // Check 3:
        AGENTS.md + CLAUDE.md
        {"helix-deps",
        c.checkHelixDeps}, // Check 4:
        helix-deps.yaml
        {"test-presence",
        c.checkTestPresence}, // Check 5: Test
        presence
        {"coverage",
        c.checkCoverage}, // Check 6:
        Coverage (if not overridden)
        {"forbidden-patterns",
        c.checkForbiddenPatterns}, // Check 7: Forbidden
        patterns
        {"gitignore",
        c.checkGitignore}, // Check
        8: .gitignore
        {"repo-structure",
        c.checkRepoStructure}, // Check 9:
        Repository structure
        {"commit-messages",
        c.checkCommitMessages}, // Check 10: Commit
        format (last 20)
    }

    if only != "" {
        for _, ch := range checks {
            if ch.name == only {
                ch.fn()
            }
        }
    }
}

```

```

        return c.buildReport()
    }
}
// §11.4.6 no-guessing mandate: fail closed on an
// unrecognized --check
// value instead of silently running every check (the
// prior bug).
// context-propagation / handler-db-access / submodule-
// duplication /
// global-state / required-files are separate AST
// detectors invoked
// directly in §7 CI steps and are not yet wired into this
// table.
// > DEFERRED (next increment): wire those five in so
// every --check
// > value used anywhere in this document resolves here
// too.
fmt.Printf("FAIL: unknown --check value %q\n", only)
os.Exit(1)
}

for _, ch := range checks {
    ch.fn()
}

return c.buildReport()
}

func (c *ConstitutionChecker) pass(rule, category, msg string) {
    c.results = append(c.results, CheckResult{
        Rule: rule, Category: category, Status: "PASS", Message:
        msg,
    })
    fmt.Printf("[PASS] %s: %s\n", rule, msg)
}

func (c *ConstitutionChecker) fail(rule, category, msg, details
    string) {
    c.results = append(c.results, CheckResult{
        Rule: rule, Category: category, Status: "FAIL",
        Message: msg, Details: details,
    })
    fmt.Printf("[FAIL] %s: %s\n", rule, msg)
    if details != "" {
        fmt.Printf("        Details: %s\n", details)
    }
}

```

```

func (c *ConstitutionChecker) warn(rule, category, msg string) {
    c.results = append(c.results, CheckResult{
        Rule: rule, Category: category, Status: "WARN", Message:
        msg,
    })
    fmt.Printf("[WARN] %s: %s\n", rule, msg)
}

func (c *ConstitutionChecker) checkConstitutionSubmodule() {
    constitutionMD := filepath.Join(c.root, "constitution",
        "Constitution.md")
    if _, err := os.Stat(constitutionMD); os.IsNotExist(err) {
        c.fail("CONST-001", "constitution",
            "constitution submodule not initialised",
            "Run: git submodule update --init --recursive")
    } else {
        c.pass("CONST-001", "constitution", "constitution
            submodule present")
    }

    agentsMD := filepath.Join(c.root, "AGENTS.md")
    if data, err := os.ReadFile(agentsMD); err != nil || !
        strings.Contains(string(data), "constitution/AGENTS.md") {
        c.fail("CONST-002", "constitution",
            "AGENTS.md does not reference constitution/AGENTS.md",
            "Add '@constitution/AGENTS.md' reference to
            AGENTS.md")
    } else {
        c.pass("CONST-002", "constitution", "AGENTS.md references
            constitution")
    }

    claudeMD := filepath.Join(c.root, "CLAUDE.md")
    if data, err := os.ReadFile(claudeMD); err != nil || !
        strings.Contains(string(data), "constitution/CLAUDE.md") {
        c.fail("CONST-003", "constitution",
            "CLAUDE.md does not reference constitution/CLAUDE.md",
            "Add '@constitution/CLAUDE.md' reference to
            CLAUDE.md")
    } else {
        c.pass("CONST-003", "constitution", "CLAUDE.md references
            constitution")
    }
}

func (c *ConstitutionChecker) checkPackageName() {

```

```

servicePattern := regexp.MustCompile(`^helixterm\.io/services/
[a-z][a-z0-9-]*$`)
pkgPattern := regexp.MustCompile(`^helixterm\.io/pkg/[a-z][a-
z0-9-]*$`)

servicesDir := filepath.Join(c.root, "services")
entries, err := os.ReadDir(servicesDir)
if err != nil {
    c.fail("NAME-001", "naming", "services/ directory not
found", err.Error())
    return
}

for _, entry := range entries {
    if !entry.IsDir() {
        continue
    }
    goModPath := filepath.Join(servicesDir, entry.Name(),
"go.mod")
    data, err := os.ReadFile(goModPath)
    if err != nil {
        c.fail("NAME-001", "naming",
            fmt.Sprintf("missing go.mod in services/%s",
entry.Name()), "")
        continue
    }
    for _, line := range strings.Split(string(data), "\n") {
        if strings.HasPrefix(line, "module ") {
            modulePath :=
strings.TrimSpace(strings.TrimPrefix(line, "module "))
            if !servicePattern.MatchString(modulePath) {
                c.fail("NAME-001", "naming",
                    fmt.Sprintf("invalid module path: %s",
modulePath),
                    fmt.Sprintf("Must match: helixterm.io/
services/<name>"))
            } else {
                c.pass("NAME-001", "naming",
                    fmt.Sprintf("valid module path: %s",
modulePath))
            }
        }
    }
}

// Check pkg/ naming
pkgDir := filepath.Join(c.root, "pkg")

```

```

if pkgEntries, err := os.ReadDir(pkgDir); err == nil {
    for _, entry := range pkgEntries {
        if !entry.IsDir() {
            continue
        }
        goModPath := filepath.Join(pkgDir, entry.Name(),
"go.mod")
        data, err := os.ReadFile(goModPath)
        if err != nil {
            continue
        }
        for _, line := range strings.Split(string(data),
"\n") {
            if strings.HasPrefix(line, "module ") {
                modulePath :=
strings.TrimSpace(strings.TrimPrefix(line, "module "))
                if !pkgPattern.MatchString(modulePath) {
                    c.fail("NAME-010", "naming",
                        fmt.Sprintf("invalid pkg module path:
%s", modulePath), "")
                } else {
                    c.pass("NAME-010", "naming",
                        fmt.Sprintf("valid pkg module path:
%s", modulePath))
                }
            }
        }
    }
}

func (c *ConstitutionChecker) checkAgentFiles() {
    for _, file := range []string{"AGENTS.md", "CLAUDE.md",
"QWEN.md"} {
        path := filepath.Join(c.root, file)
        if _, err := os.Stat(path); os.IsNotExist(err) {
            c.fail("AGENT-001", "agent-files",
                fmt.Sprintf("%s missing at repository root",
file), "")
        } else {
            c.pass("AGENT-001", "agent-files",
                fmt.Sprintf("%s present at repository root",
file))
        }
    }
}

```

```

func (c *ConstitutionChecker) checkHelixDeps() {
    helixDeps := filepath.Join(c.root, "helix-deps.yaml")
    if _, err := os.Stat(helixDeps); os.IsNotExist(err) {
        c.fail("DEPS-001", "helix-deps",
            "helix-deps.yaml missing (§11.4.31)", "Create helix-
            deps.yaml per Section 5")
        return
    }
    c.pass("DEPS-001", "helix-deps", "helix-deps.yaml present")

    data, err := os.ReadFile(helixDeps)
    if err != nil {
        c.fail("DEPS-002", "helix-deps", "cannot read helix-
            deps.yaml", err.Error())
        return
    }

    // Check required sections
    for _, required := range []string{"schema_version",
        "constitution:", "vasic_digital_dependencies:",
        "validation_rules:"} {
        if !strings.Contains(string(data), required) {
            c.fail("DEPS-002", "helix-deps",
                fmt.Sprintf("helix-deps.yaml missing required
                section: %s", required), "")
        } else {
            c.pass("DEPS-002", "helix-deps",
                fmt.Sprintf("helix-deps.yaml contains: %s",
                required))
        }
    }
}

func (c *ConstitutionChecker) checkTestPresence() {
    servicesDir := filepath.Join(c.root, "services")
    entries, _ := os.ReadDir(servicesDir)

    for _, entry := range entries {
        if !entry.IsDir() {
            continue
        }
        svcDir := filepath.Join(servicesDir, entry.Name())

        // Check for unit tests
        hasUnit := false
        filepath.Walk(svcDir, func(path string, info os.FileInfo,
            err error) error {

```



```

        if strings.HasSuffix(path, "_test.go") && !
strings.Contains(path, "integration") {
            hasUnit = true
        }
        return nil
    })
    if !hasUnit {
        c.fail("TEST-001", "tests",
            fmt.Sprintf("no unit tests in services/%s",
entry.Name()), "")
    } else {
        c.pass("TEST-001", "tests",
            fmt.Sprintf("unit tests present in services/%s",
entry.Name()))
    }

    // Check for integration tests
    integDir := filepath.Join(svcDir, "tests", "integration")
    if _, err := os.Stat(integDir); os.IsNotExist(err) {
        c.fail("TEST-002", "tests",
            fmt.Sprintf("no integration tests directory in
services/%s", entry.Name()),
            "Create tests/integration/ with at least one
*_integration_test.go file")
    } else {
        c.pass("TEST-002", "tests",
            fmt.Sprintf("integration tests directory present
in services/%s", entry.Name()))
    }
}

}

func (c *ConstitutionChecker) checkCoverage() {
    if *mockCov >= 0 {
        // Meta-mutation mode: simulate a specific coverage value
        if *mockCov < 80.0 {
            c.fail("COV-001", "coverage",
                fmt.Sprintf("simulated coverage %.1f%% is below
80%% minimum", *mockCov),
                "META-MUTATION: gate correctly identifies
insufficient coverage")
        } else {
            c.pass("COV-001", "coverage",
                fmt.Sprintf("simulated coverage %.1f%% meets 80%%
minimum", *mockCov))
        }
    }
    return
}

```

```

}
// In real CI, coverage is checked per-service in the unit-
// tests job.
c.pass("COV-001", "coverage",
      "Coverage checked per-service in unit-tests job")
}

func (c *ConstitutionChecker) checkForbiddenPatterns() {
    goFiles := c.collectGoFiles(c.root, false) // exclude test
        files

    for _, file := range goFiles {
        data, err := os.ReadFile(file)
        if err != nil {
            continue
        }
        content := string(data)

        // Check for hardcoded credentials
        credPattern := regexp.MustCompile(`(?i)(password|passwd|
        secret|apikey|api_key)\s*[:=]+\s*"^[^"]{4,}"`)
        if credPattern.MatchString(content) {
            c.fail("FP-001", "forbidden-patterns",
                fmt.Sprintf("potential hardcoded credentials in
                %s", file),
                "Move to environment variable via Vault
                injection")
        }

        // Check for global mutable state
        fset := token.NewFileSet()
        f, err := parser.ParseFile(fset, file, nil, 0)
        if err == nil {
            for _, decl := range f.Decls {
                genDecl, ok := decl.(*ast.GenDecl)
                if !ok || genDecl.Tok != token.VAR {
                    continue
                }
                for _, spec := range genDecl.Specs {
                    valSpec, ok := spec.(*ast.ValueSpec)
                    if !ok {
                        continue
                    }
                    for _, name := range valSpec.Names {
                        if ast.IsExported(name.Name) {
                            // Exported var at package level –
                            check for map/slice

```

```

        c.warn("FP-002", "forbidden-patterns",
            fmt.Sprintf("exported package-
level var in %s: %s (review for global state)", file,
name.Name))
    }
}
}
}
}

// Check for TODO/FIXME in non-test production code
for _, file := range goFiles {
    data, _ := os.ReadFile(file)
    todoPattern := regexp.MustCompile(`\/\/\s*(TODO|FIXME|HACK|
XXX)`)
    if todoPattern.Match(data) {
        c.fail("FP-003", "forbidden-patterns",
            fmt.Sprintf("TODO/FIXME found in production file:
%s", file),
            "Resolve or create a tracked issue instead")
    }
}

}

func (c *ConstitutionChecker) checkGitignore() {
    rootGitignore := filepath.Join(c.root, ".gitignore")
    if _, err := os.Stat(rootGitignore); os.IsNotExist(err) {
        c.fail("GI-001", "gitignore", "root .gitignore missing
($11.4.30)", "")
        return
    }

    data, _ := os.ReadFile(rootGitignore)
    required := []string{".env", "*.out", "*.exe", "/bin/",
        "vendor/", ".idea/"}
    for _, pattern := range required {
        if !strings.Contains(string(data), pattern) {
            c.warn("GI-001", "gitignore",
                fmt.Sprintf(".gitignore missing recommended
pattern: %s", pattern))
        }
    }
    c.pass("GI-001", "gitignore", ".gitignore present at root")
}

func (c *ConstitutionChecker) checkRepoStructure() {

```

```

required := []string{
    "AGENTS.md", "CLAUDE.md", "QWEN.md", "helix-deps.yaml",
    "README.md", "CHANGELOG.md", "Makefile", "go.work",
    "constitution/Constitution.md",
    "services/", "pkg/", "docs/", ".github/", "scripts/",
    "deploy/", "charts/",
}

for _, path := range required {
    fullPath := filepath.Join(c.root, path)
    if _, err := os.Stat(fullPath); os.IsNotExist(err) {
        c.fail("STRUCT-001", "structure",
            fmt.Sprintf("required path missing: %s", path),
            "")
    } else {
        c.pass("STRUCT-001", "structure",
            fmt.Sprintf("required path present: %s", path))
    }
}
}

func (c *ConstitutionChecker) checkCommitMessages() {
    // This check is informational in local mode; enforced in CI
    // job
    c.pass("COMMIT-001", "commits",
        "Commit message format enforced by CI job 8 (PR-only)")
}

func (c *ConstitutionChecker) collectGoFiles(root string,
    includeTests bool) []string {
    var files []string
    filepath.Walk(root, func(path string, info os.FileInfo, err
        error) error {
        if err != nil || info.IsDir() {
            return nil
        }
        if !strings.HasSuffix(path, ".go") {
            return nil
        }
        if !includeTests && strings.HasSuffix(path, "_test.go") {
            return nil
        }
        if strings.Contains(path, "/vendor/") ||
            strings.Contains(path, "/.git/") {
            return nil
        }
        files = append(files, path)
    })
}

```

```

        return nil
    })
    return files
}

func (c *ConstitutionChecker) buildReport() *ComplianceReport {
    total := len(c.results)
    passes := 0
    fails := 0
    warns := 0
    for _, r := range c.results {
        switch r.Status {
            case "PASS":
                passes++
            case "FAIL":
                fails++
            case "WARN":
                warns++
        }
    }

    // Score: 100 - (fails * 5) - (warns * 1), minimum 0
    score := 100 - (fails * 5) - (warns * 1)
    if score < 0 {
        score = 0
    }

    status := "COMPLIANT"
    if score < 95 {
        status = "NON-COMPLIANT"
    }
    if score < 100 && isRelease() {
        status = "RELEASE-BLOCKED"
    }

    fmt.Printf("\nCompliance Score: %d/100 - %s\n", score, status)
    fmt.Printf("Results: %d PASS, %d FAIL, %d WARN (of %d total\n",
        checks)\n",
        passes, fails, warns, total)

    return &ComplianceReport{
        Score: score,
        Status: status,
        Checks: c.results,
    }
}

```

```

        Summary: fmt.Sprintf("%d PASS, %d FAIL, %d WARN of %d
        checks",
            passes, fails, warns, total),
    }
}

func isRelease() bool {
    ref := os.Getenv("GITHUB_REF")
    return strings.HasPrefix(ref, "refs/tags/helixterm-")
}

func (r *ComplianceReport) ToText() string {
    var sb strings.Builder
    sb.WriteString("\n=== CONSTITUTION COMPLIANCE REPORT ===\n")
    sb.WriteString(fmt.Sprintf("Score: %d/100 - %s\n", r.Score,
        r.Status))
    sb.WriteString(fmt.Sprintf("Summary: %s\n\n", r.Summary))
    for _, check := range r.Checks {
        if check.Status != "PASS" {
            sb.WriteString(fmt.Sprintf("[%s] %s: %s\n",
                check.Status, check.Rule, check.Message))
            if check.Details != "" {
                sb.WriteString(fmt.Sprintf("    → %s\n",
                    check.Details))
            }
        }
    }
    return sb.String()
}

func (r *ComplianceReport) ToMarkdown() string {
    var sb strings.Builder
    icon := map[string]string{"PASS": "✅", "FAIL": "❌", "WARN":
        "⚠️"}
    statusIcon := map[string]string{"COMPLIANT": "✅", "NON-
        COMPLIANT": "❌", "RELEASE-BLOCKED": "🚫"}

    sb.WriteString(fmt.Sprintf("## Constitution Compliance
        Report\n\n"))
    sb.WriteString(fmt.Sprintf("**Score:** %d/100 %s **%s**\n\n",
        r.Score, statusIcon[r.Status], r.Status))
    sb.WriteString(fmt.Sprintf("**Summary:** %s\n\n", r.Summary))
    sb.WriteString("| Status | Rule | Message |\n")
    sb.WriteString("|-----|-----|-----|\n")
    for _, check := range r.Checks {
        sb.WriteString(fmt.Sprintf("| %s %s | `%s` | %s |\n",

```

```

        icon[check.Status], check.Status, check.Rule,
        check.Message))
    }
    return sb.String()
}

```

Section 8: Anti-Patterns and Forbidden Patterns

Every forbidden pattern listed below triggers an immediate CI failure. Detection methods include AST-level analysis, grep patterns, and lint rules.

8.1 Duplicating vasic-digital Submodule Functionality

Field	Description
What it is	Reimplementing functionality already provided by a vasic-digital or HelixDevelopment submodule (e.g., writing a custom circuit breaker when vasic-digital/recovery exists)
Why forbidden	Violates §11.4.74 (catalogue-first discovery + extend-don't-reimplement). Creates maintenance burden and divergence from the maintained submodule.
Detection	AST check: <code>go run scripts/constitution-check.go --check submodule-duplication</code> . Also grep for known reimplementation patterns: <code>grep -rn "type CircuitBreaker\ type TokenBucket\ type ConnectionPool" services/ pkg/</code>
Corrective action	Delete the reimplementation; import the corresponding vasic-digital submodule; open a PR against the submodule upstream if it lacks needed functionality

8.2 Global Mutable State in Microservices

Field	Description
What it is	Package-level <code>var</code> declarations that hold mutable shared state (maps, slices, counters) accessible across goroutines without synchronisation
Why forbidden	Creates race conditions; makes services non-testable in isolation; prevents horizontal scaling
Detection	<code>go test -race ./...</code> catches races at runtime. AST check: <code>go run scripts/constitution-check.go --check global-state</code> . Lint: revive with <code>global-declarations</code> rule.
Corrective action	Move state into a struct; inject via constructor; use <code>sync.RWMutex</code> if shared state is unavoidable

Example violation and fix:

```
// VIOLATION: global mutable state
var sessionCache =
    map[string]*Session{} // race condition waiting to happen

// FIX: inject state via struct
type SessionService struct {
    cache *Cache // from digital.vasic.cache submodule
}
func NewSessionService(cache *Cache) *SessionService {
    return &SessionService{cache: cache}
}
```

8.3 Missing Context Propagation

Field	Description
What it is	Functions that perform I/O, call external services, or do database operations WITHOUT accepting a <code>context.Context</code> as their first parameter
Why forbidden	Makes it impossible to cancel operations, set deadlines, or propagate tracing spans.

Field	Description
	Causes goroutine leaks on request cancellation.
Detection	AST check: go run scripts/constitution-check.go --check context-propagation. Lint rule: contextcheck (golangci-lint).
Corrective action	Add ctx context.Context as first parameter to all I/O-performing functions

```
// VIOLATION
```

```
func (r *Repository) FindUser(id string) (*User, error) {
    return r.db.QueryRow("SELECT ...", id) // no context = no
        cancellation
}
```

```
// FIX
```

```
func (r *Repository) FindUser(ctx context.Context, id string)
    (*User, error) {
    return r.db.QueryRowContext(ctx, "SELECT ...", id)
}
```

8.4 Hardcoded Credentials

Field	Description
What it is	Any secret (password, API key, token, certificate, private key) embedded as a string literal in any tracked file
Why forbidden	Violates §11.4.10. Secrets in git history are permanently compromised even after deletion.
Detection	Pre-commit hook: gitleaks detect --staged. CI: gitleaks detect --source=.. Grep: grep -rn 'password\s*=\s*' --include="*.go".
Corrective action	Rotate the leaked secret immediately. Load from os.Getenv() or Vault SDK. Add pre-store leak audit per §11.4.10.A.

8.5 Skipping Error Handling

Field	Description
What it is	Using <code>_</code> to discard error return values from functions that can fail
Why forbidden	Violates §11.4 anti-bluff covenant. Silent failures become invisible production incidents.
Detection	Lint: <code>errcheck</code> (golanci-lint). AST check via <code>go vet</code> .
Corrective action	Handle every error; wrap with context via <code>fmt.Errorf("operation: %w", err)</code> ; return to caller or log with evidence

```
// VIOLATION
result, _ := repository.Save(ctx, entity) // error silently
                                         discarded

// FIX
result, err := repository.Save(ctx, entity)
if err != nil {
    return nil, fmt.Errorf("saving entity %s: %w", entity.ID, err)
}
```

8.6 Missing Structured Logging

Field	Description
What it is	Using <code>fmt.Printf</code> , <code>log.Printf</code> , or bare <code>println</code> for operational logging instead of the structured logger from <code>digital.vasic.observability</code>
Why forbidden	Unstructured logs cannot be parsed, filtered, or correlated by tracing systems. Violates HT-010.
Detection	Grep: <code>grep -rn "^import.*\"log\"\$\" fmt\\.Printf\\ log\\.Printf\\ log\\.Println\" --include=\"*.go\" services/ pkg/</code>
Corrective action	Replace all logging calls with <code>digital.vasic.observability</code> structured logger; add key-value pairs for all relevant context

8.7 Direct Database Access from Handlers

Field	Description
What it is	HTTP or gRPC handlers that directly call database queries instead of going through the service → repository layer
Why forbidden	Violates the layered architecture. Makes handlers impossible to unit test. Prevents business logic reuse.
Detection	AST check: <code>go run scripts/constitution-check.go --check handler-db-access</code> . Check for <code>*sql.DB</code> or <code>*gorm.DB</code> fields on handler structs.
Corrective action	Move database logic to repository layer; inject repository into service; inject service into handler

8.8 Missing Circuit Breaker on External Calls

Field	Description
What it is	Calls to external services (other microservices, third-party APIs, databases) without a circuit breaker wrapping them
Why forbidden	A slow or failing dependency can cascade and take down the calling service if not isolated.
Detection	Code review: look for <code>grpc.Dial</code> or <code>http.Client.Do</code> calls not wrapped in <code>recovery.CircuitBreaker</code> .
Corrective action	Wrap all external calls with <code>digital.vasic.recovery.NewCircuitBreaker(config).Execute(...)</code>

8.9 Missing Timeout on Network Calls

Field	Description
What it is	Network I/O (HTTP requests, gRPC calls, database queries) that does not have a deadline enforced via context
Why forbidden	Goroutines blocked on network I/O without timeout will exhaust the goroutine pool under load.
Detection	

Field	Description
	Lint: noctx golangci-lint rule. Check for <code>http.Get(url)</code> without context.
Corrective action	Always derive a context with deadline: <code>ctx, cancel := context.WithTimeout(ctx, 5*time.Second); defer cancel()</code>

8.10 Using latest Docker Tag in Production

Field	Description
What it is	Kubernetes manifests or Helm values using <code>:latest</code> as the image tag
Why forbidden	<code>latest</code> is mutable and not reproducible. Production deployments MUST be pinned to exact versions. Violates HT-NAME-052.
Detection	Grep: <code>grep -rn "image:.*:latest" deploy/charts/</code>
Corrective action	Pin to exact semver tag or SHA-256 digest

8.11 init() Functions with Side Effects

Field	Description
What it is	<code>init()</code> functions that connect to databases, start goroutines, or perform other side effects at package import time
Why forbidden	Makes packages impossible to import in test without triggering side effects. Hides dependencies.
Detection	AST check: scan for <code>init()</code> functions with statements other than constant registration.
Corrective action	Move initialisation to explicit constructor functions; call from <code>main()</code>

8.12 Mocks in Non-Unit Tests (§11.4.27-A)

Field	Description
What it is	Using mock implementations, <code>httptest.NewServer</code> with fake responses, or

Field	Description
	stub services in integration, E2E, or contract tests
Why forbidden	Non-unit tests MUST exercise the real, fully implemented system per §11.4.27. Mocks in non-unit tests produce pass results that do not prove real-system behaviour.
Detection	Grep in tests/integration/, tests/e2e/, tests/contract/: <pre>grep -rn "mock\\.\\ testify/mock\\ httptest\\.NewServer" tests/ --exclude-dir=unit</pre>
Corrective action	Replace with real service calls; use <code>deploy/docker-compose.test.yml</code> to spin up real infrastructure for integration tests

8.13 Returning Errors Without Wrapping

Field	Description
What it is	return <code>err</code> without adding context using <code>fmt.Errorf("operation: %w", err)</code>
Why forbidden	Stack-less errors make debugging impossible in production. Every error boundary must add context.
Detection	Lint: <code>wrapcheck</code> <code>golangci-lint</code> plugin. Grep: <pre>return err</pre> on its own line without wrapping.
Corrective action	<code>return fmt.Errorf("helixterm.io/services/auth: ValidateToken: %w", err)</code>

8.14 Ignoring gRPC Status Codes

Field	Description
What it is	Checking only <code>err != nil</code> from gRPC calls without inspecting <code>status.Code(err)</code>
Why forbidden	<code>gRPC</code> <code>status.Unavailable</code> and <code>status.DeadlineExceeded</code> require different retry strategies.
Detection	AST: gRPC calls followed by <code>if err != nil</code> without a <code>switch status.Code(err)</code> block.

Field	Description
Corrective action	Use <code>google.golang.org/grpc/status</code> to extract code and handle retrieable vs non-retrieable errors separately

8.15 Panic in Library Code

Field	Description
What it is	<code>panic()</code> calls in library packages (<code>helixterm.io/pkg/*</code>) outside of initialisation-time invariant checks
Why forbidden	Library panics are impossible for callers to handle without <code>recover()</code> . Return errors instead.
Detection	Grep: <code>grep -rn "panic(" helixterm.io/pkg/</code> (only <code>main()</code> and <code>TestMain</code> may panic).
Corrective action	Return error from all fallible functions; reserve panic only for impossible-to-recover programmer errors at <code>init()</code> time

8.16 Embedding Secrets in Docker Images

Field	Description
What it is	ARG or ENV statements in Dockerfiles containing API keys, passwords, or tokens; or COPY of <code>.env</code> files
Why forbidden	Secrets in image layers are readable by anyone who pulls the image (docker history).
Detection	Trivy image scan in CI. Hadolint rule DL3020 / DL3025. Grep Dockerfiles for <code>ARG.*_KEY=\ ARG.*_SECRET=\ ARG.*_TOKEN=.</code>
Corrective action	Inject secrets at runtime via Kubernetes Secrets or Vault agent; never bake into image

8.17 Missing Health Endpoint on Every Service

Field	Description
What it is	A microservice that does not expose /healthz (liveness) and /readyz (readiness) HTTP endpoints
Why forbidden	Kubernetes cannot perform rolling deployments without accurate health signals.
Detection	Integration test: each service's Docker image must respond 200 to GET /healthz within 5 seconds.
Corrective action	Embed helixterm.io/pkg/health handler which registers both endpoints automatically

8.18 Unbounded Goroutine Spawning

Field	Description
What it is	<code>go func()</code> inside a loop without a semaphore, worker pool, or errgroup with a concurrency limit
Why forbidden	Under load, unbounded goroutines exhaust memory and cause OOM kills. Violates §12 host safety.
Detection	Lint: <code>govet</code> + manual review. AST: <code>go func()</code> inside <code>for</code> without accompanying semaphore or errgroup.
Corrective action	Use <code>digital.vasic.concurrency.WorkerPool(n)</code> or <code>golang.org/x/sync/errgroup</code> with <code>SetLimit(n)</code>

8.19 Not Using Structured Logging Fields

Field	Description
What it is	<code>log.Printf("user %s logged in", userID)</code> instead of <code>logger.Info("user logged in", "user_id", userID)</code>

Field	Description
Why forbidden	String-formatted log messages cannot be queried, filtered, or alerted on in log aggregation systems (Loki, Elasticsearch).
Detection	Lint: logrlint. Grep: <code>log.Printf\ log.Println\ fmt.Println</code> in non-test code.
Corrective action	Use <code>log/slog</code> with structured key-value pairs; inject logger via <code>helixterm.io/pkg/logger</code>

8.20 Missing defer for Resource Cleanup

Field	Description
What it is	Opened resources (files, database connections, HTTP response bodies) not closed via <code>defer</code>
Why forbidden	Resource leaks accumulate under load. A missed <code>resp.Body.Close()</code> leaks a socket per request.
Detection	Lint: <code>bodyclose</code> , <code>sqlclosecheck</code> <code>golangci-lint</code> plugins.
Corrective action	<code>defer resp.Body.Close()</code> immediately after checking the error from <code>http.Client.Do</code>

Section 9: Code Review Checklist (Constitution-Mandated)

Authority: Every pull request against `helixterm.io` MUST pass this complete checklist before merge. The checklist is enforced by the GitHub branch protection policy (PR template populated automatically by `.github/pull_request_template.md`). Unchecked items block merge.

Reviewer: At least one CODEOWNER approval required. Self-approval forbidden for changes to `pkg/`, `constitution/`, or `deploy/`.

9.1 Pre-Merge PR Checklist

A. Tests

☐

[TEST-01] All unit tests pass locally: `make test-unit`

☐

[TEST-02] All integration tests pass against real services: `make test-integration`

☐

[TEST-03] All contract tests pass: `make test-contract`

☐

[TEST-04] E2E smoke tests pass on staging: `make test-smoke`

☐

[TEST-05] New code has $\geq 80\%$ line coverage (verified via `make coverage-report`)

☐

[TEST-06] Branch coverage for changed packages is $\geq 70\%$

☐

[TEST-07] No test uses mocks/stubs outside of `*_test.go` unit test files (§11.4.27-A)

☐

[TEST-08] Every new gate is paired with a mutation test that proves the gate catches regression (§1.1)

☐

[TEST-09] Performance benchmarks not regressed: `make bench` delta $\leq 5\%$ on p95 latency

☐

[TEST-10] Security scan clean: `make scan-security` exit 0

B. Code Quality

☐

[QUAL-01] `golangci-lint run ./...` — zero new lint errors

☐

[QUAL-02] `go vet ./...` — zero issues

☐

[QUAL-03] `go build ./...` — zero compilation errors or warnings

☐

[QUAL-04] No new TODO, FIXME, or HACK comments without a linked issue number

☐

[QUAL-05] All exported functions, types, and constants have godoc comments

☐

[QUAL-06] No `interface{}` or `any` used where a concrete type or generic is possible

☐

[QUAL-07] No `time.Sleep` in production code (only in tests with short durations)

☐

[QUAL-08] All `context.Context` parameters are the first argument in every function signature

☐

[QUAL-09] All errors are wrapped with `fmt.Errorf("fn name: %w", err)` — no bare `return err`

☐

[QUAL-10] No `panic()` in library code (`helixterm.io/pkg/*`)

C. Constitution Compliance

☐

[CONST-01] `make constitution-check` passes (`scripts/constitution-check.go` `exit 0`)

☐

[CONST-02] Package naming follows §2 conventions (module path, Kafka topics, DB names)

☐

[CONST-03] No submodule functionality duplicated (§11.4.74 catalogue-first check)

☐

[CONST-04] `AGENTS.MD` consulted; nothing in this PR contradicts its rules

☐

[CONST-05] `helix-deps.yaml` updated if new vasic-digital/HelixDevelopment dependency added

☐

[CONST-06] All new scripts have documentation block + `docs/scripts/<name>.md` (§11.4.18)

☐

[CONST-07] `.gitignore` updated if new build artifact types are introduced (§11.4.30)

☐

[CONST-08] No secrets, API keys, or tokens in any tracked file (§11.4.10)

☐

[CONST-09] All new directories use lowercase_snake_case naming (§11.4.29)

☐

[CONST-10] Commit messages follow Conventional Commits + Helix extensions (§2.18)

D. Architecture & Design

☐

[ARCH-01] No handler directly accesses the database; all access goes through repository layer

☐

[ARCH-02] No global mutable state introduced (no package-level `var` with pointer/slice/map)

☐

[ARCH-03] All external HTTP/gRPC calls wrapped with circuit breaker (`digital.vasic.recovery`)

☐

[ARCH-04] All network I/O uses a context with deadline (`context.WithTimeout`)

☐

[ARCH-05] All external calls have retry logic with exponential backoff

☐

[ARCH-06] No `init()` function with database connections or goroutine spawning

☐

[ARCH-07] Service-to-service calls go through gRPC, not direct database sharing

☐

[ARCH-08] Health endpoints `/healthz` and `/readyz` present and passing for any new service

☐

[ARCH-09] All container workloads use `vasic-digital/containers` submodule (§11.4.76)

☐

[ARCH-10] All goroutine spawning inside loops uses a bounded worker pool or `errgroup` with limit

E. Security

☐

[SEC-01] No credentials hardcoded anywhere in source code or configuration files

☐

[SEC-02] All SQL queries use parameterized statements (no string concatenation)

☐

[SEC-03] Input validation present for all user-facing API endpoints

☐

[SEC-04] No new direct dependencies introduced without security scan (govulncheck)

☐

[SEC-05] TLS configured with minimum version 1.2 on all HTTPS/gRPC listeners

☐

[SEC-06] Rate limiting present on public-facing endpoints

☐

[SEC-07] JWT validation uses constant-time comparison (`hmac.Equal`)

☐

[SEC-08] Sensitive fields (password, token, secret) excluded from JSON serialisation responses

☐

[SEC-09] No `sudo`, root escalation, or privileged Docker operations (§11.4.161 — rootless mandate)

☐

[SEC-10] Trivy image scan for any new or modified Dockerfile — zero HIGH/CRITICAL CVEs

F. Documentation & Changelog

☐

[DOC-01] `README.md` updated if public API or setup process changed

☐

[DOC-02] `CHANGELOG.md` entry added in correct format (see §11)

☐

[DOC-03] OpenAPI/proto comments updated for any changed endpoints

☐

[DOC-04] Architecture Decision Record (ADR) created if architectural decision made

☐

[DOC-05] `docs/` updated if user-visible behaviour changed

G. Observability

☐

[OBS-01] All new code paths emit structured log lines at appropriate levels

☐

[OBS-02] New service endpoints have Prometheus metrics: request_total, request_duration_seconds, request_errors_total

☐

[OBS-03] Distributed tracing spans created for every inbound request and outbound call

☐

[OBS-04] Alert rules defined in deploy/monitoring/alerts/ for new critical metrics

☐

[OBS-05] No sensitive data (PII, credentials) in log fields

H. Deployment

☐

[DEPL-01] Helm chart values.yaml updated if new environment variables or configurations added

☐

[DEPL-02] Resource requests and limits defined for all containers

☐

[DEPL-03] PodDisruptionBudget defined for any new Deployment with replicas > 1

☐

[DEPL-04] Horizontal Pod Autoscaler configured for stateless services

☐

[DEPL-05] Database migration scripts are idempotent and rollback-safe

9.2 Checklist Enforcement

The checklist is embedded in .github/pull_request_template.md. The constitution-compliance.yml workflow validates that all items are checked before the “Ready for review” label can be applied. Any unchecked required item causes the compliance/checklist status check to fail, blocking merge.

```
# .github/pull_request_template.md excerpt
```

```
## Constitution Compliance Checklist
```

```
Before requesting review, confirm ALL items below are checked.
```

Unchecked required items WILL block merge.
<!-- Items auto-validated by .github/workflows/constitution-compliance.yml -->

Section 10: Repository Structure Compliance

Authority: The Constitution mandates a specific repository structure. Any file or directory that deviates from this structure without explicit documented exception is a constitution violation detectable by `scripts/constitution-check.go --check repo-structure`.

10.1 Complete Required Directory Tree

helixterm/	# Repository root
— AGENTS.MD	# REQUIRED – AI
agent governance (§3)	
— CLAUDE.MD	# REQUIRED – Claude-
specific guidance (§4)	
— QWEN.MD	# REQUIRED – Qwen
Code guidance (§11.4.76)	
— Constitution.md	# REQUIRED – Project
constitution (extends universal)	
— README.md	# REQUIRED – Project
overview + doc links (§11.4.57)	
— CHANGELOG.md	# REQUIRED – Semver
changelog (§11)	
— CONTINUATION.md	# REQUIRED – Agent
continuation state (§12.10)	
— Issues.md	# REQUIRED – Active
work items (§11.4.15)	
— Issues_Summary.md	# REQUIRED – Issue
index (§11.4.56)	
— Fixed.md	# REQUIRED – Closed
items archive (§11.4.19)	
— Fixed_Summary.md	# REQUIRED – Fixed
items index (§11.4.19)	
— Stats.md	# REQUIRED – Build
resource stats (§11.4.24)	
— Makefile	# REQUIRED – All dev
operations	
— go.work	# REQUIRED – Go

```

workspace (multi-module)
├─ go.work.sum                                # REQUIRED –
Workspace checksum
├─ helix-deps.yaml                            # REQUIRED –
Submodule dependency manifest (§11.4.31)
├─ .gitignore                                # REQUIRED –
Root .gitignore (§11.4.30)
├─ .gitmodules                               # REQUIRED –
Submodule declarations
├─ .env.example                              # REQUIRED –
Documented env template (§11.4.30)
|
├─ constitution/                             # REQUIRED –
HelixConstitution submodule
|   ├─ Constitution.md
|   ├─ AGENTS.md
|   ├─ CLAUDE.md
|   ├─ QWEN.md
|   └─ submodules-catalogue.md
|
├─ services/                                # REQUIRED – All 25
microservices
|   └─ auth/                                # helixterm.io/
services/auth
|   |   └─ cmd/
|   |   |   └─ server/
|   |   |       └─ main.go
|   |   └─ internal/
|   |       └─ handler/
|   |           └─ grpc/
|   |               └─ http/
|   |   └─ service/
|   |       └─ repository/
|   |           └─ domain/
|   |   └─ proto/
|   |   └─ tests/
|   |       └─ unit/
|   |           └─ integration/
|   |               └─ contract/
|   |                   └─ bench/
|   |   └─ go.mod                            # helixterm.io/
services/auth
|   |   └─ go.sum
|   |   └─ .gitignore

```

			└─ Dockerfile	
			└─ AGENTS.MD	# Service-local
agent overrides (optional)				
			└─ gateway/	# helixterm.io/
services/gateway				
			└─ session/	# helixterm.io/
services/session				
			└─ user/	# helixterm.io/
services/user				
			└─ billing/	# helixterm.io/
services/billing				
			└─ notification/	# helixterm.io/
services/notification				
			└─ terminal/	# helixterm.io/
services/terminal				
			└─ execution/	# helixterm.io/
services/execution				
			└─ scheduler/	# helixterm.io/
services/scheduler				
			└─ storage/	# helixterm.io/
services/storage				
			└─ config/	# helixterm.io/
services/config				
			└─ audit/	# helixterm.io/
services/audit				
			└─ metrics/	# helixterm.io/
services/metrics				
			└─ search/	# helixterm.io/
services/search				
			└─ webhook/	# helixterm.io/
services/webhook				
			└─ provisioner/	# helixterm.io/
services/provisioner				
			└─ identity/	# helixterm.io/
services/identity				
			└─ policy/	# helixterm.io/
services/policy				
			└─ secrets/	# helixterm.io/
services/secrets				
			└─ registry/	# helixterm.io/
services/registry				
			└─ relay/	# helixterm.io/
services/relay				
			└─ analytics/	# helixterm.io/


```

services/analytics
|   └─ backup/                                # helixterm.io/
services/backup
|   └─ health/                                # helixterm.io/
services/health
|   └─ events/                                # helixterm.io/
services/events
|
└─ pkg/                                        # REQUIRED – Shared
libraries
|   └─ logger/                                # helixterm.io/pkg/
logger
|   |   └─ logger.go
|   |   └─ logger_test.go
|   |   └─ go.mod
|   └─ health/                                # helixterm.io/pkg/
health
|   └─ middleware/                            # helixterm.io/pkg/
middleware
|   └─ tracing/                              # helixterm.io/pkg/
tracing
|   └─ metrics/                              # helixterm.io/pkg/
metrics
|   └─ errors/                              # helixterm.io/pkg/
errors
|   └─ config/                              # helixterm.io/pkg/
config
|   └─ auth/                                # helixterm.io/pkg/
auth
|   └─ proto/                               # helixterm.io/pkg/
proto (shared protobuf)
|   └─ testutil/                            # helixterm.io/pkg/
testutil (test helpers only)
|
└─ client/                                   # Flutter client
|   └─ lib/
|       └─ main.dart
|       └─ src/
|           └─ features/
|           └─ shared/
|           └─ core/
|           └─ generated/
|       └─ test/
|       └─ unit/

```

```

|   |   |─ widget/
|   |   |─ integration/
|   |   └─ golden/
|   └─ android/
|   └─ ios/
|   └─ pubspec.yaml
|   └─ pubspec.lock
|   └─ analysis_options.yaml
|   └─ .gitignore
|
└─ submodules/                                # REQUIRED – vasic-
digital + HelixDevelopment submodules
|   └─ challenges/                            # vasic-digital/
Challenges
|   └─ helix_qa/                             # HelixDevelopment/
helixqa
|   └─ containers/                           # vasic-digital/
containers (§11.4.76)
|   └─ llm_provider/                         # vasic-digital/
LLMProvider
|   └─ observability/                       # vasic-digital/
observability
|   └─ auth/                                # vasic-digital/auth
|   └─ database/                            # vasic-digital/
database
|   └─ messaging/                           # vasic-digital/
Messaging
|   └─ recovery/                             # vasic-digital/
recovery
|   └─ concurrency/                         # vasic-digital/
concurrency
|   └─ config/                              # vasic-digital/
config
|   └─ security/                            # vasic-digital/
security
|   └─ middleware/                          # vasic-digital/
middleware
|   └─ ratelimiter/                         # vasic-digital/
ratelimiter
|   └─ discovery/                           # vasic-digital/
discovery
|
└─ proto/                                    # REQUIRED – All
protobuf definitions

```

```

|   |─ helixterm/
|   |   |─ auth/
|   |   |   └─ v1/
|   |   |       └─ auth.proto
|   |   |─ gateway/
|   |   |─ terminal/
|   |   └─ ... (one dir per service)
|   └─ buf.yaml
|
|─ docs/                                     # REQUIRED – All
documentation
|   |─ 00_overview.md
|   |─ 01_architecture.md
|   |─ 02_services.md
|   |─ 03_api_reference.md
|   |─ 04_deployment.md
|   |─ 05_operations.md
|   |─ 06_security.md
|   |─ 07_development.md
|   |─ 08_testing.md
|   |─ 09_monitoring.md
|   |─ 10_contributing.md
|   |─ 11_constitution_compliance.md       # THIS DOCUMENT
|   |─ adr/                                # Architecture
Decision Records
|   |   |─ 0001_monorepo_structure.md
|   |   |─ 0002_grpc_service_mesh.md
|   |   └─ ...
|   └─ scripts/                            # Per §11.4.18 –
script docs
|       |─ constitution-check.md
|       |─ verify-all-constitution-rules.md
|       └─ ...
|
|─ .github/                                # REQUIRED – GitHub
configuration
|   |─ workflows/
|   |   |─ constitution-compliance.yml     # THIS WORKFLOW (§7)
|   |   |─ ci.yml                         # Main CI pipeline
|   |   |─ release.yml                    # Release pipeline
|   |   |─ security-scan.yml              # Security scanning
|   |   └─ chaos.yml                      # Chaos testing
|   |─ CODEOWNERS                         # REQUIRED
|   └─ pull_request_template.md           # REQUIRED – embeds

```

```

$9 checklist
|   └─ ISSUE_TEMPLATE/
|       └─ bug_report.md
|       └─ feature_request.md
|       └─ constitution_violation.md
|
|─ scripts/                                # REQUIRED – All
operational scripts
|   └─ constitution-check.go                # REQUIRED –
Constitution compliance checker (§7)
|   └─ verify-all-constitution-rules.sh    # REQUIRED – Post-
pull validation (§11.4.32)
|   └─ commit_all.sh                       # REQUIRED – Commit
wrapper (§11.4.22)
|   └─ install_upstreams.sh                # REQUIRED – Multi-
remote setup (§11.4.36)
|   └─ build.sh                            # REQUIRED – Build
wrapper with stats (§11.4.24)
|   └─ test.sh                             # REQUIRED – Full
test suite runner
|   └─ generate-proto.sh                    # REQUIRED –
Protobuf code generation
|   └─ generate-mocks.sh                    # REQUIRED – Mock
generation for unit tests only
|   └─ coverage-report.sh                  # REQUIRED –
Coverage threshold enforcer
|   └─ helix_release.sh                     # REQUIRED – Release
tag script (§11.4.151)
|
|─ deploy/                                # REQUIRED –
Deployment configuration
|   └─ docker-compose.yml                  # Local development
environment
|   └─ docker-compose.test.yml             # Integration test
environment
|   └─ kubernetes/
|       └─ namespaces/
|           └─ helixterm.yaml
|       └─ base/
|           └─ overlays/
|               └─ staging/
|               └─ production/
|   └─ monitoring/
|       └─ dashboards/

```

```

|       |─ alerts/
|       |─ recording_rules/
|
|─ charts/                                # REQUIRED – Helm
charts
|   |─ helixterm/
|   |   |─ Chart.yaml
|   |   |─ values.yaml
|   |   |─ values.staging.yaml
|   |   |─ values.production.yaml
|   |   |─ templates/
|   |       |─ _helpers.tpl
|   |       |─ namespace.yaml
|   |       |─ service-account.yaml
|   |       |─ services/
|   |           |─ auth/
|   |               |─ deployment.yaml
|   |               |─ service.yaml
|   |               |─ hpa.yaml
|   |               |─ pdb.yaml
|   |               └─ ... (one dir per service)
|
|─ tests/                                # REQUIRED – Cross-
service tests
    |─ e2e/                             # E2E tests (real
system)
    |─ chaos/                           # Chaos engineering
scenarios
    |─ perf/                            # k6 performance
scripts
    |─ security/                        # DAST + penetration
test scripts
    |─ smoke/                          # Production smoke
tests

```

10.2 Required Root-Level Files

Every file listed below **MUST** exist at the repository root on every commit to main. Their absence is detected by `scripts/constitution-check.go --check required-files` and blocks the `constitution/check` CI status.

File	Enforced By	Minimum Content Requirement
AGENTS.MD	CM-AGENTS-MD-PRESENT gate	Must contain Base agent rules: constitution/AGENTS.md reference
CLAUDE.MD	CM-CLAUDE-MD-PRESENT gate	Must contain @constitution/ CLAUDE.md pointer
QWEN.MD	CM-QWEN-MD-PRESENT gate	Must reference constitution/ QWEN.md
Constitution.md	CM- CONSTITUTION-PRESENT gate	Must contain extends constitution/Constitution.md declaration
helix-deps.yaml	CM-HELIX-DEPS-PRESENT gate	Must validate against JSON Schema schemas/helix- deps.schema.json
CHANGELOG.md	CM-CHANGELOG-PRESENT gate	Must have at least one version entry
go.work	CM-GO-WORK-PRESENT gate	Must include all 25 services + all pkg modules
Makefile	CM-MAKEFILE-PRESENT gate	Must define targets: test, test- unit, test-integration, build, lint, constitution-check
CONTINUATION.md	CM- CONTINUATION-PRESENT gate	Must exist (may be minimal placeholder)

10.3 Per-Service Required Structure

Each of the 25 services under `services/<name>/` must contain the following:

```

services/<name>/
├─ cmd/server/main.go      # REQUIRED – binary entry point
├─ internal/               # REQUIRED – all unexported
code here
|   └─ handler/            # REQUIRED – HTTP/gRPC handlers
|   └─ service/            # REQUIRED – business logic
|   └─ repository/        # REQUIRED – data access layer
|   └─ domain/            # REQUIRED – domain types +
errors

```

— proto/	# REQUIRED if service exposes
gRPC	
— tests/	# REQUIRED
— unit/	# REQUIRED – ≥80% line coverage
— integration/	# REQUIRED – real
infrastructure	
— contract/	# REQUIRED – Pact contracts
— go.mod	# REQUIRED – module
helixterm.io/services/<name>	
— go.sum	# REQUIRED
— Dockerfile	# REQUIRED
— .gitignore	# REQUIRED

Missing any of the above is a CM-SERVICE-STRUCTURE gate violation.

Section 11: Changelog Convention

Authority: The HelixTerminator changelog follows semantic versioning with Constitution-mandated categories. This section defines the complete format, process, and example entries.

11.1 CHANGELOG.md Format Specification

Changelog

All notable changes to HelixTerminator are documented in this file.

Format: [\[Keep a Changelog\]\(https://keepachangelog.com/en/1.1.0/\)](https://keepachangelog.com/en/1.1.0/)

Versioning: [\[Semantic Versioning\]\(https://semver.org/spec/v2.0.0.html\)](https://semver.org/spec/v2.0.0.html)

Constitution: This changelog follows §11 of the HelixTerminator Constitution.

[Unreleased]

Added

- ...

Changed

- ...

[helixterm-1.2.0] - 2026-07-15

Added

- . . .

Format rules: 1. Top-level heading is always # Changelog 2. Unreleased section is always present (may be empty) 3. Versions listed in reverse chronological order (newest first) 4. Version anchor format: [helixterm-X.Y.Z] — prefixed per §11.4.151 5. Date format: YYYY-MM-DD (ISO 8601) 6. Every version section contains only the defined categories (see §11.3) 7. Items within categories are bullet points beginning with a capital letter 8. Each item references the PR number and issue/ticket number if applicable: ([#123](https://github.com/. . .)), (HT-456)

11.2 Version Numbering — Semver Rules

HelixTerminator follows Semantic Versioning 2.0.0 with the mandatory prefix per §11.4.151:

helixterm-MAJOR.MINOR.PATCH[-PRERELEASE][+BUILD]

Component	Increment When
MAJOR	Breaking change to public API, incompatible gRPC proto changes, breaking service contract, removal of public endpoint, Constitution V-axis major revision
MINOR	New feature added in backward-compatible manner, new service added, new public API endpoint, new Kafka topic
PATCH	Backward-compatible bug fix, dependency security update, documentation update, performance improvement with no API change
PRERELEASE	-dev.N during active development, -rc.N for release candidates

Version code monotonicity (§11.4.151): The integer version code in `deploy/version.go` MUST increment monotonically across all releases. It is computed as $MAJOR * 10000 + MINOR * 100 + PATCH$. No release may have a lower version code than any previous release on `main`.

Release tag format:


```
helixterm-1.0.0      # stable release
helixterm-1.1.0-rc.1 # release candidate
helixterm-1.1.0-dev.3 # development snapshot
```

11.3 Category Definitions

Category	When to Use
Added	New features, new services, new API endpoints, new Kafka topics, new configuration options
Changed	Changes to existing functionality that are backward compatible; performance improvements; dependency upgrades
Deprecated	Features or APIs that will be removed in a future major version; must include migration guidance
Removed	Features or APIs removed in this version; must reference the Deprecated entry and provide migration path
Fixed	Bug fixes; must reference issue number or ticket ID
Security	Security fixes, dependency updates addressing CVEs; must reference CVE ID when applicable
Performance	Performance improvements; must include before/after benchmark numbers
Constitution	Changes driven by HelixConstitution compliance: new gates, rule adoption, submodule additions, naming corrections

Ordering within a version block: 1. Added 2. Changed 3. Deprecated 4. Removed 5. Fixed 6. Security 7. Performance 8. Constitution

11.4 Git Tag Requirements

Every release tag MUST:

1. Follow the helixterm-X.Y.Z prefix format (§11.4.151)
2. Be an annotated tag: `git tag -a helixterm-1.2.0 -m "Release helixterm-1.2.0"`
3. Be pushed to ALL four upstream remotes (GitHub, GitLab, GitFlic, GitVerse) per §11.4.36
4. Have a corresponding GitHub Release with the CHANGELOG.md section as the release notes
5. Be created only after `make test-full` (all test types) exits 0
6. Be created only after the compliance score is 100 (release gate per §12.2)

11.5 Release Process Steps

```
# 1. Ensure all upstreams are current
git fetch --all --tags

# 2. Create release branch
git checkout -b release/helixterm-1.2.0

# 3. Update CHANGELOG.md – move [Unreleased] items to new
    [helixterm-1.2.0] section
# Update helix-deps.yaml versions if needed
# Update version.go version code

# 4. Run full test suite
make test-full

# 5. Run constitution compliance check (must score 100)
make constitution-check
make compliance-score # must output SCORE=100

# 6. Commit release prep
bash scripts/commit_all.sh \
    "chore(release): helixterm-1.2.0

    - Update CHANGELOG.md
    - Bump version code to 10200
    - Constitution compliance score: 100/100"

# 7. Create and push annotated tag
git tag -a helixterm-1.2.0 -m "Release helixterm-1.2.0"
```

```
$(sed -n '/## \[helixterm-1.2.0\]/,/## \[helixterm-1.1/p'
CHANGELOG.md | head -100)"

# Push tag to all remotes
git push github helixterm-1.2.0
git push gitlab helixterm-1.2.0
git push gitflic helixterm-1.2.0
git push gitverse helixterm-1.2.0

# 8. Merge release branch to main
git checkout main
git merge --ff-only release/helixterm-1.2.0
git push github main gitlab main gitflic main gitverse main

# 9. Create GitHub Release
gh release create helixterm-1.2.0 \
  --title "HelixTerminator 1.2.0" \
  --notes-file <(sed -n '/## \[helixterm-1.2.0\]/,/## \[helixterm-1.1/p' CHANGELOG.md)
```

11.6 Full Example CHANGELOG.md

Changelog

All notable changes to HelixTerminator are documented in this file.

Format follows [\[Keep a Changelog\]\(https://keepachangelog.com/en/1.1.0/\)](https://keepachangelog.com/en/1.1.0/).

Versioning follows [\[Semantic Versioning\]\(https://semver.org/spec/v2.0.0.html\)](https://semver.org/spec/v2.0.0.html).

Release tags are prefixed ``helixterm-`` per HelixConstitution §11.4.151.

Constitution: governed by ``constitution/Constitution.md`` (HelixConstitution v2).

[Unreleased]

Added

- ``helixterm.io/services/events`` service: persistent event streaming over WebSocket for client connections

Fixed

- Gateway service: incorrect JWT expiry clock skew tolerance causing premature session invalidation (HT-089)

[helixterm-1.1.0] - 2026-07-15

Added

- ``helixterm.io/services/analytics`` service: real-time usage analytics pipeline with Kafka consumer group (HT-071) ([#142](https://github.com/HelixDevelopment/helixterm/pull/142))
- ``helixterm.io/services/backup`` service: automated cross-region backup scheduler with S3-compatible storage (HT-072) ([#144](https://github.com/HelixDevelopment/helixterm/pull/144))
- Kafka topic ``helix.terminator.analytics.session_recorded`` with schema registry integration (HT-073)
- Flutter client: analytics dashboard screen with dark/light theme support per OpenDesign tokens (io.helixterm.client.analytics) (HT-074)
- k6 performance scripts for analytics service: ``tests/perf/analytics_ingest.js`` (HT-075)

Changed

- ``helixterm.io/services/auth``: JWT signing upgraded from HS256 to EdDSA (Ed25519), per ``CANONICAL_FACTS.md`` CD-7; existing tokens remain valid until natural expiry (HT-076) ([#145](https://github.com/HelixDevelopment/helixterm/pull/145))
- ``helixterm.io/pkg/logger``: structured log fields now include ``trace_id`` and ``span_id`` automatically from context (HT-077)
- Go workspace updated to Go 1.25.2 (HT-078)
- All Helm chart resource requests updated to reflect measured baseline usage from Stats.md (HT-079)

Fixed

- ``helixterm.io/services/terminal``: goroutine leak when WebSocket client disconnected without closing frame (HT-080) ([#147](https://github.com/HelixDevelopment/helixterm/pull/147))
- ``helixterm.io/services/billing``: race condition in subscription renewal when concurrent webhook and cron job both triggered (HT-081)
- Flutter client: session list not refreshed after reconnect on iOS (HT-082) ([#148](https://github.com/HelixDevelopment/helixterm/pull/148))

Security

- Updated ``google.golang.org/grpc`` to v1.68.0 addressing CVE-2026-XXXX (GHSA-xxxx-yyyy-zzzz) (HT-083)
- Added rate limiting to ``/api/v1/auth/login``: 10 req/min per IP with exponential backoff response (HT-084)
- Enabled TLS 1.3 as minimum version on all gRPC listeners (HT-085)

Performance

- ``helixterm.io/services/execution``: p95 latency reduced from 145ms to 62ms by introducing connection pool to execution workers (HT-086). Benchmark: ``go test -bench=BenchmarkExecute -benchtime=30s``
- ``helixterm.io/services/search``: Elasticsearch query optimised; p99 reduced from 820ms to 310ms (HT-087)

Constitution

- Added ``helix-deps.yaml`` v1.1 with analytics and backup service submodule dependencies (§11.4.31) (HT-088)
- Constitution compliance score: 100/100 (was 97/100 due to missing mutation tests on HT-075 gates; remediated)
- Adopted §11.4.170 rendered-UI visual proof: all Flutter screens now have Roborazzi golden tests (HT-089)

[helixterm-1.0.0] - 2026-06-28

Added

- Initial release of HelixTerminator with 25 microservices
- Core services: auth, gateway, session, user, billing, notification, terminal, execution, scheduler, storage, config, audit, metrics, search, webhook, provisioner, identity, policy, secrets, registry, relay, analytics, backup, health, events
- Flutter client ``io.helixterm.client`` with light/dark theme support
- gRPC service mesh with mTLS between all services
- Kafka event streaming: 47 topics across all service domains
- Kubernetes Helm charts for staging and production environments
- Constitution compliance infrastructure: AGENTS.MD, CLAUDE.MD, QWEN.MD, helix-deps.yaml
- CI/CD pipelines: ci.yml, constitution-compliance.yml, release.yml, security-scan.yml
- Full test coverage: unit (≥80%), integration, contract (Pact), E2E, performance (k6), chaos, security
- Prometheus + Grafana + Loki observability stack
- OpenDesign token-based UI for Flutter client
- Roborazzi golden tests for all Flutter screens (§11.4.170)

Constitution

- HelixConstitution v2 adopted at submodule path ``constitution/``
- Pinned to ``helixterm-1.0.0`` tag
- Initial compliance score: 100/100
- All 25 services: vasic-digital submodule catalogue consulted, no duplications found

- Constitution compliance CI gate active: blocks merge below 95/100
- Release gate active: blocks tag below 100/100

```
[Unreleased]: https://github.com/HelixDevelopment/helixterm/compare/helixterm-1.1.0...HEAD
[helixterm-1.1.0]: https://github.com/HelixDevelopment/helixterm/compare/helixterm-1.0.0...helixterm-1.1.0
[helixterm-1.0.0]: https://github.com/HelixDevelopment/helixterm/releases/tag/helixterm-1.0.0
```

Section 12: Compliance Scoring and Monitoring

Authority: Constitution compliance is not a binary pass/fail — it is a scored metric tracked over time. This section defines the scoring algorithm, enforcement thresholds, dashboard design, and review cadence.

12.1 Score Calculation Algorithm

The compliance score is an integer from 0 to 100, calculated by `scripts/constitution-check.go --score`. It is a weighted sum across 8 categories:

$$\text{score} = \sum (\text{category_weight} \times \text{category_score}) / 100$$

Where `category_score` for each category is: $(\text{passed_checks} / \text{total_checks}) \times 100$.

Category	Weight	Checks Included	Max Points
Test Coverage	25	Unit ≥80%, branch ≥70%, contract tests present, mutation tests present, all test types present	25
Code Quality	20	golangci-lint clean, go vet clean, no forbidden patterns, error wrapping, context propagation	20

Category	Weight	Checks Included	Max Points
Constitution Gates	20	AGENTS.MD present+valid, CLAUDE.MD present+valid, helix- deps.yaml valid, repo structure compliant, naming conventions	20
Security	15	No hardcoded secrets, Trivy scan clean, govulncheck clean, TLS config correct, input validation	15
Documentation	10	README.md current, CHANGELOG.md entry per release, all scripts documented, ADRs for architectural decisions	10
Submodule Hygiene	5	No duplicated submodule functionality, helix- deps.yaml up-to-date, all submodules at pinned ref	5
CI/CD Gates	3	All required workflows present, all status checks enabled, branch protection configured	3
Observability	2	Health endpoints present, structured logging, Prometheus metrics, distributed tracing	2
TOTAL	100		100

Detailed Check Catalogue

The following table enumerates every individual check, its category, and its weight within that category:

Category: Test Coverage (weight 25)

- TC-01 unit test files present for all services
weight 3
- TC-02 overall line coverage $\geq 80\%$
weight 4
- TC-03 overall branch coverage $\geq 70\%$
weight 3
- TC-04 Pact contract tests present for all inter-service edges
weight 3
- TC-05 mutation tests present (go-mutesting or equivalent)
weight 2
- TC-06 integration tests present for all services
weight 3
- TC-07 E2E tests present in tests/e2e/
weight 2
- TC-08 performance tests present in tests/perf/ (k6)
weight 2
- TC-09 security tests present in tests/security/
weight 2
- TC-10 chaos tests present in tests/chaos/
weight 1

Category: Code Quality (weight 20)

- CQ-01 golangci-lint reports zero issues
weight 4
- CQ-02 go vet reports zero issues
weight 3
- CQ-03 no bare `return err` (wrapcheck)
weight 2
- CQ-04 no global mutable state (package-level vars)
weight 2
- CQ-05 context propagated in all function calls
weight 2
- CQ-06 no panic in library code
weight 2
- CQ-07 no init() side effects
weight 2
- CQ-08 no mocks in non-unit tests
weight 3

Category: Constitution Gates (weight 20)

- CG-01 AGENTS.MD present at root
weight 2
- CG-02 CLAUDE.MD present at root

weight 2

CG-03 QWEN.MD present at root

weight 1

CG-04 Constitution.md present at root

weight 2

CG-05 helix-deps.yaml present and schema-valid

weight 3

CG-06 repo directory structure matches \$10 spec

weight 2

CG-07 go.work includes all 25 services

weight 2

CG-08 all service module paths follow helixterm.io/services/

weight 2

CG-09 all directory names lowercase_snake_case

weight 2

CG-10 commit message format compliance on last 20 commits

weight 2

Category: Security (weight 15)

SE-01 no credentials in any tracked file

weight 4

SE-02 Trivy image scan: 0 HIGH/CRITICAL CVEs

weight 3

SE-03 govulncheck: 0 vulnerabilities

weight 2

SE-04 all gRPC listeners have TLS $\geq$ 1.2

weight 2

SE-05 all SQL queries parameterized (no concatenation)

weight 2

SE-06 .env not in tracked files; .env.example present

weight 2

Category: Documentation (weight 10)

DC-01 README.md present with doc links section

weight 2

DC-02 CHANGELOG.md present with $\geq$ 1 version entry

weight 2

DC-03 all scripts have docs block + docs/scripts/*.md

weight 2

DC-04 all exported Go functions have godoc comments

weight 2

DC-05 ADR present for each significant architectural decision

weight 2

Category: Submodule Hygiene (weight 5)

SM-01 no duplicated vasic-digital functionality

weight 2

SM-02 helix-deps.yaml matches actual .gitmodules entries

weight 1

SM-03 all submodules at pinned commit (not floating branch)

weight 1

SM-04 install_upstreams.sh present and executable

weight 1

Category: CI/CD Gates (weight 3)

CI-01 constitution-compliance.yml workflow present

weight 1

CI-02 ci.yml workflow present

weight 1

CI-03 branch protection on main (no direct push)

weight 1

Category: Observability (weight 2)

OB-01 all services have /healthz and /readyz endpoints

weight 1

OB-02 all services have Prometheus /metrics endpoint

weight 1

12.2 Enforcement Thresholds

Gate	Threshold	Action on Failure
PR merge gate	Score $\geq$ 95	GitHub constitution/score status check FAILS; merge blocked
Release gate	Score = 100	scripts/helix_release.sh aborts tag creation; release blocked
Warning threshold	Score 90–94	PR is not blocked but a warning comment is posted; reviewer must acknowledge
Critical alert	Score < 90	Immediate Slack alert to #helixterm-ops; on-call paged; no other PRs may merge until restored

12.3 Score Calculation Implementation

```
// scripts/compliance-score.go
// Usage: go run scripts/compliance-score.go --output json
// Returns exit code 0 if score >= threshold (default 95), exit 1
// otherwise.
```

```
package main
```

```
import (
    "encoding/json"
    "flag"
    "fmt"
    "os"
)
```

```
type CategoryScore struct {
    Name          string `json:"name"`
    Weight        int    `json:"weight"`
    Passed        int    `json:"passed"`
    Total         int    `json:"total"`
    Score         float64 `json:"score"`
    WeightedPts   float64 `json:"weighted_points"`
}
```

```
type ComplianceReport struct {
    Timestamp  string `json:"timestamp"`
    Commit     string `json:"commit"`
    Score      int    `json:"score"`
    Categories []CategoryScore `json:"categories"`
    Failures   []string `json:"failures"`
    Passed     bool    `json:"passed"`
    Threshold  int     `json:"threshold"`
}
```

```
func main() {
    threshold := flag.Int("threshold", 95, "Minimum passing score
        (0-100)")
    outputFmt := flag.String("output", "text", "Output format:
        text|json")
    flag.Parse()

    report := runAllChecks()
    report.Threshold = *threshold
    report.Passed = report.Score >= *threshold
}
```

```

    if *outputFmt == "json" {
        enc := json.NewEncoder(os.Stdout)
        enc.SetIndent("", " ")
        enc.Encode(report)
    } else {
        printTextReport(report)
    }

    if !report.Passed {
        os.Exit(1)
    }
}

func printTextReport(r ComplianceReport) {
    fmt.Printf("HelixTerminator Constitution Compliance Score\n")
    fmt.Printf("Commit: %s | Timestamp: %s\n", r.Commit,
        r.Timestamp)
    fmt.Printf("—————\n")
    for _, c := range r.Categories {
        bar := progressBar(int(c.Score), 20)
        fmt.Printf("%-22s [%s] %5.1f%% (%d/%d) → %5.1f/%d pts\n",
            c.Name, bar, c.Score, c.Passed, c.Total,
            c.WeightedPts, c.Weight)
    }
    fmt.Printf("—————\n")
    fmt.Printf("TOTAL SCORE: %d/100 (threshold: %d) → %s\n",
        r.Score, r.Threshold, passLabel(r.Passed))

    if len(r.Failures) > 0 {
        fmt.Printf("\nFailures:\n")
        for _, f := range r.Failures {
            fmt.Printf("  x %s\n", f)
        }
    }
}

func progressBar(pct, width int) string {
    filled := pct * width / 100
    bar := ""
    for i := 0; i < width; i++ {
        if i < filled {
            bar += "█"
        } else {
            bar += "░"
        }
    }
}

```

```

    }
}
return bar
}

func passLabel(passed bool) string {
    if passed {
        return "✓ PASS"
    }
    return "✗ FAIL"
}

```

12.4 Compliance Dashboard

The compliance dashboard is served at <https://helixterm.internal/compliance> and is generated by the analytics service reading from the compliance score history stored in `helixterm_analytics_db.compliance_scores`.

Dashboard panels:

Panel	Description	Refresh
Current Score	Large gauge: current main branch score with trend arrow	On every merge to main
Score History (30 days)	Line chart: daily score with threshold lines at 95 and 100	Daily
Category Breakdown	Radar/spider chart: score per category	On every merge
Failures by Category	Bar chart: count of failing checks per category	On every merge
PR Score Distribution	Histogram: score distribution across recent PRs	Weekly
Trend Forecast	Linear regression: projected score for next 30 days	Weekly

Grafana dashboard JSON: Located at `deploy/monitoring/dashboards/constitution-compliance.json`.

12.5 Alerting Rules

```
# deploy/monitoring/alerts/constitution_compliance.yaml
groups:
- name: constitution_compliance
  interval: 5m
  rules:
    - alert: ConstitutionScoreCritical
      expr: helixterm_constitution_compliance_score < 90
      for: 5m
      labels:
        severity: critical
        team: helixterm
      annotations:
        summary: "HelixTerminator constitution compliance score critically low"
        description: "Compliance score is {{ $value }}/100 (threshold: 90). Immediate action required. No PRs may merge until score is restored above 95."
        runbook: "https://docs.helixterm.internal/runbooks/constitution-compliance-recovery"

    - alert: ConstitutionScoreWarning
      expr: helixterm_constitution_compliance_score >= 90 and helixterm_constitution_compliance_score < 95
      for: 30m
      labels:
        severity: warning
        team: helixterm
      annotations:
        summary: "HelixTerminator constitution compliance score below PR merge threshold"
        description: "Compliance score is {{ $value }}/100 (threshold: 95). PRs are blocked until score recovers."

    - alert: ConstitutionScoreReleaseLocked
      expr: helixterm_constitution_compliance_score < 100 and helixterm_release_in_progress == 1
      for: 1m
      labels:
        severity: warning
        team: helixterm
      annotations:
        summary: "Release blocked by constitution compliance score"
        description: "A release is in progress but constitution score is {{ $value }}/100. Release requires 100/100."
```

12.6 Monthly Compliance Review Process

The constitution compliance review is a monthly synchronous meeting (first Tuesday of each month) attended by all active contributors and the project lead.

Agenda (60 minutes):

1. **Score review (10 min):** Review the 30-day score trend chart. Any month-over-month decrease requires root cause analysis.
2. **Failures review (15 min):** Walk through every check that failed at any point in the past month. Categorise: one-off regression (fixed), systemic issue (requires epic), known acceptable deviation (requires documented exception).
3. **Constitution updates (10 min):** Review HelixConstitution repo for new clauses added since last review. Assess applicability to HelixTerminator. Create tickets for required adoptions.
4. **Anti-pattern review (10 min):** Review `git log --grep="constitution-violation"` from the past month. Any recurring pattern → add a new check or lint rule.
5. **Roadmap (10 min):** Review constitution-related items on the product roadmap. Update `docs/11_constitution_compliance.md` if rules change.
6. **Action items (5 min):** Assign tickets for all identified issues. All compliance tickets use label `constitution` and are highest priority (above feature work).

Output: Meeting notes in `docs/compliance-reviews/YYYY-MM.md`. Score history entry in `docs/compliance-reviews/score_history.csv`.

12.7 Compliance Score Prometheus Metric

```
// pkg/metrics/compliance.go
// Provides the Prometheus gauge published by the health service.
```

```
package metrics
```

```
import "github.com/prometheus/client_golang/prometheus"
```

```
var ConstitutionComplianceScore =
    prometheus.NewGauge(prometheus.GaugeOpts{
```

```

Namespace: "helixterm",
Subsystem: "constitution",
Name:      "compliance_score",
Help:      "Current HelixTerminator constitution compliance
            score (0-100). " +
            "Updated on every merge to main. " +
            "PR merge requires >=95. Release requires 100.",
}))

func init() {
    prometheus.MustRegister(ConstitutionComplianceScore)
}

```

The score is published by the health service which runs `scripts/compliance-score.go --output json` on startup and every 5 minutes, setting the gauge to the returned score.

12.8 Score Recovery Runbook

When the score drops below threshold, follow this runbook:

```

# Constitution Score Recovery Runbook

## Trigger
- Score < 95: PR merge blocked
- Score < 90: Critical alert; on-call paged

## Step 1: Identify failing checks (< 5 minutes)
```bash
go run scripts/constitution-check.go --output json | jq
 '.failures[]'

```

## Step 2: Categorise failures

For each failure:

- Test coverage drop → identify which service/package lost coverage; write missing tests
- New lint error → fix lint error or update lint config with documented exception
- Missing file → create required file from template
- Naming violation → rename with atomic reference update
- Security finding → rotate/remove credential or fix CVE dependency



## Step 3: Create remediation PRs

Each failure category → one focused PR. PRs labelled constitution-remediation get expedited review (within 4 hours).

## Step 4: Verify score recovery

After PRs merge:

```
go run scripts/compliance-score.go --threshold 95
Must exit 0
```

## Step 5: Post-recovery report

Post a message to #helixterm-ops: “Constitution compliance score restored to [SCORE]/100. Root cause: [description]. PRs: [links]. Prevention: [what lint/gate prevents recurrence].”

### 12.9 Compliance History Record

Every merge to `main` appends one row to `docs/compliance-reviews/score\_history.csv`:

```
```csv
timestamp,commit_sha,branch,score,test_coverage,code_quality,const_
2026-06-28T18:00:00Z,abc1234def,main,100,25,20,20,15,10,5,3,2,0
2026-07-01T14:30:00Z,bcd2345ef0,main,98,25,20,18,15,10,5,3,2,2
```

The CSV is generated by `scripts/compliance-score.go --format csv --append docs/compliance-reviews/score_history.csv` called from the `constitution-compliance.yml` workflow post-merge.

Appendix A: Quick Reference

A.1 All Constitution Rule References Used in This Document

Rule	Title	HelixTerminator Impact
§1		≥80% line, ≥70% branch

Rule	Title	HelixTerminator Impact
	Test coverage for every change	
§1.1	Mutation-paired gates	Every gate has a mutation test
§2.1	Multi-upstream push	Push to all 4 remotes on release
§9	Data & host safety	Hardlinked backup before destructive ops
§11.4	Anti-bluff covenant	Every PASS carries runtime evidence
§11.4.2	Recorded-evidence	E2E and UI tests require captured recording
§11.4.4	Test-interrupt-on-discovery	Stop cycle on defect discovery
§11.4.6	No-guessing mandate	Forbidden vocabulary enforced in PR reviews
§11.4.10	Credentials handling	.env gitignored; pre-store leak audit
§11.4.15	Issues lifecycle	7-value Status; Issues.md+Summary+Fixed in sync
§11.4.16	Item-type tracking	Bug/Feature/Task; Type column in Issues_Summary
§11.4.17	Universal-vs-project classification	Every new rule classified
§11.4.18	Script documentation	In-source block + docs/scripts/*.md
§11.4.19	Fixed-document column alignment	Fixed.md mirrors Issues.md columns
§11.4.20	Subagent-driven-by-default	≥3 phases → subagent delegation
§11.4.22	Document-sync commit discipline	Lightweight doc-only commit wrapper
§11.4.24	Build-resource stats tracking	Stats.md per build
§11.4.25		

Rule	Title	HelixTerminator Impact
	Full-Automation-Coverage	Every feature covered by all test types
§11.4.26	Constitution-Submodule Update Workflow	Fetch+pull before editing constitution
§11.4.27	No-fakes-beyond-unit-tests	Mocks only in *_test.go unit files
§11.4.28	Submodules-as-equal-codebase	All owned submodules receive equal engineering attention
§11.4.29	Lowercase-snake_case-naming	All dirs/files/submodules
§11.4.30	.gitignore + no build artifacts	Every module has .gitignore; no tracked build artifacts
§11.4.31	Submodule-dependency-manifest	helix-deps.yaml in every owned submodule
§11.4.32	Post-constitution-pull validation	verify-all-constitution-rules.sh after every pull
§11.4.36	Mandatory install_upstreams	Run after clone/add
§11.4.40	Full-suite retest before release tag	make test-full must pass
§11.4.44	Document revision header	Spec docs carry V.Revision header
§11.4.57	README doc-link section	README has links to all docs
§11.4.73	Spec-versioning	Two-axis: V (primary) + Revision (secondary)
§11.4.74	Submodule-catalogue-first discovery	Check catalogue before scaffolding
§11.4.75	Mechanical enforcement	5-layer git-hook discipline
§11.4.76	Containers-submodule mandate	vasic-digital/containers for all containerised workloads

Rule	Title	HelixTerminator Impact
§11.4.77	Regeneration-mechanism-required	Every generated file has documented regen mechanism
§11.4.78	CodeGraph mandate	npm @colbymchenry/codegraph for code intelligence
§11.4.151	Project-prefixed release-tag	helixterm-X.Y.Z format
§11.4.161	Rootless container runtime	Podman rootless; no sudo/rootful Docker
§11.4.162	OpenDesign UI mandate	Flutter client uses OpenDesign tokens
§11.4.170	Rendered-UI visual-proof mandate	Roborazzi golden tests for all Flutter screens
§12	Host session safety	Bounded execution; ≤60% RAM
§12.10	Continuation document	CONTINUATION.md updated every non-trivial state change

A.2 Services Quick Reference — See Canonical Registry (CD-3)

This table previously enumerated its own divergent 25-service list — a THIRD list, disagreeing with both §2.11's (now-removed) list and doc-01's canonical registry — and used the forbidden :latest image tag (§8 forbidden pattern) throughout. Per CANONICAL_FACTS.md CD-3, the roster and its per-service Docker image / DB / Kafka-topic values are owned by 01_core_architecture.md's canonical service registry; do not re-enumerate them here.

Applying this document's naming convention (§2.11) to any service in doc-01's registry:

Field	Pattern
Module Path	helixterm.io/services/<service-name>
Docker Image	ghcr.io/helixdevelopment/helixterm-<service-

Field	Pattern
	name>:<version> — an explicit pinned <version>, e.g. 1.0.0; never :latest
DB Name	helixterm_<service_snake>_db
Primary Kafka Topics	helix.terminator.<domain>.<event>

Appendix B: Makefile Reference

The root Makefile is the primary developer interface. All operations MUST be accessible through Make targets. Direct script invocation is permitted but the Makefile targets are the canonical interface.

```
# Makefile — HelixTerminator
# Constitution §11.4.18: all Make targets have documentation
# Usage: make <target>

.DEFAULT_GOAL := help
SHELL          := /bin/bash
GO             := go
GOFLAGS        := -trimpath
SERVICES       := auth gateway session user billing notification
                terminal execution \
                    scheduler storage config audit metrics search
                webhook provisioner \
                    identity policy secrets registry relay analytics
                backup health events

.PHONY: help
help: ## Show this help message
    @awk 'BEGIN {FS = ":.*##"} /^[a-zA-Z_-]+:.*##/ {printf "
        %-30s %s\n", $$1, $$2}' $(MAKEFILE_LIST)

# — Build

.PHONY: build
build: ## Build all 25 services
    @bash scripts/build.sh

.PHONY: build-service
build-service: ## Build a single service: make build-service
                SERVICE=auth
    $(GO) build $(GOFLAGS) ./services/$(SERVICE)/cmd/server/
```

```
.PHONY: build-docker
build-docker: ## Build all Docker images
    @for s in $(SERVICES); do \
        docker build -t ghcr.io/helixdevelopment/helixterm-$$s:dev \
            -f services/$$s/Dockerfile . ; \
    done
```

```
# — Testing
```

```
.PHONY: test
test: test-unit test-integration ## Run unit + integration tests
    (standard CI)
```

```
.PHONY: test-unit
test-unit: ## Run all unit tests with coverage
    $(GO) test -race -count=1 -covermode=atomic \
        -coverprofile=coverage.out ./services/.../unit/... ./
        pkg/.../...
    $(GO) tool cover -html=coverage.out -o coverage.html
```

```
.PHONY: test-integration
test-integration: ## Run integration tests (requires Docker)
    $(GO) test -race -count=1 -tags=integration -timeout=10m \
        ./services/.../integration/...
```

```
.PHONY: test-contract
test-contract: ## Run Pact contract tests
    $(GO) test -race -count=1 -tags=contract ./services/.../
        contract/...
```

```
.PHONY: test-e2e
test-e2e: ## Run E2E tests against staging environment
    $(GO) test -race -count=1 -tags=e2e -timeout=30m ./tests/
        e2e/...
```

```
.PHONY: test-perf
test-perf: ## Run k6 performance tests
    @for script in tests/perf/*.js; do k6 run $$script; done
```

```
.PHONY: test-security
test-security: ## Run security tests (SAST + DAST)
    govulncheck ./...
    gosec -fmt=json -out=security-report.json ./...
```

```
.PHONY: test-chaos
test-chaos: ## Run chaos engineering scenarios
    $(GO) test -race -count=1 -tags=chaos -timeout=60m ./tests/chaos/...

.PHONY: test-smoke
test-smoke: ## Run smoke tests against running environment
    $(GO) test -race -count=1 -tags=smoke ./tests/smoke/...

.PHONY: test-full
test-full: test test-contract test-e2e test-perf test-security
    test-chaos ## Run ALL test types (required before release)

.PHONY: test-flutter
test-flutter: ## Run Flutter client tests
    cd client && flutter test
```

— Coverage

```
.PHONY: coverage-report
coverage-report: test-unit ## Generate and display coverage report
    @bash scripts/coverage-report.sh

.PHONY: coverage-check
coverage-check:
    ## Fail if coverage below threshold (80% line, 70% branch)
    @$(GO) test -covermode=atomic -coverprofile=coverage.out ./...
    @$(GO) tool cover -func=coverage.out | \
        awk '/total:/{if ($$3+0 < 80) {print "FAIL: line coverage "
            $$3 " < 80%"; exit 1}}'
```

— Lint & Vet

```
.PHONY: lint
lint: ## Run golangci-lint
    golangci-lint run ./...

.PHONY: vet
vet: ## Run go vet
    $(GO) vet ./...

.PHONY: fmt
fmt: ## Format all Go code
    gofmt -w -s .
```

```
goimports -w .
```

— Constitution

```
.PHONY: constitution-check
```

```
constitution-check: ## Run constitution compliance checker  
$(GO) run scripts/constitution-check.go
```

```
.PHONY: compliance-score
```

```
compliance-score: ## Calculate and display compliance score  
$(GO) run scripts/compliance-score.go
```

```
.PHONY: compliance-score-json
```

```
compliance-score-json: ## Output compliance score as JSON  
$(GO) run scripts/compliance-score.go --output json
```

```
.PHONY: verify-constitution
```

```
verify-constitution: ## Run post-constitution-pull validation  
($11.4.32)  
@bash scripts/verify-all-constitution-rules.sh
```

— Security

```
.PHONY: scan-security
```

```
scan-security: ## Run full security scan (govulncheck + gosec +  
trivy)  
govulncheck ./...  
gosec ./...  
@for s in $(SERVICES); do trivy image ghcr.io/  
helixdevelopment/helixterm-$$s:dev; done
```

```
.PHONY: scan-secrets
```

```
scan-secrets: ## Scan for accidentally committed secrets  
gitleaks detect --source . --verbose
```

— Proto

```
.PHONY: proto
```

```
proto: ## Generate protobuf code  
@bash scripts/generate-proto.sh
```

— Docker & Deploy

```

.PHONY: up
up: ## Start local development environment
    docker compose -f deploy/docker-compose.yml up -d

.PHONY: down
down: ## Stop local development environment
    docker compose -f deploy/docker-compose.yml down

.PHONY: up-test
up-test: ## Start test infrastructure
    docker compose -f deploy/docker-compose.test.yml up -d

.PHONY: down-test
down-test: ## Stop test infrastructure
    docker compose -f deploy/docker-compose.test.yml down

# — Release

.PHONY: release
release: ## Create a new release (usage: make release
    VERSION=1.2.0)
    @bash scripts/helix_release.sh $(VERSION)

```

Appendix C: Environment Variables Reference

All environment variables are documented in `.env.example`. No variable may be used in code without appearing in `.env.example` with a description.

```

# .env.example – HelixTerminator environment variables
# DO NOT commit the actual .env file. This file documents all
#   variables.
# Copy to .env and fill in values before running locally.

# — Service Discovery

```

HELIXTERM_GATEWAY_PORT=8080	# HTTP port for the gateway
service	
HELIXTERM_GRPC_PORT=9090	# gRPC base port (each
service: 9090+N)	
HELIXTERM_ENVIRONMENT=development	# development staging
production	

— Database

```
HELIXTERM_DB_HOST=localhost
HELIXTERM_DB_PORT=5432
HELIXTERM_DB_USER=helixterm
HELIXTERM_DB_PASSWORD=          # REQUIRED – set in secrets
                                manager in prod
HELIXTERM_AUTH_DB_NAME=helixterm_auth_db
HELIXTERM_USER_DB_NAME=helixterm_user_db
# ... (one per service that has a database)
```

— Kafka

```
HELIXTERM_KAFKA_BROKERS=localhost:9092
HELIXTERM_KAFKA_SCHEMA_REGISTRY_URL=http://localhost:8081
HELIXTERM_KAFKA_CONSUMER_GROUP=helixterm-main
```

— Auth

```
HELIXTERM_JWT_PUBLIC_KEY_PATH=    # REQUIRED – path to EdDSA
                                   (Ed25519) public key (CD-7)
HELIXTERM_JWT_PRIVATE_KEY_PATH=   # REQUIRED – path to EdDSA
                                   (Ed25519) private key; NEVER commit
HELIXTERM_JWT_EXPIRY_SECONDS=3600
```

— Observability

```
HELIXTERM_OTEL_ENDPOINT=http://localhost:4317
HELIXTERM_PROMETHEUS_PORT=9100
HELIXTERM_LOG_LEVEL=info          # debug|info|warn|error
```

— Release

```
HELIX_RELEASE_PREFIX=helixterm    # Mandatory per §11.4.151
```

— Secrets Service

```
HELIXTERM_VAULT_ADDR=http://localhost:8200
HELIXTERM_VAULT_TOKEN=            # REQUIRED in dev; use
                                   Kubernetes SA in prod
```

— Flutter Client

```
HELIXTERM_API_BASE_URL=http://localhost:8080
HELIXTERM_WS_BASE_URL=ws://localhost:8080
```

This document is the authoritative governance specification for HelixTerminator constitution compliance. It is governed by the HelixConstitution (github.com/HelixDevelopment/HelixConstitution) and must be kept in sync with any updates to the universal constitution. Any conflict between this document and constitution/Constitution.md is resolved in favour of constitution/Constitution.md unless an explicit override is declared with an Override §X.Y section.

Last reviewed: 2026-06-28 **Compliance score at time of writing:** 100/100 (initial specification) **Next scheduled review:** 2026-07-29 (monthly cadence per §12.6)